



全国智能车竞赛 ◆ 室外光电组

机器人操作系统ROS原理与应用



北京钢铁侠科技有限公司

前言

机器人是集机械、电子、自动化、计算机、传感器、人工智能等多学科及前沿技术于一身的可编程设备，是国家制造技术能力发展的制高点。目前，工业机器人的发展已经趋于标准化、模块化、功能也越来越强大，可以说，工业机器人自第一次工业革命起到现在已经进入了稳定平滑的发展时期。近年来，随着科学技术的发展，特别是人工智能技术的发展，以家用服务、医疗服务等专业服务类为代表的智能机器人呈现出欣欣向荣的发展态势。

智能机器人具备的自主感、运动规划、动作控制、协同能力不仅取决于硬件平台的配置，也与其软件平台的功能和性能密切相关。作为开源的机器人操作系统（**Robot**

Operating System，简称 **ROS**）不仅提供了智能机器人所需的环境感知、运动控制、可视化操作等运行支撑功能，而且也支持智能机器人的应用软件和系统的功能验证、虚拟仿真等开发活动，到2019年ROS已经走过了11年的发展，因其分布式架构、多语言支持、易于扩展等特点，逐渐已经成为智能机器人开发的标准平台。在智能服务机器人、无人驾驶、工业机器人等领域都取得了良好的应用。

在智能服务机器人领域，定位与导航是一项核心技术，通过多传感器数据融合结合智能控制技术制定行之有效的路径规划方法可以让机器人更加智能化，而ROS提供了常用的**SLAM**技术，使得服务机器人这一核心功能的开发变的十分简单。

提到无人驾驶就不得不提到百度的Apollo平台，其背后就隐藏着ROS的身影，自动驾驶系统非常复杂，包含感知、障碍物检测、决策、车辆控制等模块，把这么多功能各异的模块集成在一起，组成一个完整的系统并完成自动驾驶的任务，这是一个非常大挑战。百度通过对ROS的集成与修改实现了ROS在无人驾驶领域的适配，并成功的造就了Apollo平台的诞生。详细内容可参考<https://github.com/ApolloAuto/apollo-platform>。

就工业机器人来讲，ROS有一个单独的分支，叫做ROS-Industrial（ROS-I）。ROS作为逐渐流行的软件框架，当然不会放过工业机器人这个庞大而成熟的市场，所以ROS-I出现的目的就是希望将ROS赋能于工业机器人，在解决兼容性问题的基础上，拓展更多垂直行业的典型应用。ROS-I针对工业机器人定义了一套通用协议—**simple_message**，已经开始对各大品牌工业机器人的支持。

说了这么多，那如何行之有效的学习ROS呢？

首先，ROS1.0是安装运行在Ubuntu系统之上的，所以第一需要初学者对Linux和Ubuntu有一定的了解。

其次，虽然说ROS不依赖任何特定的编程语言，也已经支持多种不同的语言实现，例如C++、Python、Octave和LISP，但对于初学者来说，我们建议对Python有一定的了解和编程功底。

最后，我们需要对ROS的基础框架、通信协议、开发工具、功能包等进行系统的学习。本书的编写亦是按照这样的步骤来编写，并且对Linux和Python知识点进行了选择性裁剪，希望能以微薄之力，帮助初学者尽快上手ROS，进而为我国机器人事业添砖加瓦。

由于编者能力有限，书中难免有不到之处，烦请各位指正。

编委组



ART ROBOT

目录

预备篇.....	12
Chapter 1 LINUX 基础.....	12
1.1 Linux 简介.....	14
1.1.1 Linux 的发行版.....	14
1.1.2 Linux 应用领域.....	15
1.2 目录结构.....	15
1.3 用户管理.....	18
1.3.1 查看用户列表.....	18
1.3.2 查看用户组.....	18
1.3.3 创建用户账号.....	19
1.3.4 删除用户账号.....	20
1.4 文件管理.....	20
1.4.1 查看文件属性.....	20
1.4.2 修改文件权限.....	22
1.5 目录管理.....	23
1.5.1 常用命令.....	24
1.5.3 echo 重定向与追加.....	28
1.6 打包操作.....	29
1.7 文本编辑.....	30
1.7.1 什么是 vim.....	30
1.7.2 vim 的三种模式.....	30

1.7.3 vim 的使用方法.....	30
Chapter 2 PYTHON 基础.....	32
2.1 平台搭建.....	33
2.1.1 Python 发行版.....	33
2.1.2 Python 的特点.....	33
2.1.3 python 平台搭建.....	34
2.2 语言基础.....	35
2.2.1 语法初探.....	36
2.2.2 数据类型.....	36
2.2.3 变量与常量.....	39
2.2.4 控制流.....	40
2.3 数据结构.....	45
2.3.1 列表 list.....	45
2.3.2 元组 tuple.....	47
2.3.3 字典 dict.....	49
2.3.4 集合 set.....	51
2.3.4 切片索引.....	52
2.6 函数编程.....	54
2.6.1 创建函数.....	54
2.6.2 递归函数.....	56
2.6.3 内置函数.....	57
2.7 面向对象.....	58

2.7.1 类和实例.....	60
2.7.2 数据封装.....	61
2.7.3 继承和多态.....	65
2.7.4 实例属性和类属性.....	70
基础篇.....	71
Chapter 1 机器人及其发展概述.....	71
1.1 机器人的概念及特点.....	72
1.1.1 机器人的由来.....	72
1.1.2 机器人的定义.....	72
1.1.3 机器人的主要特点.....	73
1.2 机器人的组成与分类.....	74
1.2.1 机器人的组成.....	74
1.2.2 机器人的分类.....	75
1. 按照机器人的技术发展水平.....	75
2. 按机器人的机构特征.....	76
3. 按照机器人的用途分.....	78
1.3 智能机器人.....	80
1.3.1 智能机器人的定义.....	80
1.3.2 脑科学与智能机器人.....	80
1.3.3 人工智能在机器人中的应用.....	81
1.3.4 智能机器人发展展望.....	83
Chapter 2 ROS 概述.....	84

2.1 初识 ROS 操作系统.....	85
2.2 ROS 架构分析.....	88
2.3 ROS 通信机制.....	91
Chapter 2 ROS 基础.....	92
2.1 搭建 ROS 操作系统.....	93
2.2 第一个 ROS 例程.....	96
2.3 文件系统工具.....	98
2.3.1 使用 rospack.....	98
2.3.2 使用 roscd.....	99
2.3.3 使用 rosls.....	99
2.3.4 使用 rosed.....	99
2.3.5 使用 roscp.....	100
2.4 创建工作空间.....	100
2.4.1 创建工作空间.....	100
2.5.2 工作空间目录解析.....	101
2.5 功能包的创建与编译.....	102
2.6.1 catkin 功能包的组成.....	103
2.6.2 创建程序包.....	103
2.6.3 程序包依赖关系.....	104
2.6.4 程序包命名规范.....	105
2.6.5 删除程序包.....	105
2.6.6 编译功能包.....	105

2.6 ROS 节点.....	108
2.7.1 启动 roscore.....	108
2.7.2 使用 rosnode.....	109
2.8 ROS 话题.....	110
2.8.1 控制 turtle 移动.....	110
2.8.2 查看节点信息.....	111
2.8.3 查看话题信息.....	112
2.8.4 使用 rqt_graph	115
2.9 ROS 服务和参数.....	118
2.9.1 rosservice.....	118
2.9.2 使用 rosservice list.....	119
2.9.3 使用 rosservice type.....	119
2.9.4 rosparam.....	121
2.9.5 使用 rosparam list.....	121
2.9.6 使用 rosparam set 和 get.....	121
2.9.7 使用 rosparam dump 和 load.....	122
2.10 创建 ROS 消息和 ROS 服务.....	123
2.10.1 消息(msg)和服务(srv)介绍.....	123
2.10.2 使用 msg.....	124
2.10.3 使用 srv.....	126
2.11 编写发布者与订阅器.....	127
2.11.1 定义 msg 消息.....	127

2.11.2 创建发布者节点.....	129
2.11.3 创建订阅器节点.....	131
2.11.4 启动 rqt 查看节点信息.....	133
2.12 编写服务端和客户端.....	133
2.12.1 定义 srv 服务.....	133
2.12.2 编写服务端.....	135
2.12.3 编写客户端代码.....	137
2.13 编写 launch 文件.....	140
2.13.1 roslaunch 使用方法.....	140
2.13.2 编写 launch 文件.....	144
Chapter 3 ROS 常用工具.....	148
3.1 Rviz.....	149
3.1.1 RViz 安装和使用.....	149
3.2.1 RViz 界面显示.....	150
3.2 Rqt.....	152
3.1.1 rqt 安装和使用.....	152
3.1.2 rqt 主要插件.....	153
Chapter 4 SLAM 技术.....	156
4.1 SLAM 简介.....	157
4.1.1 SLAM 分类.....	159
4.1.2 激光 SLAM 框架.....	159
4.2 激光雷达数据的采集和处理.....	160

4.2.1 激光雷达测距的基本原理.....	161
4.2.2 运动畸变去除.....	163
4.3 激光 SLAM 前端配准方法.....	164
4.3.1 ICP 匹配方法（Point-Point）	164
4.3.2 PL-ICP 匹配方法（Point-Line）	165
4.3.3 基于优化的匹配方法.....	166
4.4 后端优化.....	166
4.4.1 基于滤波器的激光 SLAM 方法.....	166
4.4.2 贝叶斯滤波.....	167
4.4.3 粒子滤波.....	169
4.4.4 基于图优化的激光 SLAM 方法.....	171
首先，我们需要考虑一个最简单的最小二乘问题：	172
4.5 回环检测.....	174
4.6 基于定位的地图构建.....	176
4.6.1 地图分类.....	176
4.6.2 覆盖栅格建图算法.....	178
4.7 SALM_Gmapping 功能包.....	180
4.7.1 约束条件.....	181
4.7.2 SLAM 执行过程.....	181
4.8 ROS 机器人导航简介.....	182
4.8.1 全局路径规划.....	185
4.8.2 局部实时路径规划.....	186

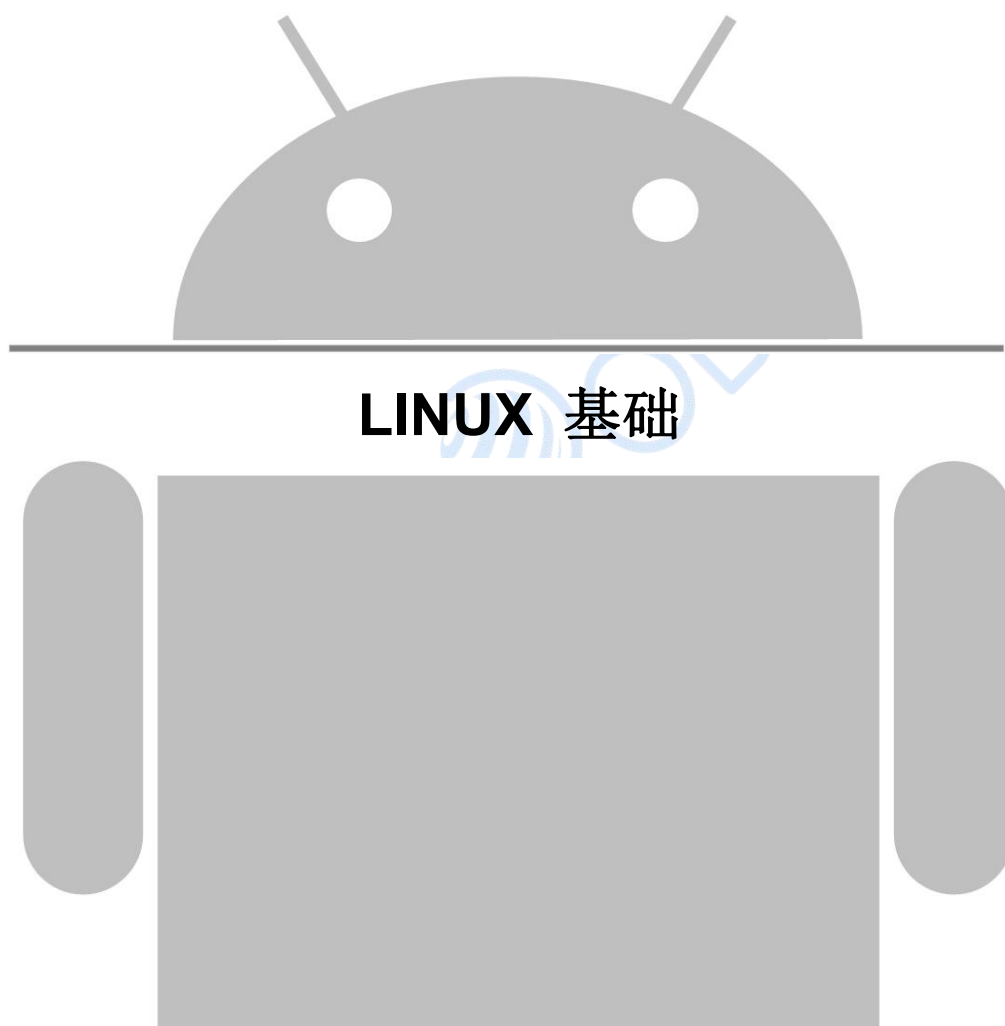
4.8.3 Costmap.....	186
4.9 本章小结.....	187
5 竞速车.....	189



ARTROBOT

预备篇

Chapter 1



1.1.2 Linux应用领域

今天各种场合都有使用各种Linux发行版，从嵌入式设备到超级计算机，并且在服务器领域确定了地位，通常服务器使用LAMP（Linux + Apache + MySQL + PHP）或LNMP（Linux + Nginx+ MySQL + PHP）组合。目前Linux不仅在家庭与企业中使用，并且在政府中也很受欢迎。

巴西联邦政府由于支持Linux而世界闻名。

有新闻报道俄罗斯军队自己制造的Linux发布版的，做为G.H.ost项目已经取得成果。

印度的Kerala联邦计划在向全联邦的高中推广使用Linux。

中华人民共和国为取得技术独立，在龙芯过程中排他性地使用Linux。

在西班牙的一些地区开发了自己的Linux发布版，并且在政府与教育领域广泛使用，如Extremadura地区的gnuLinEx和Andalusia地区的Guadalinex。

葡萄牙同样使用自己的Linux发布版Caixa Magica，用于Magalhes笔记本电脑和e-escola政府软件。

法国和德国同样开始逐步采用Linux。

1.2 目录结构

Linux下的文件系统为树形结构，入口为/ 树形结构下的文件目录：无论哪个版本的Linux系统，都有这些目录，这些目录是标准的。各个Linux发行版本会存在一些小小的差异，但总体来说，还是大体相近的。

登录linux系统后，点击鼠标右键，open Terminal，在当前命令窗口下输入命令：

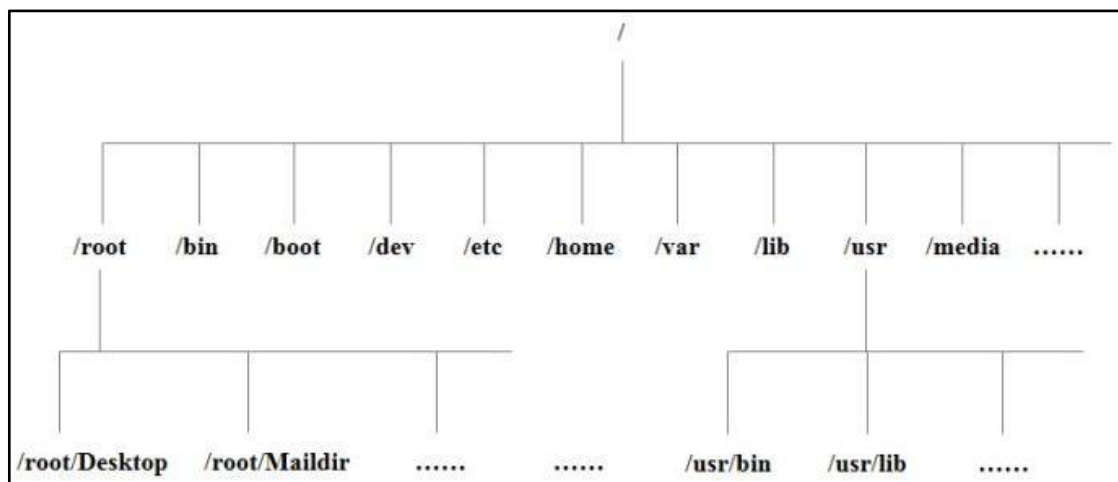
```
ls /
```

可以看到如下所示的目录结构：

```
$ ls /
bin boot dev etc home lib lib64 media mnt opt proc root run sbin .....
```

ls 指令时list单词的缩写，/ 类似于windows的磁盘符，ls / 相当于再windows下打开C盘等盘符。

Linux树状目录结构：



◆ **/bin:**

是 **Binary** 的缩写，这个目录存放着最经常使用的命令。

◆ **/boot:**

这里存放的是启动 **Linux** 时使用的一些核心文件，包括连接文件以及镜像文件。

◆ **/dev:**

dev 是 **Device**(设备)的缩写，该目录下存放的是 **Linux** 的外部设备，在 **Linux** 中访问设备的方式和访问文件的方式是相同的。

◆ **/etc:**

这个目录用来存放所有的系统管理所需要的配置文件和子目录。

◆ **/home:**

用户的主目录，在 **Linux** 中，每个用户都有一个自己的目录，一般该目录名是以用户的账号命名的。

◆ **/lib:**

这个目录里存放着系统最基本的动态连接共享库，其作用类似于 **Windows** 里的 **DLL** 文件。几乎所有的应用程序都需要用到这些共享库。

◆ **/lost+found:**

这个目录一般情况下是空的，当系统非法关机后，这里就存放了一些文件。

◆ **/media:**

linux 系统会自动识别一些设备，例如 **U** 盘、光驱等等，当识别后，**linux** 会把识别的设备挂载到这个目录下。

◆ **/mnt:**

系统提供该目录是为了让用户临时挂载别的文件系统的，我们可以将光驱挂载在

/mnt/上，然后进入该目录就可以查看光驱里的内容了。

◆ /opt:

这是给主机额外安装软件所摆放的目录。比如你安装一个 **ORACLE** 数据库则就可以放到这个目录下。默认是空的。

◆ /proc:

这个目录是一个虚拟的目录，它是系统内存的映射，我们可以通过直接访问这个目录来获取系统信息。这个目录的内容不在硬盘上而是在内存里，我们也可以直接修改里面的某些文件，比如可以通过下面的命令来屏蔽主机的 **ping** 命令，使别人无法 **ping** 你的机器：

```
echo 1 > /proc/sys/net/ipv4/icmp_echo_ignore_all
```

◆ /root:

该目录为系统管理员，也称作超级权限者的用户主目录。

◆ /sbin:

s 就是 **Super User** 的意思，这里存放的是系统管理员使用的系统管理程序。

◆ /srv:

该目录存放一些服务启动之后需要提取的数据。

◆ /tmp:

这个目录是用来存放一些临时文件的。

◆ /usr:

这是一个非常重要的目录，用户的很多应用程序和文件都放在这个目录下，类似与 **windows** 下的 **program files** 目录。

◆ /usr/bin:

系统用户使用的应用程序。

◆ /usr/sbin:

超级用户使用的比较高级的管理程序和系统守护程序。

◆ /usr/src:

内核源代码默认的放置目录。

◆ /var:

这个目录中存放着在不断扩充着的东西，我们习惯将那些经常被修改的目录放在这个目录下。包括各种日志文件。

注意:

在linux系统中,有几个目录是比较重要的,平时需要注意不要误删除或者随意更改内部文件。

/etc: 系统配置文件,如果你更改了该目录下的某个文件可能会导致系统不能启动。

/bin, /sbin, /usr/bin, /usr/sbin: 这是系统预设的执行文件的放置目录,比如 `ls` 就是在 `/bin/ls` 目录下的。值得提出的是, `/bin, /usr/bin` 是给系统用户使用的指令(除root外的通用户),而 `/sbin, /usr/sbin` 则是给root使用的指令。

/var: 这是一个非常重要的目录,系统上跑了很多程序,那么每个程序都会有相应的日志产生,而这些日志就被记录到这个目录下,具体在 `/var/log` 目录下。

1.3 用户管理

Linux系统是一个多用户多任务的分时操作系统,任何一个要使用系统资源的用户,都必须首先向系统管理员申请一个账号,然后以这个账号的身份进入系统。

用户的账号一方面可以帮助系统管理员对使用系统的用户进行跟踪,并控制他们对系统资源的访问;另一方面也可以帮助用户组织文件,并为用户提供安全性保护。

每个用户账号都拥有一个惟一的用户名和各自的口令。用户在登录时键入正确的用户名和口令后,就能够进入系统和自己的主目录。

1.3.1 查看用户列表

```
#查看用户列表
$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
postfix:x:89:89::/var/spool/postfix:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
silver:x:500:500:/home/silver:/bin/bash
.....(省略)
```

1.3.2 查看用户组

```
#查看用户组
$ cat /etc/group
root:x:0:
bin:x:1:bin,daemon
```

```
postdrop:x:90:
postfix:x:89:
fuse:x:499:
sshd:x:74:
silver:x:500:
.....(省略)
```

相关知识点：

- 1、 Linux `/etc/group` 与 `/etc/passwd` 文件都是有关于系统管理员对用户或用户组管理时的相关文件。
- 2、 Linux `/etc/group` 是用户管理员对用户或用户组管理的文件。Linux用户组的信息都存在`/etc/group`中。具有某些共有特征的用户集合起来就是用户组（group）。
- 3、 用户组的配置文件主要有`/etc/group`和`/etc/gshadow`,其`/etc/gshadow`是`/etc/group`的加密信息文件。

1.3.3 创建用户账号

```
useradd 用户名
```

```
passwd 用户名
```

注意：创建用户账号必须再 root 账户下进行，设置完密码后才可以切换用户（su 用户名）

实例演示：

```
#创建用户
$ cd /
[silver@zkxs /]$ useradd echo
-bash: /usr/sbin/useradd: 权限不够
[silver@zkxs /]$ sudo useradd echo
[silver@zkxs /]$ sudo passwd echo
更改用户 echo 的密码 。
新的 密码：
重新输入新的 密码：
passwd: 所有的身份验证令牌已经成功更新。
[silver@zkxs /]$ su echo
密码：
[echo@zkxs /]$
```

这样用户和用户密码就设置完成了，可以使用su + 用户名切换。可以使用 `sudo su` 命令直接切换到root用户下。**注意：**

```
[silver@zkxs ~]$ useradd echo
```

在这一行代码中 `silver`是用户名称，`zkxs`是主机名称，`$`表示目前是以普通用户登陆主机，当要执行一些特殊操作时候，普通用户就会出现权限不够的问题，这时候我们只要在命令前面加 `sudo` 即可，添加`sudo` 前置命令后，表示以超级用户权限执行改命令。

```
[silver@zkxs ~]$ sudo useradd echo
```

运行`sudo` 指令后会提示输入密码，输入密码之后就可以以超级用户权限执行命令，不会再出现权限不够的错误了。

1.3.4 删除用户账号

如果一个用户的账号不再使用，可以从系统中删除。删除用户账号就是要将`/etc/passwd`等系统文件中的该用户记录删除，必要时还删除用户的主目录。

删除一个已有的用户账号使用`userdel`命令，其格式如下：

```
userdel 选项 用户名
```

常用的选项是`-r`，它的作用是把用户的主目录一起删除。例如：我们删除`echo`用户：

```
$ userdel -r echo
```

1.4 文件管理

Linux系统是一种典型的多用户系统，不同的用户处于不同的地位，拥有不同的权限。为了保护系统的安全性，Linux系统对不同的用户访问同一文件（包括目录文件）的权限做了不同的规定。

1.4.1 查看文件属性

在Linux中我们可以使用`ll`或者`ls -l`命令来显示一个文件的属性以及文件所属的用户和组，如：

```
$ ll
total 108
```

```
drwxr-xr-x  2 root root  4096 12月 14 12:03 bin/
drwxr-xr-x  3 root root  4096  3月 16 13:13 boot/
drwxrwxr-x  2 root root  4096  9月  8 2017 cdrom/
drwxr-xr-x 19 root root  4000  3月 16 13:13 dev/
drwxr-xr-x 145 root root 12288  3月 16 13:13 etc/
drwxr-xr-x  3 root root  4096  9月  8 2017 home/
..... (省略)
```

实例中，boot文件的第一个属性用"d"表示。"d"在Linux中代表该文件是一个目录文件。

1、在Linux中第一个字符代表这个文件是目录、文件或链接文件等等。

- 当为[d]则是目录
- 当为[-]则是文件；
- 若是[l]则表示为链接文档(link file)；
- 若是[b]则表示为装置文件里面的可供储存的接口设备(可随机存取装置)；
- 若是[c]则表示为装置文件里面的串行端口设备，例如键盘、鼠标。

2、接下来的字符中，以三个为一组，且均为『rwx』的三个参数的组合。其中，[r]代表可读(read)、[w]代表可写(write)、[x]代表可执行(execute)。要注意的是，这三个权限的位置不会改变，如果没有权限，就会出现减号[-]而已。

3、对于文件来说，它都有一个特定的所有者，也就是对该文件具有所有权的用户。而在Linux系统中，用户是按组分类的，一个用户属于一个或多个组。文件所有者以外的用户又可以分为文件所有者的同组用户和其他用户。因此，Linux系统按照文件所有者、文件所有者同组用户和其他用户规定了不同的文件访问权限。

每个文件的属性由左边第一部分的 10 个字符来确定（如下图）。

文件 类型	属主 权限			属组 权限			其他用户 权限		
0	1	2	3	4	5	6	7	8	9
d	rwX			r-X			r-X		
目录 文件	读	写	执行	读	写	执行	读	写	执行

1.4.2 修改文件权限

Linux 文件属性有两种设置方法，一种是数字，一种是符号。

Linux 文件的基本权限就有九个，分别是 owner/group/others 三种身份各有自己的 read/write/execute 权限。

文件的权限字符为：『-rwx-rwx-rwx』，这九个权限是三个三个一组的！其中，我们可以使用数字来代表各个权限，各权限的分数对照表如下：

r:4	w:2	x:1
-----	-----	-----

每种身份(owner/group/others)各自的三个权限(r/w/x)分数是需要累加的，例如当权限为：[-rwx-rwx---] 分数则是：

```
owner = rwx = 4+2+1 = 7
group = rwx = 4+2+1 = 7
others= --  = 0+0+0 = 0
```

所以我们设定权限的变更时，该文件的权限数字就是 770 了！变更权限的指令 **chmod** 的语法是这样的：

```
chmod [-R] xyz 文件或目录 (-R, --recursive可递归遍历子目录，把修改应到目录下所有文件和子目录)
```

选项与参数：

xyz：就是刚刚提到的数字类型的权限属性，为 **rwx** 属性数值的相加。

-R：进行递归(recursive)的持续变更，亦即连同次目录下的所有文件都会变更。

实例演示：

```
#修改文件权限
[silver@zkxs ~]$ ll a.txt
-rw-rw-r--. 1 root silver 0 6月 11 23:29 a.txt
[silver@zkxs ~]$ chmod 777 a.txt
chmod: 更改"a.txt" 的权限: 不允许的操作
[silver@zkxs ~]$ sudo chmod 777 a.txt
[sudo] password for silver:
[silver@zkxs ~]$ ll
总用量 0
-rwxrwxrwx. 1 root silver 0 6月 11 23:29 a.txt
[silver@zkxs ~]$
```

那如果要将权限变成 `-rwxr-xr--` 呢？那么权限的分数就成为 $[4+2+1][4+0+1][4+0+0]=754$ 。

从之前的介绍中可以发现，基本上就九个权限分别是(1)user (2)group (3)others三种身份，那么我们就可以由u, g, o来代表三种身份的权限！

此外，a 则代表 all 亦即全部的身份！那么读写的权限就可以写成r, w, x！也就是可以使用底下的方式来看：

chmod	u	+	r	文件或目录
	g	-	w	
	o	=	x	
	a			

如果我们需要将文件权限设置为 `-rwxr-xr--`，可以使用 `chmod u=rwx,g=rx,o=r` 文件名来设定：

#去掉 group 组的可执行权限

```
[silver@zkxs ~]$ sudo chmod g=rx a.txt
[sudo] password for silver:
[silver@zkxs ~]$ ll
总用量 0
-rwxr-xrwx. 1 root silver 0 6月 11 23:29 a.txt
```

#去掉 other 组的可执行权限

```
[silver@zkxs ~]$ sudo chmod o-x a.txt
[silver@zkxs ~]$ ll
总用量 0
-rwxr-xrw-. 1 root silver 0 6月 11 23:29 a.txt
```

1.5 目录管理

我们知道Linux的目录结构为树状结构，最顶级的目录为根目录 /，其他目录通过挂载可以将它们添加到树中，通过解除挂载可以移除它们。

1.5.1 常用命令

- **ls**: 列出目录
- **cd**: 切换目录
- **pwd**: 显示目前的目录
- **mkdir**: 创建一个新的目录
- **rmdir**: 删除一个空的目录
- **cp**: 复制文件或目录
- **rm**: 移除文件或目录

● **ls**: 列出目录

- **ls** 查看当前目录包含哪里些内容
- **ll** 详细方式列出目录中的内容
- **ls -al** 以长格式列出目录中所有的内容，包括隐藏文件

● **cd** (切换目录)**Change Directory** 的缩写，这是用来变换工作目录的命令。

- **cd** 路径
- **cd ..** 回到上一级目录
- **cd /** 跳到根目录
- **cd ~** 回到家目录

● **pwd** (显示目前所在的目录)

pwd 是 **Print Working Directory** 的缩写，也就是显示目前所在目录的命令。

```
# pwd
/usr/local/zookeeper/zookeeper-3.4.9
```

● **mkdir** (创建新目录)

如果想要创建新的目录的话，就使用 **mkdir (make directory)**。语法：

```
mkdir [-mp] 目录名称
```

● 选项与参数：

- **-m** : 直接配置文件的权限
- **-p** : 创建多级目录

#创建单个目录

```
[silver@localhost ~]$ mkdir test
[silver@localhost ~]$ ls
perl5  test
```

#创建多个目录

```
[silver@localhost ~]$ mkdir -p test1/test2/test3
[silver@localhost ~]$ ls
perl5  test  test1
[silver@localhost ~]$ ls test1
test2
```

由此可见加了这个 `-p` 的选项，可以自行帮你创建多层目录，接下来我们创建权限为 `rwX--X--X` 的目录

```
[silver@localhost ~]$ mkdir -m 711 test2
[silver@localhost ~]$ ls
perl5  test  test1  test2
[silver@localhost ~]$ ll
总用量 0
drwx--x--x. 2 silver silver  6 6月  10 14:11 test2
```

上面的权限部分，如果没有加上 `-m` 来强制配置属性，系统会使用默认属性。

● **rmdir** (删除空的目录)

如果想要删除一个空的目录，就使用 **rmdir** (rmove directory)。语法：

```
rmdir [-p] 目录名称
```

实例演示：

```
[silver@localhost ~]$ rmdir test
[silver@localhost ~]$ ls
perl5  test1  test2
[silver@localhost ~]$ rmdir test1
rmdir: 删除 "test1" 失败: 目录非空
[silver@localhost ~]$ rmdir -p test1/test2/test3/
[silver@localhost ~]$ ls
perl5  test2
```


利用 `-p` 这个选项，立刻就可以将 `test1/test2/test3/test4` 一次删除。不过要注意的是，这个 `rmdir` 仅能删除空的目录，通常使用 `rm -rf` 命令来删除非空目录。

● touch (创建文件)

如果想创建一个新的文件，可以使用 `touch` 命令，语法：

Touch 文件名称

实例演示：

```
#创建一个
[silver@localhost ~]$ touch test.txt
[silver@localhost ~]$ ls
perl5  test2  test.txt

#创建多个
[silver@zkxs ~]$ touch d.txt f.txt g.txt
[silver@zkxs ~]$ ls
b.txx  c.txt  cxfs  d.py  d.txt  f.txt  g.txt
```

● cp (复制文件或目录)

如果要对文件进行复制，可以使用 `cp` 命令，语法：

`cp [-adfilprsu] 来源档(source) 目标档(destination)`

实例演示：

```
[silver@localhost ~]$ cp test.txt test2.txt
[silver@localhost ~]$ ls
perl5  test2  test2.txt  test.txt
[silver@localhost ~]$ cp test.txt test2/cxx.txt
[silver@localhost ~]$ ls test2
cxx.txt
```

● rm 删除文件或目录

如果要对文件进行删除操作，可以使用 `rm` 命令，该命令包含两个选项：

- `rm -i` 删除前提示用户进行确认
- `rm -rf` 强制删除目录

实例演示：

```
[silver@localhost ~]$ rm test2.txt
[silver@localhost ~]$ ls
perl5  test2  test.txt
[silver@localhost ~]$ rm test2
rm: 无法删除"test2": 是一个目录
[silver@localhost ~]$ rm -rf test2
[silver@localhost ~]$ ls
perl5  test.txt
```

● mv (移动文件与目录，或修改名称)

○ 移动文件

```
[silver@localhost ~]$ mkdir test
[silver@localhost ~]$ ls
perl5  test  test.txt
[silver@localhost ~]$ mv test.txt test
[silver@localhost ~]$ ls test/
test.txt
```

○ 修改名称

```
[silver@localhost ~]$ ls
perl5  test
[silver@localhost ~]$ mv test test1
[silver@localhost ~]$ ls
perl5  test1
```

1.5.2 文件内容查看指令

- cat 由第一行开始显示文件内容(常用)
- tac 从最后一行开始显示，可以看出 tac 是 cat 的倒著写！
- nl 显示的时候，顺道输出行号！
- more 一页一页的显示文件内容
- less 与 more 类似，但是比 more 更好的是，他可以往前翻页

● cat 查看文件内容

○ 查看文件

#查看文件

```
[silver@localhost ~]$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
..... (省略)
zkxs:x:1000:1000:zkxs:/home/zkxs:/bin/bash
silver:x:1001:1001::/home/silver:/bin/bash
```

○ 利用cat快速复制

#利用 cat 与追加进行文件的快速复制

```
[silver@zkxs ~]$ cat b.txt
dsddsfdsdifsdfsdfsdfsdf
[silver@zkxs ~]$ cat d.txt
[silver@zkxs ~]$ cat b.txt >> d.txt
[silver@zkxs ~]$ cat d.txt
dsddsfdsdifsdfsdfsdfsdf
```

● du查看文件大小

du -sh	查看文件总大小
du -sh 文件名	查看单个文件大小
du -sh *	列出所有文件大小(*再 linux 中代表所有的意思)
df -h	查看磁盘分区大小

实例演示：

```
[silver@zkxs ~]$ du -sh *
4.0K    a.txt
4.0K    b.txx
[silver@zkxs ~]$ df -h
Filesystem              Size  Used Avail Use% Mounted on
/dev/mapper/vg_zkxs-lv_root
                        18G   1.3G   16G   8% /
tmpfs                    491M    0    491M   0% /dev/shm
/dev/sda1                477M   28M   425M   7% /boot
```

1.5.3 echo重定向与追加

#重定向

```
[silver@localhost ~]$ echo 'i love you' >a.txt
[silver@localhost ~]$ cat a.txt
```

```
i love you
[silver@localhost ~]$ echo '!!!' >a.txt
[silver@localhost ~]$ cat a.txt
!!!
#追加
[silver@localhost ~]$ echo 'i love you' >a.txt
[silver@localhost ~]$ cat a.txt
i love you
[silver@localhost ~]$ echo '!!!' >>a.txt
[silver@localhost ~]$ cat a.txt
i love you
!!!
```

重定向会覆盖原文件的内容，追加会在原文件中加入内容。

1.6 打包操作

压缩有两个明显的好处，一是可以减少存储空间，二是通过网络传输文件时，可以减少传输的时间。在linux中，我们使用标准压缩工具gzip以及tar包操作。

● tar

使用 tar 命令打包文件格式如下：

```
tar -cvf 打包文件名 源文件
-c 打包
-v 显示过程
-f 指定打包后的文件名
```

打包压缩：

```
tar -zcvf 压缩包名.tar.gz 源文件
```

解包解压缩：

```
tar -zxvf 压缩包名.tar.gz
tar -zxvf 压缩包名.tar.gz -C 新目录
```

实例演示：

#打包压缩

```
[echo@zkxs ~]$ ls
a.txt b.txt c.txt d.py f.txt
[echo@zkxs ~]$ tar -zcvf txt.tar.gz a.txt b.txt c.txt
a.txt
b.txt
c.txt
[echo@zkxs ~]$ ls
a.txt b.txt c.txt d.py f.txt txt.tar.gz
```

#解包解压缩

```
[echo@zkxs ~]$ mkdir txt
[echo@zkxs ~]$ ls
a.txt b.txt c.txt d.py f.txt txt txt.tar.bz2 txt.tar.gz
[echo@zkxs ~]$ tar -zxvf txt.tar.gz -C txt
[echo@zkxs ~]$ ls txt
a.txt b.txt c.txt
```

1.7 文本编辑

1.7.1 什么是vim

vi(vim)是Linux 非常常用的编辑器，很多Linux发行版都默认安装了vi(vim)。vi(vim)命令繁多但是如果使用灵活之后将会大大提高效率。vi是“visual interface”的缩写，vim是vi Improved(增强版的vi)。vim是从vi发展出来的一个文本编辑器。代码补全、编译及错误跳转等方便编程的功能特别丰富，在程序员中被广泛使用，和Emacs并列成为类Unix系统用户最喜欢的文本编辑器。vim的使用包含vim的基本介绍、使用模式、文件的打开关闭保存、插入文本或新建行、移动光标、删除、恢复字符或行、搜索等等。

1.7.2 vim的三种模式

- ◆ 插入模式：在此模式下可以输入字符，按ESC将回到命令模式。
- ◆ 命令模式：可以移动光标、删除字符等。
- ◆ 底行模式：可以保存文件、退出vi、设置vi、查找等功能。

1.7.3 vim的使用方法

例如我们要通过 vim 编辑器来编写一个脚本 hello.sh，通过命令行输入：

```
$ vim hello.sh
```

此时会弹出文本编辑区域，此时整个文本编辑区处于命令模式，在文本编辑区域按下键盘 ‘I’ 键，进入插入模式（底行显示 **INSERT**），在插入模式下可以输入有效的代码。如下所示：



The screenshot shows a terminal window with a dark background. The title bar at the top reads "guest-ho0nah@ubuntun: ~". The prompt is "guest-ho0nah@ubuntun: ~". The command "echo \"hello\"" has been entered, and the output "hello" is displayed on the next line. The terminal is currently in insert mode, as indicated by the "-- INSERT --" text at the bottom left. The bottom right corner shows the page number "1.13" and the word "All".

```
guest-ho0nah@ubuntun: ~
echo "hello"
hello
-- INSERT --
```

注意：如果文件系统中之前存在 `hello.sh` 文件，那么执行 `vim hello.sh` 这条指令时，系统将默认打开原有 `hello.sh` 进行修改，否则将直接创建 `hello.sh` 脚本文件。

键入有效代码后，下一步需要对脚本进行保存，首先，按键 **ESC** 将回到命令模式，其次按键 **Shift + ;**(冒号)进入底行模式，在底行模式下输入 **wq**，即可保存退出。

wq : 保存退出

q : 退出

q! : 强制退出



A terminal window titled "guest-ho0nah@ubuntun: ~" is shown. The command `echo "hello"` has been entered and executed. The output `hello` is displayed on the line immediately following the command. A red arrow points from the bottom right towards the output text.

Chapter 2



2.1 平台搭建

Python (<https://www.python.org>) 是一种面向对象的解释型计算机程序设计语言，创始人为Guido van Rossum，1989年圣诞节期间，在阿姆斯特丹，Guido为了打发圣诞节的无趣，决心开发一个新的脚本解释程序，作为ABC 语言的一种继承。之所以选中Python（大蟒蛇的意思）作为该编程语言的名字，是因为他是一个叫Monty Python的喜剧团体的爱好者。



现在，全世界差不多有600多种编程语言，但流行的编程语言也就那么20来种。IEEE 发布2018年编程语言排行榜：Python高居前三位。

2.1.1 Python发行版

Python 2 发布于 2000 年年底，较之前版本，这是一种更加清晰和更具包容性的语言开发过程。而先前版本的 Python 应用的是 PEP (Python 增强协议)，这种技术规范能向 Python 社区成员提供信息或描述这种语言的新特性。

Python 3 被视为Python的未来，是目前正在开发中的语言版本。作为一项重大改革，Python 3 于2008年年末发布以解决和修正以前语言版本的内在设计缺陷。Python 3开发的重点是清理代码库并删除冗余，清晰地表明只能用一种方式来执行给定的任务。

在2008年Python 3.0的发布之后，Python 2.7于 2010年7月3日发布，并计划作为2.x 版本的最后一版。发布 Python 2.7的目的在于，通过提供一些测量两者之间兼容性的措施，使 Python 2.x 的用户更容易将功能移植到 Python 3 上。这种兼容性支持包括了 2.7 版本的增强模块，如支持测试自动化的 unittest，用于解析命令行选项的 argparse，以及更方便的集合类。

Python 2.7 具有 Python 2 和 Python 3.0 之间的早期迭代版本的独特位置，它因为对许多具有鲁棒性的库具有兼容性，对于程序员而言一直是非常流行的选择。当我们今天讨论 Python 2 时，我们通常指的是 Python 2.7 版本，因为它是最常用的版本。官方对于该版本的支持将在 2020 年完全停止。

2.1.2 Python的特点

既然有各种各样的编程语言可以选择，我们为什么要选择 Python 呢？主要有以下几个原因。

1、最初创建 Python 语言的出发点就是为了便于学习。是目前所有计算机语言中最易读、最容易编写，也是最容易理解的编程语言。

2、Python 是免费的。你可以下载 Python，还可以下载很多很多用 Python 编写的既好玩又有用的程序，所有这些都是在免费的。

3、Python 是开源（open source）软件。从某个角度来讲，“开源”的含义是指任何用户都可以扩展（extend）Python，也就是创建一些新“工具”。补充这些新工具后，就可以用 Python 做更多的事情，或者尽管是做同样的事情，但是有了这些新工具后会比原先更容易。

4、Python 并不是一个“玩具”。确实，它非常适合学习编程，不过实际上全世界每天都有成千上万的专业人士在使用这种语言，甚至包括类似 NASA（美国航空航天局）和 Google 这些机构的程序员。所以，学习 Python 后，你不用转换语言再去学一种“真正的”语言来编写“真正的”程序；很多工作都完全可以使用 Python 完成。

5、Python 可以在各种不同类型的计算机上运行。Windows 电脑、苹果电脑和运行 Linux 的计算机上都可以使用 Python。大多数情况下，如果一个 Python 程序可以在你家里的 Windows 电脑上运行，那么这个程序同样可以在你学校的苹果电脑上运行，兼容性高。

2.1.3 python平台搭建

要搭建python开发环境有两个问题需要考虑，第一是选择什么操作系统，是windows还是linux？第二是选择python哪个版本，在上一节中我们阐述过python的发行版，我们选用python 3.6版本作为本书的开发版本。

关于操作系统，主要是在windows和linux之间选择。Python是跨平台的语言，因此脚本可以跨平台运行。然而，不同的平台运行效率是不一样的，一般来说在Linux下运行速度是比Windows要快的，此外，在Linux下搭建Python环境相对来说容易一些，很多Linux发行版自带了Python2.7版本。当然，linux的操作门槛比较高，对于初学者，还是建议先选择Windows开发，然后再考虑迁移到Linux开发环境中。

◆ python 3.6发行版安装

Windows系统安装Python比较容易，用户可直接到Python官网下载相应版本的msi安装包即可，软件分32位与64位，用户可按照需求自行安装，和一般的软件安装无异。

◆ Anaconda 3安装

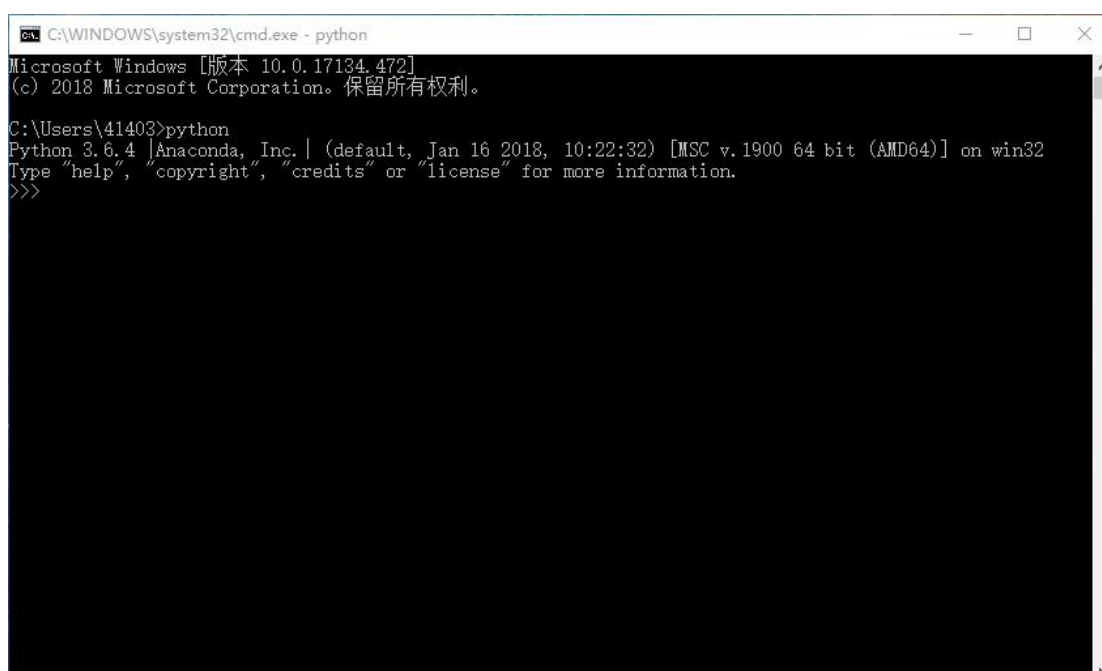
安装Python核心程序只是第一步，为了实现更丰富的功能，还需要安装一些第三方的扩展库，这对于初学者来说是比较困难的，尤其在Windows系统下，幸好，已经有专门的软件已经为我们将常用的第三方库以及编译器都安装好了，Anaconda就是这样的软件。

Anaconda具有如下特点：

- 1、包含了众多流行的科学、数学、工程、数据分析的Python包。
- 2、包含了Spyder和Jupyter Notebook常用编译器，无需再下载其他编译器。
- 3、完全开源和免费。

因此，推荐初学者使用Anaconda进行学习，Anaconda的安装与python一样，只需要到Anaconda官方网站（<https://www.anaconda.com/download/>）下载安装包安装即可，和一般的软件安装无异。

安装好Python和Anaconda后，只需要在命令窗口输入python就可以进入python环境，如下图所示，在Windows下启动python的界面：



```
C:\WINDOWS\system32\cmd.exe - python
Microsoft Windows [版本 10.0.17134.472]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\41403>python
Python 3.6.4 |Anaconda, Inc.| (default, Jan 16 2018, 10:22:32) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

2.2 语言基础

Python是一种计算机编程语言。计算机编程语言和我们日常使用的自然语言有所不同，最大的区别就是，自然语言在不同的语境下有不同的理解，而计算机要根据编程语言执行任务，就必须保证编程语言写出的程序决不能有歧义，所以，任何一种编程语言都有自己的一

套语法，编译器或者解释器就是负责把符合语法的程序代码转换成CPU能够执行的机器码，Python也不例外。

2.2.1 语法初探

Python的语法比较简单，采用缩进方式代替一般高级语言的{ }括号对：

```
# print absolute value of an integer:
a = 100
if a >= 0:
    print(a)
else:
    print(-a)
```

1、以 # 开头的语句是注释，注释是给人看的，可以是任意带有提示色彩内容，解释器编译时会忽略掉注释。

2、当语句以冒号 : 结尾时，缩进的语句视为代码块，冒号之后的所有代码的缩进量必须相同，直到代码块结束。等同于其他编程语言的{ }对。

3、缩进好处是强迫你写出格式化的代码，但没有规定缩进是几个空格还是Tab。一般编译器的默认设置都是4个空格的缩进。

4、Python语言的一个重要特性就是它的对象模型的一致性。每个数字、字符串、数据结构、函数、类、模块等等，都是在Python解释器的自有“盒子”内，它被认为是Python对象。每个对象都有类型（例如，字符串或函数）和内部数据。在实际中，这可以让语言非常灵活，因为函数也可以被当做对象使用。

2.2.2 数据类型

计算机是一种用于高速计算的电子计算机器，可以进行数值计算，又可以进行逻辑计算，还具有存储记忆功能。是能够按照程序运行，自动、高速处理海量数据的现代化智能电子设备。除此以外，计算机能处理的远不止数值，还可以处理文本、图形、音频、视频、网页等各种各样的数据，不同的数据，需要定义不同的数据类型。在Python中，能够直接处理的数据类型有以下几种：

◆ 整数（int）

Python可以处理任意大小的整数，当然包括负整数，在程序中的表示方法和数学上的写法一模一样，例如：1，100，-8080，0，等等。

```
>>> ival = 123497
>>> ival * 0.2
24699.4
```

◆ 浮点数 (float)

浮点数也就是小数，之所以称为浮点数，是因为按照科学记数法表示时，一个浮点数的小数点位置是可变的，比如， 1.23×10^9 和 12.3×10^8 是完全相等的。浮点数可以用数学写法，如1.23，3.14，-9.01，等等。但是对于很大或很小的浮点数，就必须用科学计数法表示，把10用e替代， 1.23×10^9 就是1.23e9，或者12.3e8，0.000012可以写成1.2e-5，等等。

另外整数的除法结果系统会自动保存为浮点数类型。

```
>>> fval = 1.45
>>> fval = 3/2
>>> print(fval)
1.5
```

◆ 字符串 (str)

强大、灵活的内建字符串操作时很多人喜欢python的原因。字符串是以单引号 ' 或双引号 " 括起来的任意文本，比如'abc'，"xyz"等等。请注意，"或""本身只是一种表示方式，不是字符串的一部分，因此，字符串'abc'只有a，b，c这3个字符。另外字符串还具有拼接功能。

```
>>> a = 'I am'
>>> b = 'YX'
>>> print(a+b)
I am YX
```

◆ 布尔值

布尔值和布尔代数的表示完全一致，一个布尔值只有True、False两种值，要么是True，要么是False，在Python中，可以直接用True、False表示布尔值（请注意大小写），也可以通过布尔运算计算出来：

```
>>> 3 > 2
True
>>> 3 > 5
False
```

布尔值可以用and、or和not运算。and运算是与运算，只有所有都为True，and运算结果才是True：

```
>>> True and True
True
>>> True and False
False
>>> 5 > 3 and 3 > 1
True
```

or运算是或运算，只要其中有一个为True，or运算结果就是True：

```
>>> True or True
True
>>> True or False
True
>>> False or False
False
>>> 5 > 3 or 1 > 3
True
```

not运算是非运算，它是一个单目运算符，把True变成False，False变成True：

```
>>> not True
False
>>> not False
True
>>> not 1 > 2
True
```

布尔值经常用在条件判断中，比如：

```
if age >= 18:
    print('adult')
else:
    print('teenager')
```

◆ 空值

空值是Python里一个特殊的值，用None表示。None不能理解为0，因为0是有意义的，而None是一个特殊的空值。

◆ 类型转换

Python数据类型之间可以通过转换来达到一致的效果，如下所示：

```
In [34]: str1 = '3.14'
In [35]: type(str1)
Out[35]: str1

In [36]: fval = float(str1)
In [37]: type(fval)
Out[37]: float

In [44]: ivla = int(fval)
In [45]: type(ivla)
Out[45]: int
```

2.2.3 变量与常量

◆ 变量

变量的概念基本上和初中代数的方程变量是一致的，只是在计算机程序中，变量不仅可以是数字，还可以是任意数据类型。

变量在程序中就是用了一个变量名表示了，变量名必须是大小写英文、数字和_的组合，且不能用数字开头，比如：

```
a = 1           // 变量 a 是一个整数。
b = 'T007'      // 变量 b 是一个字符串。
Answer = True   // 变量 Answer 是一个布尔值 True。
```

在Python中，“=”是表示赋值，可以把任意数据类型赋值给变量，同一个变量可以多次赋值，而且可以是不同类型的变量。

当在Python中创建变量（或名字），如下图所示，你就在等号右边创建了一个对这个变量的引用。考虑一个整数列表。

```
>>> a = [1,2,3]
```

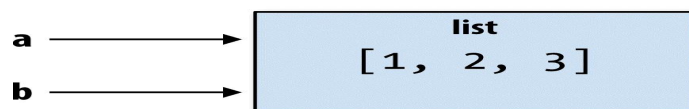
假设将a赋值给一个新变量b：

```
>>> b = a
```

在有些方法中，这个赋值会将数据[1, 2, 3]也复制。在Python中，a和b实际上是同一个对象，即原有列表[1, 2, 3]（如下图）。你可以在a中添加一个元素，然后检查b：

```
>>> a.append(4)
>>> a
[1, 2, 3, 4]
>>> b
[1, 2, 3, 4]
```

上述执行过程a和b两个引用指向了同一个对象，理解python引用复制数据的时机、方法、原因，对利用python处理大数据尤为重要。



赋值有时候也被称为绑定，这是应为我们将一个变量名绑定到了一个对象上。已被赋值的变量名有时候也称为被绑定变量。

◆ 常量

所谓常量就是不能变的变量，比如常用的数学常数 π 就是一个常量。在Python中，通常用全部大写的变量名表示常量。

```
PI = 3.14159265359
```

但事实上PI仍然是一个变量，Python根本没有任何机制保证PI不会被改变，所以，用全部大写的变量名表示常量只是一个习惯上的用法。

2.2.4 控制流

和其他语言中的标准控制流概念一样，Python有若干内建的关键字进行条件逻辑、循环和其它控制流操作。

◆ 条件语句 if

所谓条件语句是通过一条或多条语句的执行结果（True或者False）来决定执行的代码块。if是最广为人知的控制流语句。它检查一个条件，如果为True，就执行后面的语句：

```
>>> x = 10
>>> if x > 0:
    print('Yes')
Yes
```

if后面可以跟一个或多个elif，所有条件都是False时，还可以添加一个else：

```
if 条件 1:
    执行语句。块 1……
elif 条件 2:
    执行语句块 2……
elif 条件 3:
    执行语句块 3……
else:
    执行语句块 4……
```

当判断需要多个条件同时判断时，可以用 >（大于）、<（小于）、==（等于）、>=（大于等于）、<=（小于等于）等关系来表示，同时可以利用 or 表示多个条件中存在一个成立时，判断条件为True；利用 and 表示只有所有条件同时成立的时候，判断条件才为True。

```
# elif 用法
num = 90
if num == 100:           # 判断 num 的值
    print 'perfect'
elif num >= 80 and num <= 100:
    print 'good'
elif num >= 60 and num <= 80:  # 值在某个区间时输出 不同结果
    print 'pass'
else:
    print 'fail'           # 条件均不成立时输出不及格
# 输出结果
good
```

当一个 if 判断中有多个条件时，则需要对判断条件的先后顺序有一个规定：括号中的判断优先级别较高，然后是 >（大于）、<（小于）等判断符号，最后是 and 和 or 的优先级较低。

```
a = 5
b = 7
c = 8
d = 4
if a < b or c > d:
    print('YX')
# 输出结果
YX
```


在这个例子中，`c > d`不会被执行，因为第一个比较是`True`。

◆ for循环

循环语句允许我们多次执行一个语句或语句组，`for`循环是在一个集合（列表或元组）中进行迭代，或者就是一个迭代器。`for`循环的标准语法是：

```
for value in sequence:
    # statements(s)
```

例如，我们对字符串以及列表进行循环访问：

```
for i in 'YX':
    print('当前字母为: ',i)
balls = ['Basketball', 'football', 'pingpangball']
for ball in balls:
    print(ball)
```

#执行结果：

当前字母为: Y

当前字母为: X

Basketball

football

pingpangball

可以用`continue`使`for`循环提前结束本次循环剩下的部分，直接进入下一轮循环。例如下面这个例子，将一个列表中的整数相加，跳过`None`：

```
sequence = [1,2,None,5,6,7,None]
total = 0
for value in sequence:
    if value == None:
        continue
    total +=value
```

#执行结果：

计算结果为: 21

也可以用`break`直接跳出整个`for`循环。下面的代码将各元素相加，直到遇到5：

```
sequence = [1,2,3,4,6,5,8,10,20]
total = 0
```

```
total_limit = 5
for value in sequence:
    if value == 5:
        break
    total +=value
#执行结果：
计算结果为： 16
```

注意：break只中断for循环的最内层，其余的for循环仍会运行：

```
for i in range(4):
    for j in range(4):
        if j > i:
            break
        print((i,j))
#执行结果：
(0, 0)
(1, 0)
(1, 1)
(2, 0)
(2, 1)
(2, 2)
(3, 0)
(3, 1)
(3, 2)
(3, 3)
```

◆ while循环

Python 编程中 while 语句用于循环执行程序，即在某条件下，循环执行某段程序，以处理需要重复处理的相同任务。当条件为False或用break退出循环，代码才会退出其基本形式为：

```
while 条件表达式:
    Python 代码
...
```

执行语句可以是单个语句或者是语句块。判断条件可以是任何表达式，任何非零、或非空（null）的值均为true。当判断条件假false时，循环结束。

```
num = 1
total = 0
while num <= 50:
    total = total + num
    num += 1
print(total)
#执行结果：
1275
```

`while` 语句也是可以通过 `continue`, `break` 来跳过循环, `continue` 用于跳过本次循环, `break` 则是用于退出整个循环, 此外"判断条件"还可以是个常值, 表示循环必定成立。

```
i = 1
while i < 10:
    i += 1
    if i%2 > 0:
        continue
    print i
#执行结果：
2
4
6
8
10
```

上面的代码如果我们改为 `while 1`, 则整个循环会陷入无法停止即“死循环”的状态, 灵活使用`while 1`循环, 对我们的代码维护起重要作用。

◆ 三元表达式

Python中的三元表达式可以将一个if-else语句放到一行里, 完成一项功能, 语法如下:

```
value = true-expr if condition else false-expr
```

`true-expr`或`false-expr`可以是任何Python代码。它和下面的代码效果相同:

```
if condition:
    value = true-expr
else:
    value = false-expr
```

来看一个具体示例：

```
x = 5
'Yes' if x>=0 else 'No'
```

和if-else一样，只有一个表达式会被执行。因此，三元表达式中的if和else可以包含大量的计算，但只有True的分支会被执行。但是当条件判断比较复杂是，会降低代码的可读性。

2.3 数据结构

Python有4个内建的数据结构，分别是列表、元组、字典以及集合，它们可以统称为容器(container)，因为它们实际上是一些“元素”组合而成的结构，而这些“元素”可以是数字、字符甚至是列表，或者是它们之间的组合。通俗的讲，容器里存储说明都可以，而且对存储元素的类型也不要求必须统一。

2.3.1 列表 list

列表(list)是Python中最基本的数据结构，是一种有序的集合，可以随时添加和删除其中的元素(可以被随时修改)，且每个元素都会分配一个数字即索引，索引从0开始计数，元素可通过索引方式检索。

列表(list)的数据项不需要具有相同的类型，创建一个列表，只要把逗号分隔的不同的数据项，使用方括号括起来即可。

比如，列出班里所有同学的名字，就可以用一个list表示：

```
>>> classmates = ['Michael', 'Bob', 'Tracy']
>>> classmates
['Michael', 'Bob', 'Tracy']
```

变量classmates就是一个list。用len()函数可以获得list元素的个数：

```
>>> len(classmates)
3
```

用索引来访问list中每一个位置的元素，记得索引是从0开始的：

```
>>> classmates[0]
'Michael'
>>> classmates[1]
'Bob'
```

```
>>> classmates[2]
'Tracy'
>>> classmates[3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

当索引超出了范围时，Python会报一个IndexError错误，所以，要确保索引不要越界，记得最后一个元素的索引是`len(classmates) - 1`。

如果要取最后一个元素，除了计算索引位置外，还可以用-1做索引，直接获取最后一个元素：

```
>>> classmates[-1]
'Tracy'
```

以此类推，可以获取倒数第2个、倒数第3个，当然，倒数第4个就越界了，系统会出现报错：

```
>>> classmates[-2]
'Bob'
>>> classmates[-3]
'Michael'
>>> classmates[-4]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

`list`是一个可变的有序表，所以，可以利用`append()`方法往`list`末尾追加元素：

```
>>> classmates.append('Adam')
>>> classmates
['Michael', 'Bob', 'Tracy', 'Adam']
```

也可以利用`insert()`方法把元素插入到指定的位置，比如索引号为1的位置：

```
>>> classmates.insert(1, 'Jack')
>>> classmates
['Michael', 'Jack', 'Bob', 'Tracy', 'Adam']
```

如果要删除`list`末尾的元素，用`pop()`方法：

```
>>> classmates.pop()
'Adam'
>>> classmates
['Michael', 'Jack', 'Bob', 'Tracy']
```

如果要删除指定位置的元素，用`pop(i)`方法，其中`i`是索引位置：

```
>>> classmates.pop(1)
'Jack'
>>> classmates
['Michael', 'Bob', 'Tracy']
```

如果要把某个元素替换成别的元素，可以直接赋值给对应的索引位置：

```
>>> classmates[1] = 'Sarah'
>>> classmates
['Michael', 'Sarah', 'Tracy']
```

◆ Python列表脚本操作符：

另外列表还支持系列脚本操作符，对 `+` 和 `*` 的操作符与字符串相似，`+` 号用于组合列表，`*` 号用于重复列表。如表所示：

Python 表达式	结果	描述
<code>len([a,b,c])</code>	3	长度
<code>[a,b,c]+[d,e,f]</code>	<code>[a,b,c,d,e,f]</code>	组合
<code>['hello'] * 3</code>	<code>['hello', 'hello', 'hello']</code>	重复
<code>a in [a,b,c]</code>	True	归属
<code>for x in [a,b,c]: print x,</code>	a,b,c	迭代

2.3.2 元组 tuple

另一种有序列表叫元组（`tuple`）。`tuple`和`list`非常类似，但是`tuple`序列中的数值一旦初始化就**不能再被修改**，比如同样是列出同学的名字：

```
>>> classmates = ('Michael', 'Bob', 'Tracy')
```

但是此处的`classmates`是不可变的，它也没有`append()`，`insert()`这样的方法。其他获取元素的方法和`list`是一样的，可以正常地使用`classmates[0]`，`classmates[-1]`，但不能赋值成另外

的元素。

元组与列表最大的区别就是元组不能被修改，列表可以被修改。那不可变的tuple有什么意义？因为tuple不可变，所以代码更安全。如果可能，能用tuple代替list就尽量用tuple。

当你定义一个tuple时，tuple的元素就必须被确定下来，比如：

```
>>> t = (1, 2)
>>> t
(1, 2)
```

如果要定义一个空的tuple，可以写成():

```
>>> t = ()
>>> t
()
```

但是，要定义一个只有1个元素的tuple，如果你这么定义：

```
>>> t = (1)
>>> t
1
```

定义的不是tuple，是1这个数！这是因为括号()既可以表示tuple，又可以表示数学公式中的小括号，这就产生了歧义，因此，Python规定，这种情况下，按小括号进行计算，计算结果自然是1。

所以，只有1个元素的tuple定义时必须加一个逗号，来消除歧义：

```
>>> t = (1,) # 注意逗号
>>> t
(1,)
```

Python在显示只有1个元素的tuple时，也会加一个逗号，以免你误解成数学计算意义上的括号。

最后来看一个“可变的”tuple：

```
>>> t = ('a', 'b', ['A', 'B'])
>>> t[2][0] = 'X'
>>> t[2][1] = 'Y'
>>> t
('a', 'b', ['X', 'Y'])
```

这个tuple定义的时候有3个元素，分别是'a'、'b'和一个list。表面上看，tuple的元素确实变了，但其实变的不是tuple的元素，而是list的元素。tuple一开始指向的list并没有改成别的list，所以，tuple所谓的“不变”是说，tuple的每个元素，指向永远不变。即指向'a'，就不能改成指向'b'，指向一个list，就不能改成指向其他对象，但指向的这个list本身是可变的！

由上所知，list和tuple都是Python内置的有序集合，list中元素是的可变，tuple中元素是不可变。所以再创建数据时，我们应该根据需求来选择使用list还时tuple。例如，如果希望被定义的数据不被篡改，应该使用tuple。

2.3.3 字典 dict

字典（dict）是一个简单的结构，也叫关联数组。Python内置了字典，全称dictionary，使用键-值（key-value）存储，具有极快的查找速度。

例如，根据同学的名字查找对应的成绩，如果用list实现，则需要两个list：

```
names = ['Michael', 'Bob', 'Tracy']
scores = [95, 75, 85]
```

给定一个名字，要查找对应的成绩，就先要在names中找到对应的位置，再从scores取出对应的成绩，list越长，耗时越长。

如果用dict实现，只需要一个“名字” - “成绩”的对照表，直接根据名字查找成绩，无论这个表有多大，查找速度都不会变慢。用Python创建一个dict如下：

```
>>> d = {'Michael': 95, 'Bob': 75, 'Tracy': 85}
>>> d['Michael']
95
```

为什么dict查找速度这么快？因为dict的实现原理和查字典是一样的。假设字典包含了1万个汉字，我们要查某一个字，一个办法是把字典从第一页往后翻，直到找到我们想要的字为止，这种方法就是在list中查找元素的方法，list越大，查找越慢。第二种方法是先在字典的索引表里（比如部首表）查这个字对应的页码，然后直接翻到该页，找到这个字。无论找哪个字，这种查找速度都非常快，不会随着字典大小的增加而变慢。dict就是第二种实现方式，给定一个名字，比如'Michael'，dict在内部就可以直接计算出Michael对应的存放成绩的“页码”，也就是95这个数字存放的内存地址，直接取出来，所以速度非常快。

除了上面的直接创建非空字典外，也可以直接创建一个空字典，然后向其中加入东西。


```

#创建一个空字典
>>> d0 = {}
>>> d0['name']='yx'
>>> d0['age']=18
>>> d0
{'name': 'yx', 'age': 18}
#创建一个非空字典
>>> d1={'name':'yx','age':18,'add':'xa'}
>>> d1
{'name': 'yx', 'age': 18, 'add': 'xa'}

```

另外如果键值都相同的话，可以用fromkeys来完成初始化工作。

```

>>> d3 = dict.fromkeys(['A','B','C'],0)
>>> d3
{'A': 0, 'B': 0, 'C': 0}

```

可以像访问列表或元组中的元素一样，访问、插入或设定字典中的元素。采用dict[key]形式访问字典，系统返回键key对应的值value。

```

#访问字典
>>> d3 = dict.fromkeys(['A','B','C'],0)
>>> d3['A']
0

```

如果要修改某一项value值，可以通过key值来操作：

```

#修改字典
>>> adict ['age"] = 20
>>> adit['age']
20

```

由于一个key只能对应一个value，所以，多次对一个key赋值value，后面的值会把前面的值覆盖掉。如果key不存在，dict就会报错：

```

>>> adit['Thomas']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Thomas'

```

要避免key不存在的错误，有两种办法，一是通过in判断key是否存在：

#判断 Key 时候存在

```
>>> 'Thomas' in d
False
```

二是通过dict提供的get()方法，如果key不存在，可以返回None，但是None在交互式环境下不显示，当然也可以自己指定的返回值：

```
>>> d.get('Thomas')
>>>                                     # None 在交互状态下不显示
>>> d.get('Thomas', -1)
-1                                     # 指定返回值
```

如果要删除一个key，用pop(key)方法，对应的value也会从dict中删除：

```
#删除 key
>>> d.pop('Bob')
75
>>> d
{'Michael': 95, 'Tracy': 85}
```

和list比较，dict的查找和插入的速度极快，不会随着key的增加而变慢；而list相反，查找和插入的时间随着元素的增加而增加；所以，dict是用空间来换取时间的一种方法。dict可以用在需要高速查找的很多地方，在Python代码中几乎无处不在，正确使用dict非常重要，但需要注意的是dict的key必须是不可变的对象。

2.3.4 集合 set

集合（set）是一个无序不重复元素的集。基本功能包括关系测试和消除重复元素，还可以计算交集、差集、并集等。例如，要创建一个set，需要提供一个list作为输入集合：

```
>>> s = set([1, 2, 3])
>>> s
{1, 2, 3}
```

注意，传入的参数[1, 2, 3]是一个list，而显示的{1, 2, 3}，这只是告诉你这个set内部有1, 2, 3这3个元素，显示的顺序也不表示set是有序的。

重复元素在set中自动被过滤：

```
>>> s = set([1, 1, 2, 2, 3, 3])
```

```
>>> s
{1, 2, 3}
```

通过`add(key)`方法可以添加元素到`set`中，可以重复添加，但不会有效果：

```
>>> s.add(4)
>>> s
{1, 2, 3, 4}
>>> s.add(4) # 重复添加无效
>>> s
{1, 2, 3, 4}
```

通过`remove(key)`方法可以删除元素：

```
>>> s.remove(4)
>>> s
{1, 2, 3}
```

`set`可以看成数学意义上的无序和无重复元素的集合，因此，两个`set`可以做数学意义上的交集、并集、差集等操作：

```
>>> x= set([1,2,3,4,2,3,4])
>>> y= set([3,4,5,6])
>>> x,y
({1, 2, 3, 4}, {3, 4, 5, 6}) # 重复的被删除
>>> x&y # 交集
{3, 4}
>>> x|y # 并集
{1, 2, 3, 4, 5, 6}
>>> x-y # 差集
{1, 2}
```

`set`和`dict`的唯一区别仅在于没有存储对应的`value`，但是，`set`的原理和`dict`一样，所以，同样不可以放入可变对象，因为无法判断两个可变对象是否相等，也就无法保证`set`内部不会有重复元素。

2.3.4 切片索引

使用切片可以选取列表和元组中的子集，其基本形式是在方括号中使用`start:stop`，切片选取的原则是取前不取后，因此，结果中包含的元素个数是`stop - start`，而且`start`或`stop`都

可以被省略，省略之后，分别默认序列的开头和结尾。如下所示：

```
In [4]: data = [1,4,5,7,3,13,35,657,768,'yx','alex']
```

```
In [5]: data[1:5]
```

```
Out[5]: [4, 5, 7, 3]
```

```
In [6]: data[:5]
```

```
Out[6]: [1, 4, 5, 7, 3]
```

```
In [7]: data[3:7]
```

```
Out[7]: [7, 3, 13, 35]
```

```
In [8]: data[5:]
```

```
Out[8]: [13, 35, 657, 768, 'yx', 'alex']
```

切片也可以被序列赋值：

```
In [9]: data[2:4] = [6,34]
```

```
In [10]: data
```

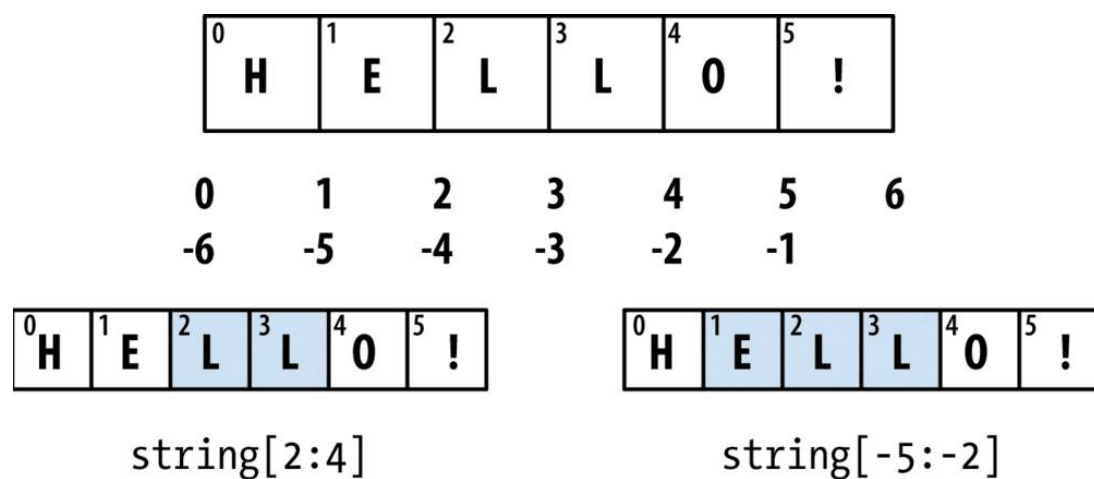
```
Out[10]: [1, 4, 6, 34, 3, 13, 35, 657, 768, 'yx', 'alex']
```

在第二个冒号后面使用`step`表示选取的步长，可以隔`step`个位置取一个元素：

```
In [11]: data[3::2]
```

```
Out[11]: [34, 13, 657, 'yx']
```

如下图所示展示了正整数和负整数的切片过程。在图中，指数标示在边缘以表明切片是在哪里开始哪里结束的。



从上图可看出，Python支持L[-1]取倒数第一个元素，那么它是否也同样支持倒数切片呢？我们可以尝试：

```
data[-3:]  
Out[34]: [768, 'yx', 'alex']
```

可以看到，我们取出了列表中的后三个元素，所以倒数切片的方法也是同样适用的，合理的使用切片操作可以有效的提高编程效率，这需要我们再实践编程中不断练习。

2.6 函数编程

基本上所有的高级语言都支持函数，Python也不例外。函数是Python中最主要的代码组织和复用手段。对于一些重复使用或非常类似的代码，可以通过编写一个函数来提高代码的可读性。并且Python不但能非常灵活地定义函数，而且本身内置了很多有用的函数，可以直接调用。

2.6.1 创建函数

首先我们来看一个示例，我们知道圆的面积计算公式为：

$$S = \pi r^2$$

当我们知道半径r的值时，就可以根据公式计算出面积。假设我们需要计算3个不同大小的圆的面积：

```
r1 = 12.34  
r2 = 9.08  
r3 = 73.1  
s1 = 3.14 * r1 * r1  
s2 = 3.14 * r2 * r2  
s3 = 3.14 * r3 * r3
```

当代码出现有规律的重复的时候，你就需要当心了，每次写`3.14 * x * x`不仅很麻烦，而且，如果要把3.14改成3.14159265359的时候，得全部替换。那么利用函数如何实现代码复用呢？

```
def area_of_circle(r):  
    return 3.14 * r * r
```

有了函数，我们就不再每次写`s = 3.14 * x * x`，而是通过函数调用去完成计算，而函

数`area_of_circle`本身只需要写一次，就可以多次调用。如下所示：

```
In [16]: area_of_circle(2)
```

```
Out[16]: 12.56
```

```
In [17]: area_of_circle(3.5)
```

```
Out[17]: 38.465
```

在Python中，定义一个函数要使用`def`语句，依次写出函数名、括号、括号中的参数和冒号:，然后，在缩进块中编写函数体，函数的返回值用`return`语句返回。例如：

```
def my_function(x,y,z):  
    if z>0:  
        return x+y  
    else:  
        return x-y
```

定义完函数主体之后，我们可以在有代码需求的地方运行函数调用。

```
In [19]: my_function(2,4,5)
```

```
Out[19]: 6
```

```
In [20]: my_function(6,2,-1)
```

```
Out[20]: 4
```

函数可以返回多个值吗？答案是肯定的。例如在游戏中，游戏角色经常需要从一个点移动到另一个点，给出坐标、位移和角度，就可以计算出新的新的坐标：

```
import math  
def move(x, y, step, angle=0):  
    nx = x + step * math.cos(angle)  
    ny = y - step * math.sin(angle)  
    return nx, ny
```

`import math`语句表示导入`math`包，并允许后续代码引用`math`包里的`sin`、`cos`等函数，通过函数调用，我们就可以获得返回值，得到游戏角色的新坐标：

```
>>> x, y = move(100, 100, 60, math.pi / 6)
```

```
>>> print(x, y)
```

```
151.96152422706632 70.0
```

但其实这只是一种假象，Python函数返回的仍然是单一值：

```
>>> r = move(100, 100, 60, math.pi / 6)
>>> print(r)
(151.96152422706632, 70.0)
```

由此可知函数的返回值实际上是一个**tuple**，但是，在语法上，返回一个**tuple**可以省略括号，而多个变量可以同时接收一个**tuple**，按位置赋给对应的值，这样使用起来会更加便捷。

2.6.2 递归函数

在函数内部，可以调用其他函数。如果一个函数在内部调用自己本身，这个函数就是递归函数。

举个例子，我们来计算阶乘（factorial） $n! = 1 * 2 * 3 * \dots * n$ ，用函数fact(n)表示，可以看出：

$$\text{fact}(n) = n! = 1 * 2 * 3 * \dots * (n-1) * n = (n-1)! * n = \text{fact}(n-1) * n$$

所以， $\text{fact}(n)$ 可以表示为 $n * \text{fact}(n-1)$ ，只有 $n=1$ 时需要特殊处理。

于是，`fact(n)`用递归的方式写出来就是：

```
def fact(n):
    if n==1:
        return 1
    return n * fact(n - 1)
```

上面就是一个递归函数。可以试试：

```
>>> fact(1)
1
>>> fact(5)
120
>>> fact(100)
9332621544394415268169923885626670049071596826438162146859296389521759999322991560
89414639761565182862536979208272237582511852109168640000000000000000000000
```

递归函数的优点是定义简单，逻辑清晰。理论上，所有的递归函数都可以写成循环的方式，但循环的逻辑不如递归清晰。

2.6.3 内置函数

Python的除了可以自己创建函数以外还提供了强大的内置函数支持，了解学习这些内置函数能有效的提高编程效率。

◆ sorted 函数

排序也是在程序中经常用到的算法。无论使用冒泡排序还是快速排序，排序的核心是比较两个元素的大小。如果是数字，我们可以直接比较，但如果是字符串或者两个dict呢？直接比较数学上的大小是没有意义的，因此，比较的过程必须通过函数抽象出来。

`sorted` 函数可以从任意序列的元素返回一个新的排好序的列表：

```
In [37]: a = [4,5,2,7,12,34,8]

In [38]: sorted(a)
Out[38]: [2, 4, 5, 7, 8, 12, 34]
```

此外，`sorted()`函数也是一个高阶函数，它还可以接收一个`key`函数来实现自定义的排序，例如按绝对值大小排序：

```
In [39]: sorted([36, 5, -12, 9, -21], key=abs)
Out[39]: [5, 9, -12, -21, 36]
```

◆ zip 函数

`zip`可以将多个列表、元组或其它序列成对组合成一个元组列表：

```
In [46]: seq1 = ['alex',18,'bj']
In [47]: seq2 = ['one','two','three']
In [48]: zipped = zip(seq1,seq2)
In [49]: list(zipped)

Out[49]: [('alex', 'one'), (18, 'two'), ('bj', 'three')]
```

`zip`可以处理任意多的序列，元素的个数取决于最短的序列：

```
In [50]: seq3 = ['bj','tj',]
In [51]: list(zip(seq1,seq2,seq3))

Out[51]: [('alex', 'one', 'bj'), (18, 'two', 'tj')]
```

◆ lambda 函数

Python支持一种被称为匿名的、或**lambda**函数。它仅由单条语句组成，该语句的结果就是返回值。它是通过**lambda**关键字定义的，这个关键字没有别的含义，仅仅是说“我们正在声明的是一个匿名函数”。例如，我们实现如下功能：

```
#函数方式实现
def my_function(x):
    return x * 2

#lambda 方式实现
data = lambda x:x * 2
```

执行函数调用，可得结果：

```
In [53]: my_function(32)
Out[53]: 64

In [55]: data(32)
Out[55]: 64
```

在数据分析工作中使用**lambda**函数是非常方便，因为很多数据转换函数都以函数作为参数。直接传入**lambda**函数比编写完整函数声明要少输入很多代码，例如,直接将**lambda**函数作为参数传入：

```
data = [3,4,5,2,7,8]
def apply_to_list(some_list,f):
    return [f(x) for x in some_list]

#函数调用
apply_to_list(data,lambda x :x*2)

#执行结果
[6, 8, 10, 4, 14, 16]
```

从上面的示例可以看到，匿名函数有个限制，就是只能有一个表达式，不用写**return**，返回值就是该表达式的结果，因此当我们的函数代码只有一个表达式的时候，我们可以采用匿名函数来优化代码结构。

2.7 面向对象

面向对象编程——Object Oriented Programming，简称OOP，是一种程序设计思想。OOP把对象作为程序的基本单元，一个对象包含了数据和操作数据的函数。

面向过程的程序设计把计算机程序视为一系列的命令集合，即一组函数的顺序执行。为了简化程序设计，面向过程把函数继续切分为子函数，即把大块函数通过切割成小块函数来降低系统的复杂度。而面向对象的程序设计把计算机程序视为一组对象的集合，而每个对象都可以接收其他对象发过来的消息，并处理这些消息，计算机程序的执行就是一系列消息在各个对象之间传递。

在Python中，所有数据类型都可以视为对象，当然也可以自定义对象。自定义的对象数据类型就是面向对象中的类（Class）的概念。

我们以一个例子来说明面向过程和面向对象在程序流程上的不同之处。

假设我们要处理学生的成绩表，为了表示一个学生的成绩，面向过程的程序可以用一个dict表示：

```
std1 = { 'name': 'Michael', 'score': 98 }
std2 = { 'name': 'Bob', 'score': 81 }
```

而处理学生成绩可以通过函数实现，比如打印学生的成绩：

```
def print_score(std):
    print('%s: %s' % (std['name'], std['score']))
```

如果采用面向对象的程序设计思想，我们首选思考的不是程序的执行流程，而是Student这种数据类型应该被视为一个对象，这个对象拥有name和score这两个属性（Property）。如果要打印一个学生的成绩，首先必须创建出这个学生对应的对象，然后，给对象发一个print_score消息，让对象自己把自己的数据打印出来。

```
class Student(object):

    def __init__(self, name, score):
        self.name = name
        self.score = score

    def print_score(self):
        print('%s: %s' % (self.name, self.score))
```

给对象发消息实际上就是调用对象对应的关联函数，我们称之为对象的方法（Method）。面向对象的程序写出来就像这样：

```
ming = Student('Xiao_ming', 59)
hong = Student('Xiao_hong', 87)
```

```
ming.print_score()
hong.print_score()
```

面向对象的设计思想是从自然界中来的，因为在自然界中，类（Class）和实例（Instance）的概念是很自然的。Class是一种抽象概念，比如我们定义的Class——Student，是指学生这个概念，而实例（Instance）则是一个个具体的Student，比如，Xiao_ming和Xiao_hong是两个具体的Student。

所以，面向对象的设计思想是抽象出Class，根据Class创建Instance。面向对象的抽象程度又比函数要高，因为一个Class既包含数据，又包含操作数据的方法。数据封装、继承和多态是面向对象的三大特点。

2.7.1 类和实例

面向对象最重要的概念就是类（Class）和实例（Instance），必须牢记类是抽象的模板，比如Student类，而实例是根据类创建出来的一个个具体的“对象”，每个对象都拥有相同的方法，但各自的数据可能不同。

仍以Student类为例，在Python中，定义类是通过class关键字：

```
class Student(object):
    pass
```

class后面紧接着是类名，即Student，类名通常是大写开头的单词，紧接着是(object)，表示该类是从哪个类继承下来的，继承的概念我们后面再讲，通常，如果没有合适的继承类，就使用object类，这是所有类最终都会继承的类。

定义好了Student类，就可以根据Student类创建出Student的实例，创建实例是通过类名+()实现的：

```
>>> ming = Student()
>>> ming
<__main__.Student object at 0x10a67a590>
>>> Student
<class '__main__.Student'>
```

可以看到，变量ming指向的就是一个Student的实例，后面的0x10a67a590是内存地址，每个object的地址都不一样，而Student本身则是一个类。

可以自由地给一个实例变量绑定属性，比如，给实例ming绑定一个name属性：

```
>>> ming.name = 'Xiao_ming'
>>> ming.name
'Xiao_ming'
```

由于类可以起到模板的作用，因此，可以在创建实例的时候，把一些我们认为必须绑定的属性强制填写进去。通过定义一个特殊的`__init__`方法，在创建实例的时候，就把`name`，`score`等属性绑上去：

```
class Student(object):

    def __init__(self, name, score):
        self.name = name
        self.score = score
```

注意：特殊方法“`__init__`”前后分别有两个下划线！！

注意到`__init__`方法的第一个参数永远是`self`，表示创建的实例本身，因此，在`__init__`方法内部，就可以把各种属性绑定到`self`，因为`self`就指向创建的实例本身。

有了`__init__`方法，在创建实例的时候，就不能传入空的参数了，必须传入与`__init__`方法匹配的参数，但`self`不需要传，Python解释器自己会把实例变量传进去：

```
>>> ming = Student('Xiao_ming', 59)
>>> ming.name
'Xiao_ming'
>>> ming.score
59
```

和普通的函数相比，在类中定义的函数只有一点不同，就是第一个参数永远是实例变量`self`，并且，调用时，不用传递该参数。除此之外，类的方法和普通函数没有什么区别，所以，你仍然可以用默认参数、可变参数、关键字参数和命名关键字参数。

2.7.2 数据封装

面向对象编程的一个重要特点就是数据封装。在上面的`Student`类中，每个实例就拥有各自的`name`和`score`这些数据。我们可以通过函数来访问这些数据，比如打印一个学生的成绩：

```
>>> def print_score(std):
...     print('%s: %s' % (std.name, std.score))
```

```
...
>>> print_score(ming)
Xiao_ming: 59
```

但是，既然**Student**实例本身就拥有这些数据，要访问这些数据，就没有必要从外面的函数去访问，可以直接在**Student**类的内部定义访问数据的函数，这样，就把“数据”给封装起来了。这些封装数据的函数是和**Student**类本身是关联起来的，我们称之为类的方法：

```
class Student(object):

    def __init__(self, name, score):
        self.name = name
        self.score = score

    def print_score(self):
        print('%s: %s' % (self.name, self.score))
```

要定义一个方法，除了第一个参数是**self**外，其他和普通函数一样。要调用一个方法，只需要在实例变量上直接调用，除了**self**不用传递，其他参数正常传入：

```
>>> ming.print_score()
Xiao_ming: 59
```

这样一来，我们从外部看**Student**类，就只需要知道，创建实例需要给出**name**和**score**，而如何打印，都是在**Student**类的内部定义的，这些数据和逻辑被“封装”起来了，调用很容易，但却不用知道内部实现的细节。

封装的另一个好处是可以给**Student**类增加新的方法，比如**get_grade**：

```
class Student(object):

    ...

    def get_grade(self):
        if self.score >= 90:
            return 'A'
        elif self.score >= 60:
            return 'B'
        else:
            return 'C'
```

同样的，`get_grade`方法可以直接在实例变量上调用，不需要知道内部实现细节。

◆ 访问限制

在Class内部，可以有属性和方法，而外部代码可以通过直接调用实例变量的方法来操作数据，这样，就隐藏了内部的复杂逻辑。

但是，从前面Student类的定义来看，外部代码还是可以自由地修改一个实例的`name`、`score`属性：

```
>>> ming = Student('Xiao_ming', 59)
>>> ming.score
59
>>> ming.score = 99
>>> ming.score
99
```

如果要想让内部属性不被外部访问，可以把属性的名称前加上两个下划线`__`，在Python中，实例的变量名如果以`__`开头，就变成了一个私有变量（`private`），只有内部可以访问，外部不能访问，所以，我们把Student类改一改：

```
class Student(object):

    def __init__(self, name, score):
        self.__name = name
        self.__score = score

    def print_score(self):
        print('%s: %s' % (self.__name, self.__score))
```

改完后，对于外部代码来说，没什么变动，但是已经无法从外部访问实例变量`__name`和实例变量`__score`了：

```
>>> ming = Student('Xiao_ming', 59)
>>> ming.__name
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'Student' object has no attribute '__name'
```

这样就确保了外部代码不能随意修改对象内部的状态，这样通过访问限制的保护，代码更加安全。

但是如果外部代码要获取name和score怎么办？可以给Student类增加get_name和get_score这样的方法：

```
class Student(object):  
    ...  
  
    def get_name(self):  
        return self.__name  
  
    def get_score(self):  
        return self.__score
```

如果又要允许外部代码修改score怎么办？可以再给Student类增加set_score方法：

```
class Student(object):  
    ...  
  
    def set_score(self, score):  
        self.__score = score
```

你也许会问，原先那种直接通过ming.score = 99也可以修改啊，为什么要定义一个方法大费周折？因为在方法中，可以对参数做检查，避免传入无效的参数：

```
class Student(object):  
    ...  
  
    def set_score(self, score):  
        if 0 <= score <= 100:  
            self.__score = score  
        else:  
            raise ValueError('bad score')
```

需要注意的是，在Python中，变量名类似__xxx__的，也就是以双下划线开头，并且以双下划线结尾的，是**特殊变量**，特殊变量是可以直接访问的，不是private变量，所以，不能用__name__、__score__这样的变量名。

有些时候，你会看到以一个下划线开头的实例变量名，比如_name，这样的实例变量外部是可以访问的，但是，按照约定俗成的规定，当你看到这样的变量时，意思就是，“虽然我可以被访问，但是，请把我视为私有变量，不要随意访问”。

双下划线开头的实例变量是不是一定不能从外部访问呢？其实也不是。不能直接访问`__name`是因为Python解释器对外把`__name`变量改成了`_Student__name`，所以，仍然可以通过`_Student__name`来访问`__name`变量：

```
>>> ming._Student__name
'Xiao_ming'
```

但是强烈建议你不要这么干，因为不同版本的Python解释器可能会把`__name`改成不同的变量名。

总的来说就是，Python本身没有任何机制阻止你，一切全靠自觉。

最后注意下面的这种错误写法：

```
>>> ming = Student('Xiao_ming', 59)
>>> ming.get_name()
'Xiao_ming'
>>> ming.__name = 'New Name'  # 设置__name 变量！
>>> ming.__name
'New Name'
```

表面上看，外部代码“成功”地设置了`__name`变量，但实际上这个`__name`变量和class内部的`__name`变量不是一个变量！内部的`__name`变量已经被Python解释器自动改成了`_Student__name`，而外部代码给ming新增了一个`__name`变量。不信试试：

```
>>> ming.get_name()  # get_name()内部返回 self.__name
'Xiao_ming'          # 得到的依旧是原来的名字。
```

2.7.3 继承和多态

◆ 继承

在OOP程序设计中，当我们定义一个class的时候，可以从某个现有的class继承，新的class称为子类（Subclass），而被继承的class称为基类、父类或超类（Base class、Super class）。

比如，我们已经编写了一个名为Animal的class，有一个run()方法可以直接打印：

```
class Animal(object):
    def run(self):
        print('Animal is running...')
```


当我们需要编写Dog和Cat类时，就可以直接从Animal类继承：

```
class Dog(Animal):  
    pass  
  
class Cat(Animal):  
    pass
```

对于Dog来说，Animal就是它的父类，对于Animal来说，Dog就是它的子类。Cat和Dog类似。

继承的好处是子类获得了父类的全部功能。由于Animal实现了run()方法，因此，Dog和Cat作为它的子类，不需要特殊的代码，就自动拥有了run()方法：

```
dog = Dog()  
dog.run()  
  
cat = Cat()  
cat.run()
```

运行结果如下：

```
Animal is running...  
Animal is running...
```

当然，也可以对子类增加一些方法，比如Dog类：

```
class Dog(Animal):  
  
    def run(self):  
        print('Dog is running...')  
  
    def eat(self):  
        print('Eating meat...')
```

继承的第二个好处需要我们对代码做一点改进。你看到了，无论是Dog还是Cat，它们run()的时候，显示的都是Animal is running...，符合逻辑的做法是分别显示Dog is running...和Cat is running...，因此，对Dog和Cat类改进如下：

```
class Dog(Animal):  
  
    def run(self):
```

```
print('Dog is running...')

class Cat(Animal):

    def run(self):
        print('Cat is running...')
```

再次运行，结果如下：

```
Dog is running...
Cat is running...
```

当子类 and 父类都存在相同的`run()`方法时，我们说，子类的`run()`覆盖了父类的`run()`，在代码运行的时候，总是会调用子类的`run()`。

这样，我们就获得了继承的另一个好处：多态。

◆ 多态

要理解什么是多态，我们首先要对数据类型再作一点说明。当我们定义一个`class`的时候，我们实际上就定义了一种数据类型。我们定义的数据类型和Python自带的数据类型，比如`str`、`list`、`dict`没什么两样：

```
a = list()      # a 是 list 类型
b = Animal()    # b 是 Animal 类型
c = Dog()       # c 是 Dog 类型
```

判断一个变量是否是某个类型可以用`isinstance()`判断：

```
>>> isinstance(a, list)    # 判断 a 是否是 list 类型
True
>>> isinstance(b, Animal)
True
>>> isinstance(c, Dog)
True
```

看来`a`、`b`、`c`确实对应着`list`、`Animal`、`Dog`这3种类型。

但是等等，试试：

```
>>> isinstance(c, Animal)
True
```

看来c不仅仅是Dog，c还是Animal！

不过仔细想想，这是有道理的，因为Dog是从Animal继承下来的，当我们创建了一个Dog的实例c时，我们认为c的数据类型是Dog没错，但c同时也是Animal也没错，Dog本来就是Animal的一种！

所以，在继承关系中，如果一个实例的数据类型是某个子类，那它的数据类型也可以被看做是父类。但是，反过来就不行：

```
>>> b = Animal()
>>> isinstance(b, Dog)
False
```

Dog可以看成Animal，但Animal不可以看成Dog。要理解多态的好处，我们还需要再编写一个函数，这个函数接受一个Animal类型的变量：

```
def run_twice(animal):
    animal.run()
    animal.run()
```

当我们传入Animal的实例时，run_twice()就打印出：

```
>>> run_twice(Animal())
Animal is running...
Animal is running...
```

当我们传入Dog的实例时，run_twice()就打印出：

```
>>> run_twice(Dog())
Dog is running...
Dog is running...
```

当我们传入Cat的实例时，run_twice()就打印出：

```
>>> run_twice(Cat())
Cat is running...
Cat is running...
```

看上去没啥意思，但是仔细想想，现在，如果我们再定义一个Tortoise类型，也从Animal派生：

```
class Tortoise(Animal):
    def run(self):
```

```
print('Tortoise is running slowly...')
```

当我们调用`run_twice()`时，传入`Tortoise`的实例：

```
>>> run_twice(Tortoise())  
Tortoise is running slowly...  
Tortoise is running slowly...
```

你会发现，新增一个`Animal`的子类，不必对`run_twice()`做任何修改，实际上，任何依赖`Animal`作为参数的函数或者方法都可以不加修改地正常运行，原因就在于多态。

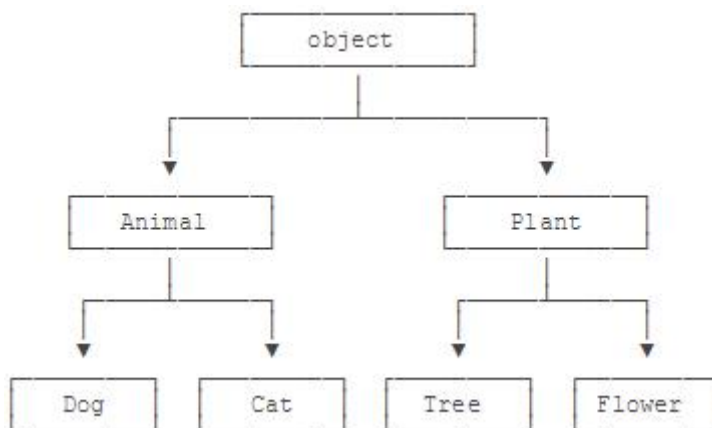
多态的好处就是，当我们需要传入`Dog`、`Cat`、`Tortoise`……时，我们只需要接收`Animal`类型就可以了，因为`Dog`、`Cat`、`Tortoise`……都是`Animal`类型，然后，按照`Animal`类型进行操作即可。由于`Animal`类型有`run()`方法，因此，传入的任意类型，只要是`Animal`类或者子类，就会自动调用实际类型的`run()`方法，这就是多态的意思：

对于一个变量，我们只需要知道它是`Animal`类型，无需确切地知道它的子类型，就可以放心地调用`run()`方法，而具体调用的`run()`方法是作用在`Animal`、`Dog`、`Cat`还是`Tortoise`对象上，由运行时该对象的确切类型决定，这就是多态真正的威力：调用方只管调用，不管细节，而当我们新增一种`Animal`的子类时，只要确保`run()`方法编写正确，不用管原来的代码是如何调用的。

这就是著名的“开闭”原则：

- 1) 对扩展开放：允许新增`Animal`子类；
- 2) 对修改封闭：不需要修改依赖`Animal`类型的`run_twice()`等函数。

继承还可以一级一级地继承下来，就好比从爷爷到爸爸、再到儿子这样的关系。而任何类，最终都可以追溯到根类`object`，这些继承关系看上去就像一颗倒着的树。比如如下的继承树：



2.7.4 实例属性和类属性

由于Python是动态语言，根据类创建的实例可以任意绑定属性。

给实例绑定属性的方法是通过实例变量，或者通过self变量：

```
class Student(object):
    def __init__(self, name):
        self.name = name

s = Student('Bob')
s.score = 90
```

但是，如果Student类本身需要绑定一个属性呢？可以直接在class中定义属性，这种属性是类属性，归Student类所有：

```
class Student(object):
    name = 'Student'
```

当我们定义了一个类属性后，这个属性虽然归类所有，但类的所有实例都可以访问到。来测试一下：

```
>>> class Student(object):
...     name = 'Student'
...
>>> s = Student() # 创建实例 s
>>> print(s.name) # 打印 name 属性，因为实例并没有 name 属性，所以会继续查找 class 的 name 属性
Student
>>> print(Student.name) # 打印类的 name 属性
Student
>>> s.name = 'Michael' # 给实例绑定 name 属性
>>> print(s.name) # 由于实例属性优先级比类属性高，因此，它会屏蔽掉类的 name 属性
Michael
>>> print(Student.name) # 但是类属性并未消失，用 Student.name 仍然可以访问
Student
>>> del s.name # 如果删除实例的 name 属性
>>> print(s.name) # 再次调用 s.name，由于实例的 name 属性没有找到，类的 name 属性就显示出来了
Student
```

在编写程序的时候，不要对实例属性和类属性使用相同的名字，因为相同名称的实例属性将屏蔽掉类属性，但是当你删除实例属性后，再使用相同的名称，访问到的将是类属性。

基础篇

Chapter 1



1.1 机器人的概念及特点

1.1.1 机器人的由来

机器人是众所周知的一种高新技术产品，然而，“机器人”一词最早并不是一个技术名词，而且至今尚未形成统一的、严格而准确的定义。机器人(Robot)一词最早由一位名叫Karel Capak的捷克斯洛伐克剧作家使用。在捷克语中，Robot这个词是指一个赋役的奴隶。在1921年Capak写了一出戏剧，名叫《洛桑万能机器人公司》(Rossum's Universal Robots)。在这出剧中，机器人是洛桑和他儿子研制的类人的生物，用来作为人类的奴仆。随后，一名叫Isaac Asimov的科学幻想小说家使用了机器人学(Robotics)这个词来描述与机器人有关的科学。他还提出了“机器人”的三个原则，值得今天的机器人设计者和使用者的注意。这三个原则的原语如下：

- 1: 机器人不得伤害人类，或看到人类受到伤害而袖手旁观；
- 2: 机器人必须服从人类的命令，除非这条命令与第一条相矛盾；
- 3: 机器人必须保护自己，除非这种保护与以上两条相矛盾。

实际上，真正能够代替人类进行生产劳动的机器人，是在20世纪60年代才问世的。伴随着机械工程、电气工程、控制技术以及信息技术等相关科技的不断发展，到20世纪80年代，机器人开始在汽车制造业、电机制造业等工业生产中大量采用。现在，机器人不仅在工业，而且在农业、商业、医疗、旅游、空间、海洋以及国防等诸多领域获得越来越广泛的应用。

1.1.2 机器人的定义

国际上，关于机器人的定义主要有如下几种：

(1) 英国简明牛津字典的定义。机器人是“貌似人的自动机，具有智力的和顺从于人的但不具人格的机器”。这一定义并不完全正确，因为还不存在与人类相似的机器人在运行。这是一种理想的机器人。

(2) 美国机器人协会的定义。机器人是“一种用于移动各种材料、零件、工具或专用装置的，通过可编程序动作来执行种种任务的，并具有编程能力的多功能机械手。尽管这一定义较实用些，但并不全面。这里指的是工业机器人。

(3) 日本工业机器人协会的定义。工业机器人是“一种装备有记忆装置和末端执行器的，能够转动并通过自动完成各种移动来代替人类劳动的通用机器”。

(4) 美国国家标准局(NBS)的定义。机器人是“一种能够进行编程并在自动控制下执行某些操作和移动作业任务的机械装置”。这也是一种比较广义的工业机器人定义。

(5) 国际标准化组织的定义。“机器人是一种自动的、位置可控的、具有编程能力的多功能机械手,这种机械手具有几个轴,能够借助于可编程序操作来处理各种材料、零件、工具和专用装置,以执行种种任务。显然,这一定义与美国机器人协会的定义相似。

关于我国机器人的定义。随着机器人技术的发展,我国也面临讨论和制订关于机器人技术的各项标准问题,其中包括对机器人的定义。蒋新松院士曾建议把机器人定义为“一种拟人功能的机械电子装置”。我国可以参考各国的定义,结合我国情况,对机器人作统一的定义。上述各种定义有共同之处,即认为机器人: a: 像人或人的上肢,并能模仿人的动作; b: 具有智力或感觉与识别能力; c: 是人造的机器或机械电子装置。

1.1.3 机器人的主要特点

1.通用性

机器人的通用性(**versatility**)取决于其几何特性和机械能力。通用性指的是某种执行不同的功能和完成多样的简单任务的实际能力。通用性也意味着,机器人具有可变的集合结构,即根据生产需要进行变更的集合结构;或者说,在机械结构上许机器人执行不同的任务或以不同的方式完成同一的工作。现有的大多数机器人都具有不同程度的通用性,包括机械手的机动性和控制系统的灵活性。必须指出,通用性不是自由度单独决定的。增加自由度一般能提高通用性程度。不过,还必须考虑其他因素,特别是末端装置的结构能力,如它们能否适应不同的工具等。

2.适应性

机器人的适应性(**adaptivity**)是指其对环境的自适应的能力,即所设计的机器人能够自我执行未经完全指定的任务,而不管任务执行过程中所发生的没有预计到的环境变化。这一能力要求机器人认识其环境,即具有人工知觉。在这方面,机器人使用它的下述能力(1)运用传感器测环境的能力;(2)分析任务空间和执行操作规划的能力;(3)自动指令模式能力。迄今为止所开发的机器人知觉与人类对环境的解释能力相比,仍然是十分有限的。这个领域内的某些重要研究工作正在进行之中。

1.2 机器人的组成与分类

1.2.1 机器人的组成

机器人是典型的机电一体化产品，一般由机械本体、控制系统、传感器、和驱动器等四部分组成。机械本体是机器人实施作业的执行机构。为对本体进行精确控制，传感器应提供机器人本体或其所处环境的信息，控制系统依据控制程序产生指令信号，通过控制各关节运动坐标的驱动器，使各臂杆端点按照要求的轨迹、速度和加速度，以一定的姿态达到空间指定的位置。驱动器将控制系统输出的信号变换成大功率的信号，以驱动执行器工作。

1. 机械本体

机械本体，是机器人赖以完成作业任务的执行机构，一般是一台机械手，也称操作器、或操作手，可以在确定的环境中执行控制系统指定的操作。典型工业机器人的机械本体一般由手部（末端执行器）、腕部、臂部、腰部和基座构成。机械手多采用关节式机械结构，一般具有6个自由度，其中3个用来确定末端执行器的位置，另外3个则用来确定末端执行装置的方向（姿势）。机械臂上的末端执行装置可以根据操作需要换成焊枪、吸盘、扳手等作业工具。

2. 控制系统

控制系统是机器人的指挥中枢，相当于人的大脑功能，负责对作业指令信息、内外环境信息进行处理，并依据预定的本体模型、环境模型和控制程序做出决策，产生相应的控制信号，通过驱动器驱动执行机构的各个关节按所需的顺序、沿确定的位置或轨迹运动，完成特定的作业。从控制系统的构成看，有开环控制系统和闭环控制系统之分；从控制方式看有程序控制系统、适应性控制系统和智能控制系统之分。

3. 驱动器

驱动器是机器人的动力系统，相当于人的心血管系统，一般由驱动装置和传动机构两部分组成。因驱动方式的不同，驱动装置可以分成电动、液动和气动三种类型。驱动装置中的电动机、液压缸、气缸可以与操作机直接相连，也可以通过传动机构与执行机构相连。传动机构通常有齿轮传动、链传动、谐波齿轮传动、螺旋传动、带传动等几种类型。

4. 传感器

传感器是机器人的感测系统，相当于人的感觉器官，是机器人系统的重要组成部分，包括内部传感器和外部传感器两大类。内部传感器主要用来检测机器人本身的状态，为机器人的运动控制提供必要的本体状态信息，如位置传感器、速度传感器等。外部传感器则用来感知机器人所处的工作环境或工作状况信息，又可分成环境传感器和末端执行器传感器两种类

型；前者用于识别物体和检测物体与机器人的距离等信息，后者安装在末端执行器上，检测处理精巧作业的感觉信息。常见的外部传感器有力觉传感器、触觉传感器、接近觉传感器、视觉传感器等。

1.2.2 机器人的分类

经过几十年的发展，机器人的技术水平不断提高，应用范围越来越广，从早期的焊接、装配等工业应用，逐步向军事、空间、水下、农业、建筑、服务和娱乐等领域不断扩展，结构形式也多种多样。因此，机器人的分类也出现了多种方法、多种标准，本章主要介绍以下三种分类法。

1. 按照机器人的技术发展水平

按照机器人的技术发展水平可以将机器人分为三代。

第一代机器人是“示教再现”型。这类机器人能够按照人类预先示教的轨迹、行为、顺序和速度重复作业。示教可以由操作员“手把手”地进行，比如，操作人员抓住机器人上的喷枪，沿喷漆路线示范一遍，机器人记住了这一连串运动，工作时，自动重复这些运动，从而完成给定位置的喷漆工作。这种方式即是所谓的“直接示教”。但是，比较普遍的方式是通过控制面板示教。操作人员利用控制面板上的开关或键盘来控制机器人一步一步地运动，机器人自动记录下每一步，然后重复。目前在工业现场应用的机器人大多属于第一代。

第二代机器人具有环境感知装置，能在一定程度上适应环境的变化。以焊接机器人为例，机器人焊接的过程一般是通过示教方式给出机器人的运动曲线，机器人携带焊枪走这个曲线，进行焊接。这就要求工件的一致性很好，也就是说工件被焊接的位置必须十分准确。否则，机器人走的曲线和工件上的实际焊缝位置会有偏差。为了解决这个问题，第二代机器人采用了焊缝跟踪技术，通过传感器感知焊缝的位置，再通过反馈控制，机器人就能够自动跟踪焊缝，从而对示教的位置进行修正，即使实际焊缝相对于原始设定的位置有变化，机器人仍然可以很好地完成焊接工作。类似的技术正越来越多地应用在机器人上。

第三代机器人称为“智能机器人”，具有发现问题，并且能自主地解决问题的能力。作为发展目标，这类机器人具有多种传感器，不仅可以感知自身的状态，比如所处的位置、自身的故障情况等等；而且能够感知外部环境的状态，比如自动发现路况、测出协作机器的相对位置、相互作用的力等等。更为重要的是，能够根据获得的信息，进行逻辑推理、判断决策，在变化的内部状态与变化的外部环境中，自主决定自身的行为。这类机器人具有高度的适应性和自治能力。尽管经过多年来的不懈研究，人们研制了很多各具特点的试验装置，提出大量新思想、新方法，但现有机器人的自适应技术还是十分有限的。

2. 按机器人的机构特征

机器人的机械配置形式多种多样，典型机器人的机构运动特征是用其坐标特性来描述的。按机构运动特征，机器人通常可分为直角坐标机器人、柱面坐标机器人、球面坐标机器人和关节型机器人等类型。

(1) 直角坐标机器人。如图1-1所示，直角坐标机器人具有空间上相互垂直的两根或三根直线移动轴，通过直角坐标方向的3个独立自由度确定其手部的空间位置，其动作空间为一长方体。直角坐标机器人结构简单，定位精度高，空间轨迹易于求解；但其动作范围相对较小，设备的空间因数较低，实现相同的动作空间要求时，机体本身的体积较大。主要用于印刷电路基板的元件插入、紧固螺丝等作业。

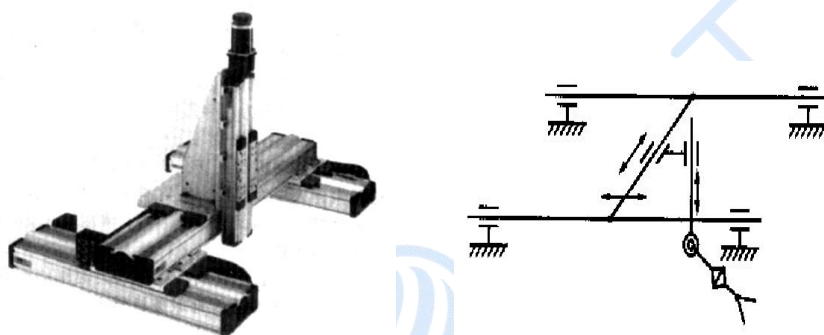


图1-1 直角坐标机器人

(2) 柱面坐标机器人。如图1-2所示，柱面坐标机器人的空间位置机构主要由旋转基座、垂直移动和水平移动轴构成，具有一个回转和两个平移自由度，其动作空间呈圆柱形。这种机器人结构简单、刚性好，但缺点是在机器人的动作范围内，必须有沿轴线前后方向的移动空间，空间利用率较低。主要用于重物的装卸、搬运等作业。

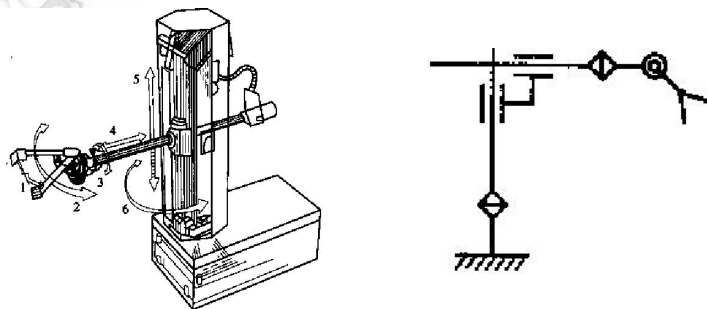


图1-2 柱面坐标机器人

(3) 球面坐标机器人。如图1-3所示，其空间位置分别由旋转、摆动和平移3个自由度确定，动作空间形成球面的一部分。其机械手能够作前后伸缩移动、在垂直平面上摆动以

及绕底座在水平面上转动。著名的Unimate就是这种类型的机器人。其特点是结构紧凑，所占空间体积小于直角坐标和柱面坐标机器人，但仍大于多关节型机器人。

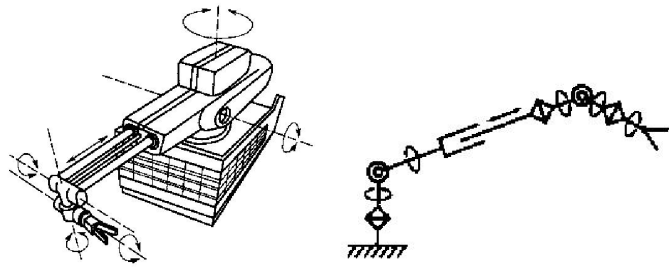


图1-3 球面坐标机器人

（4）多关节型机器人。由多个旋转和摆动机构组合而成。这类机器人结构紧凑、工作空间大、动作最接近人的动作，对喷漆、装配、焊接等多种作业都有良好的适应性，应用范围越来越广。不少著名的机器人都采用了这种型式，其摆动方向主要有铅垂方向和水平方向两种，因此这类机器人又可分为垂直多关节机器人和水平多关节机器人。

垂直多关节机器人（如图1-4）模拟了人类的手臂功能，由垂直于地面的腰部旋转轴（相当于大臂旋转的肩部旋转轴）带动小臂旋转的肘部旋转轴以及小臂前端的手腕等构成。手腕通常由2~3个自由度构成。其动作空间近似一个球体，所以也称多关节球面机器人。其优点是可以自由地实现三维空间的各种姿势，可以生成各种复杂形状的轨迹。相对机器人的安装面积，其动作范围很宽。缺点是结构刚度较低，动作的绝对位置精度磨较低。它广泛应用于代替人完成装配作业、货物搬运、电弧焊接、喷涂、点焊接等作业场合。

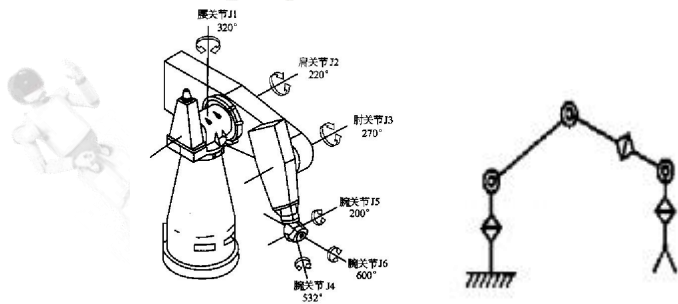


图 1-4 垂直多关节机器人

水平多关节机器人（如图1-5）在结构上具有串联配置的二个能够在水平面内旋转的手臂，其自由度可以根据用途选择2~4个，动作空间为一圆柱体。水平多关节机器人的优点是在垂直方向上的刚性好，能方便地实现二维平面上的动作，在装配作业中得到普遍应用。

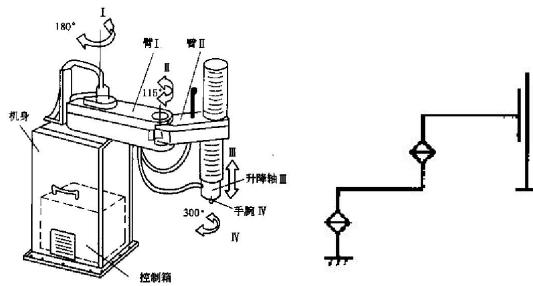


图 1-5 水平多关节机器人

3. 按照机器人的用途分

机器人首先在制造业大规模应用，所以，机器人曾被简单地分为两类，即用于汽车等制造业的机器人称为工业机器人，其他的机器人称为特种机器人。随着机器人应用的日益广泛，这种分类显得过于粗糙。现在除工业领域之外，机器人技术已经广泛地应用于农业、建筑、医疗、服务、娱乐，以及空间和水下探索等多种领域。

(1) 工业机器人

工业机器依据具体应用的不同，通常又可以分成焊接机器人、装配机器人、喷漆机器人、码垛机器人、搬运机器人等多种类型。焊接机器人，包括点焊（电阻焊）和电弧焊机器人，用途是实现自动的焊接作业。装配机器人，比较多地用于电子部件电器的装配。喷漆机器人，代替人进行喷漆作业。码垛、上下料、搬运机器人的功能则是根据一定的速度和精度要求，将物品从一处运到另一处。在工业生产中应用机器人，可以方便迅速地改变作业内容或方式，以满足生产要求的变化。比如，改变焊缝轨迹，改变喷漆位置，变更装配部件或位置等等。随着对工业生产线柔性的要求越来越高，对各种机器人的需求也就越来越强烈。

(2) 农业机器人

随着机器人技术的进步，以定型物、无机物为作业对象的工业机器人正在向更高层次的以动、植物之类复杂作业对象为目标的农业机器人发展，农业机器人或机器人化的农业机械的应用范围正在逐步扩大。农业机器人的应用不仅能够大大减轻以致代替人们的生产劳动、解决劳动力不足的问题，而且可以提高劳动生产率，改善农业的生产环境，防止农药、化肥等对人体的伤害，提高作业质量。但由于农业机器人所面临的是非结构、不确定、不宜预估的复杂环境和工作对象，所以与工业机器人相比，其研究开发的难度更大。农业机器人的研究开发目前主要集中耕种、施肥、喷药、蔬菜嫁接、苗木株苗移栽、收获、灌溉、养殖和各种辅助操作等方面。日本是机器人普及最广泛的国家，目前已经有数千台机器人应用于农业领域。

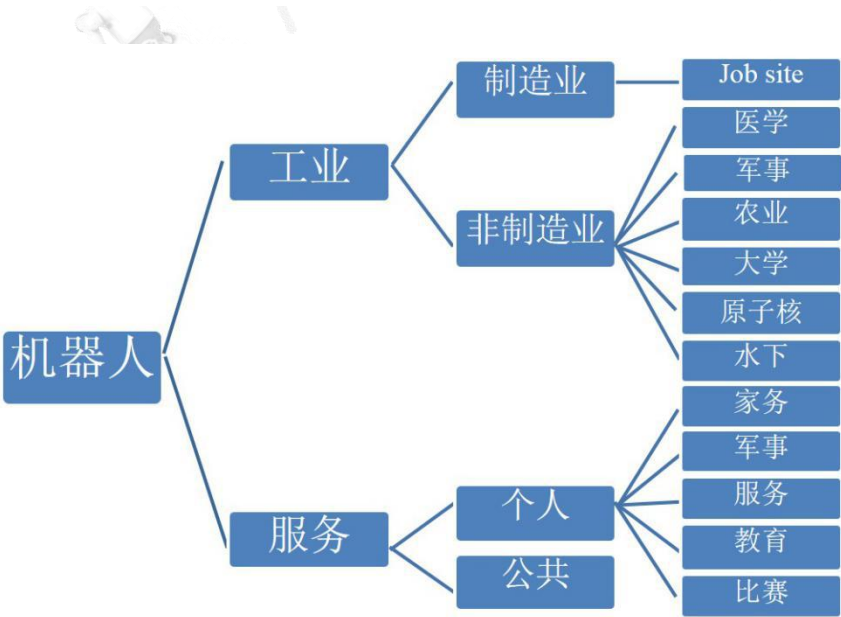
(3) 探索机器人

机器人除了在工农业上广泛应用之外，还越来越多地用于极限探索，即在恶劣或不适于人类工作的环境中执行任务。例如，在水下（海洋）、太空以及在放射性(有毒或高温等环境中进行作业。人类借助潜水器具潜人到深海之中探秘，已有很长的历史。然而，由于危险很大、费用极高，所以水下机器人就成了代替人在这一危险的环境中工作的最佳工具。空间机器人是指在大气层内和大气层外从事各种作业的机器人，包括在内层空间飞行并进行观测、可完成多种作业的飞行机器人，到外层空间其他星球上进行探测作业的星球探测机器人和在各种航天器里使用的机器人。

（4）服务机器人

机器人技术不仅在工农业生产、科学探索中得到了广泛应用，也逐渐渗透到人们的日常生活领域，服务机器人就是这类机器人的一个总称。尽管服务机器人的起步较晚，但应用前景十分广泛，目前主要应用在清洁、护理、执勤、救援、娱乐、和代替人对设备维护保养等场合。国际机器人联合会给服务机器人的一个初步定义是，一种以自主或半自主方式运行，能为人类的生活、康复提供服务的机器人，或者是能对设备运行进行维护的一类机器人。

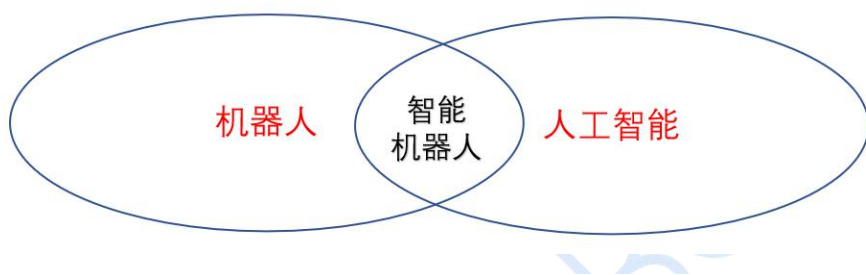
我国的机器人专家从应用环境出发，将机器人分为两大类，即工业机器人和特种机器人。所谓工业机器人就是面向工业领域的多关节机械手或多自由度机器人。而特种机器人则是除工业机器人之外的、用于非制造业并服务于人类的各种先进机器人，包括：服务机器人、水下机器人、娱乐机器人、军用机器人、农业机器人、机器人化机器等。在特种机器人中，有些分支发展很快，有独立成体系的趋势，如服务机器人、水下机器人、军用机器人、微操作机器人等。目前，国际上的机器人学者，从应用环境出发将机器人也分为两类：制造环境下的工业机器人和非制造环境下的服务与仿人型机器人，这和我国的分类是一致的。



1.3 智能机器人

1.3.1 智能机器人的定义

人工智能与机器人，前者主要解决学习、感知、语言理解或逻辑推理等任务，若想在物理世界完成这些工作，人工智能必然需要一个载体，机器人便是这样的一个载体。机器人是可编程机器，通常能够自主或者半自主的执行一系列动作。机器人与人工智能相结合，由人工智能算法程序控制的机器人称为智能机器人。



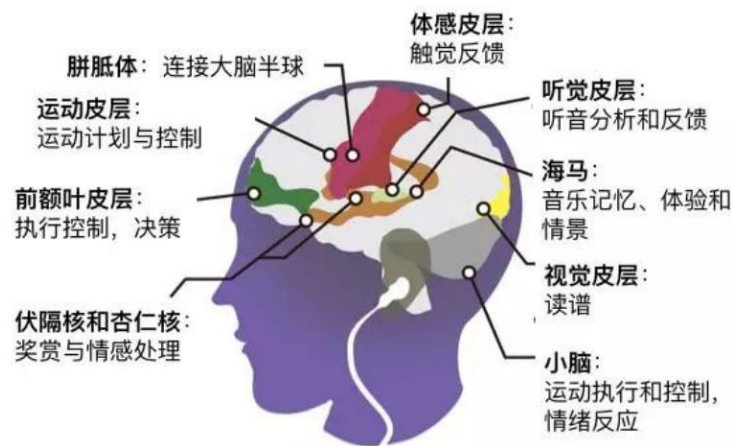
智能机器人在近几十年里发展迅速，代表性的工作包括：1988年日本东京电力公司研制的具有自动越障能力的巡检机器人；1944年中科院沈阳自动化所等单位研制成功的中国第一台无缆水下机器人“探索者”；1999年美国直觉外壳研制的达芬奇机器人手术系统；2000年本田研制的第一代仿人机器人阿西莫；2005年美国波士研制的四足大狗机器人、双足机器人atlas、2015年软银控股研制的情感机器人paper、2016年北京钢铁侠科技研制的artrobot双足仿人机器人。

1.3.2 脑科学与智能机器人

虽然人工智能科学发展得如火如荼，但总有一个界限无法逾越，这个界限就是人类大脑的思维能力，比如拥有随机搜索和卷积神经网络技术的阿尔法狗只是下棋的高手而已，其他能力连人类的婴儿都不如。全球科学家也逐步达成共识，即要想突破人工智能的技术壁垒，必须先要在脑科学领域有所建树。中国科学院谭铁牛院士曾说向生物学习，开展生物启发的模式识别有望实现模式识别理论与方法的新突破，达到对不同任务无缝切换、对环境自主适应、对知识凝练抽取等，是具有十分广阔的创新空间与发展前景。

脑是人类的决策、情感、感知、控制等中枢，是人体最为核心的组成部分，脑可大体分为大脑、小脑和脑干。大脑支配人的一切生命活动，如语言、运动、听觉、视觉、情感表达等；小脑主要的功能是协调骨骼肌的运动，维持和调节肌肉的紧张，保持身体的平衡。脑干

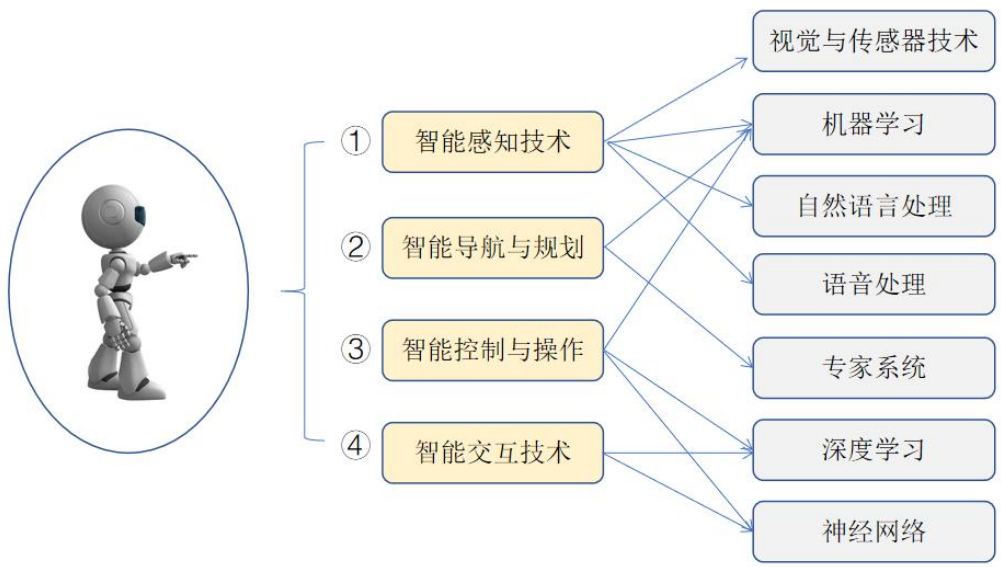
的主要是维持个体生命，包括心跳、呼吸、消化、体温、睡眠等重要生理功能。脑结构的具体划分，如下图所示：



人工智能是计算机科学的一个分支,它涉及如何创建计算机和计算机软件使之具有智能行为,主要解决学习、感知、语言理解或逻辑推理等任务,类似模拟实现人体大脑的功能。机器人工程是自动化学科的一个分支,是可编程机器,其通常能够自主地或半自主地执行一系列动作,类似实现人脑中运动皮层、小脑等与运动控制及规划相关的功能。

1.3.3人工智能在机器人中的应用

人工智能技术的应用提高了机器人的智能化程度,同时智能机器人的研究又促进了人工智能理论和技术的发展。智能机器人是人工智能技术的综合试验场,可以全面的检验考察人工智能各个研究领域的技术发展状况。如下图所示：



1. 智能感知技术

传感器是指能够感受被测量并按照一定规律转换成可用输出信息的器件或者装置，是机器人获取信息的重要源头，类似人的“五官”。从仿生学观点来看，如果把计算机看成是处理和识别信息的“大脑”，把通信系统堪称是传递信息的“神经系统”，那么传感器就是“感官系统”。

以下将重点介绍人工智能技术在机器人“视觉”、“触觉”、“听觉”三类最基本的感知模态中的应用。

（1）视觉在机器人中的应用

人类获取的70%的信息来自于视觉，因此，为机器人配备视觉系统是非常自然的想法。机器视觉可以通过视觉传感器获取环境图像，并通过处理器进行分析和解释，让机器能够辨识物体并确定其位置，以此来辅助机器人完成作业。类比人的视觉系统，摄像机等成像设备是机器的眼睛，而计算机视觉就是实现人类大脑（主要是视觉皮层）的视觉能力。

机器视觉的应用包括为机器人的动作控制提供视觉反馈、移动机器人的视觉导航、代替或帮助人工进行质量控制、安全检查所需的视觉检验以及无人驾驶等方向。

（2）触觉在机器人中的应用

触觉传感器是机器人中用于模仿触觉功能的传感器，如压力传感器、接近传感器等，触觉传感器对灵巧手的操作意义重大。在过去的三十年，人们一直尝试在灵巧手端加触觉传感器来提高抓取能力，但由于触觉传感器所传输的信息十分复杂且高纬度而导致灵巧手功效性较低。

近年来，随着传感器、控制和人工智能技术的发展，科研人员对触觉传感器所采集的信息结合不同的机器学习算法实现对灵巧手抓取物体的检测识别以及抓取稳定性分析等展开了研究。目前，主要通过机器学习中的聚类、分类等监督或无监督学习算法来完成触觉建模。

（3）听觉在机器人中的应用

人的耳朵同眼睛一样是重要的感觉器官，声波叩击耳膜，刺激听觉神经的冲动，之后传给大脑的听觉区形成人的听觉。

听觉传感器是一种可以检测、测量并显示声音波形的传感器，被广泛用于日常生活、军事、医疗、工业等众多领域，并且成为机器人发展必不可少的部分。声源定位、语音唤醒、语音合成、语音识别、语义交流、语音控制等技术使得人机交互更加智能，其中自然语言处理与语音处理技术起到了重要作用。

（4）机器学习在机器人传感器数据融合中的应用

随着传感器技术的迅速发展，各种不同的模态（如视、听、触）的动态数据正在以前所未有的速度涌现。对于一个待描述的目标或场景，通过不同的方法或视角收集到的是一个多模态的数据。通常把收集这些数据的每一种方法或视角称之为一个模态。多模态感知与学习这一问题与信号处理领域的“多源融合”“多传感器融合”以及机器学习领域的“多视学习”或“多视融合”等有密切联系。

机器人系统上配置的传感器复杂多样，从摄像头到激光雷达，从听觉到触觉，几乎所有的传感器在机器人上都有应用。但限于任务的复杂性、成本和使用效率等因素，目前市场上的机器人采用最多的任然是视觉和语音传感器，这两类模态一般独立处理（视觉用于目标检测、听觉用于语音交互）。综合考虑任务特性、环境特性和传感器特性，开展机器人多模态融合感知技术是一项重要任务。

2. 智能导航及规划

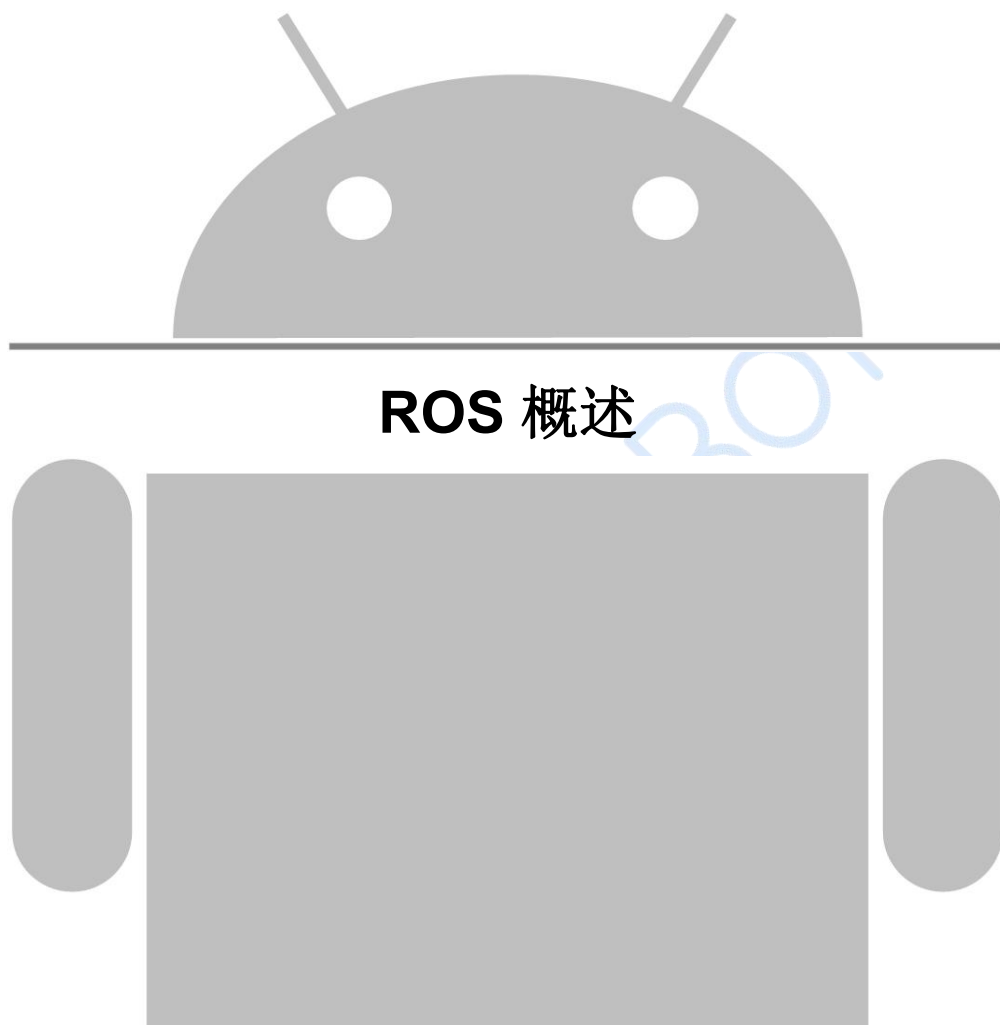
随着信息科学、计算机技术、人工智能及现代控制等技术的发展，人们尝试采用智能导航技术与规划的方式来解决机器人运行的安全问题，这既是作为机器人相关研究和开发的一项核心技术，同时也是机器人能够顺利完成各种服务和操作（如安全巡逻、物体抓取）的必要条件。

3. 智能控制与操作

4. 机器人智能交互

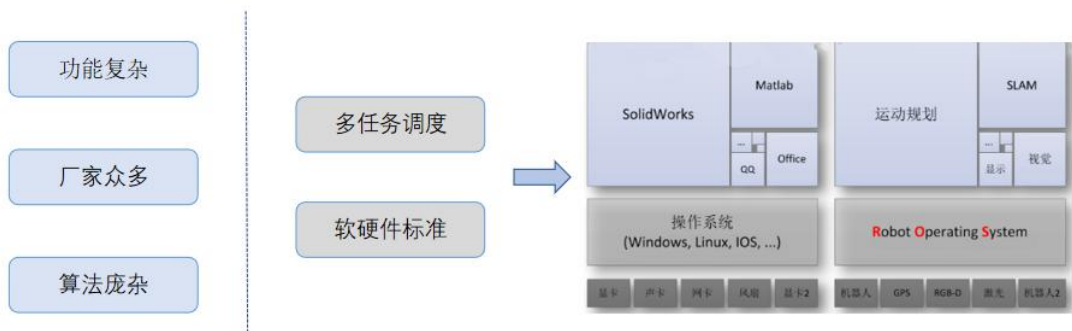
1.3.4 智能机器人发展展望

Chapter 2



2.1 初识ROS操作系统

近年来，机器人领域取得了举世瞩目的进展。性价比较高的机器人平台，包括地面移动机器人、旋翼无人机和类人机器人等，得到了广泛应用。更令人感到振奋的是，越来越多的高级智能算法让机器人的自主等级逐步提高。尽管如此，对于机器人软件开发人员来说，仍然存在着诸多挑战。例如，智能机器人研发面临功能复杂、硬件厂家众多，算法应用场景庞杂、软硬件标准不统一等问题。本书主要介绍一个软件平台，即机器人操作系统（Robot Operating System，或简称 ROS），它可以帮助提高机器人软件的开发效率。



ROS起源于斯坦福大学人工智能实验室与机器人技术公司Willow Garage合作的个人机器人项目，2008年后由Willow Garage维护。该项目研发的PR2机器人在ROS框架的基础上可以完成打台球、插插座、做早饭等惊人的功能，由此ROS引起了广泛的关注。2010年，Willow Garage正式开放ROS源代码，很快在机器人领域掀起了新的浪潮。



ROS，英文全称 Robot Operating System，即机器人操作系统，它提供了操作系统应有的服务，包括硬件抽象，底层设备控制，常用函数的实现，进程间消息传递，以及包管理。它也提供用于获取、编译、编写、和跨计算机运行代码所需的工具和库函数。ROS作为一个开源的软件系统，在某些方面ROS相当于一种“机器人框架（robot frameworks）”。宗旨是构建一个能够整合不同研究成果，实现算法发布、代码重用、的通用机器人软件平台，

其中包含一些列的工具、库和约定。同时，ROS还可以为异质计算机集群，提供了类似操作系统的中间件。很多开源的运动规划、定位导航、仿真、感知等软件功能包使得这一平台的功能变得更加丰富，发展更加迅速。到目前为止，ROS在机器人的感知、物体识别、脸部识别、姿势识别、运动、运动理解、结构与运动、立体视觉、控制、规划等多个领域都有相关应用。如下图所示：ROS由通信机制、开发工具、应用功能和生态系统四个部分组成。

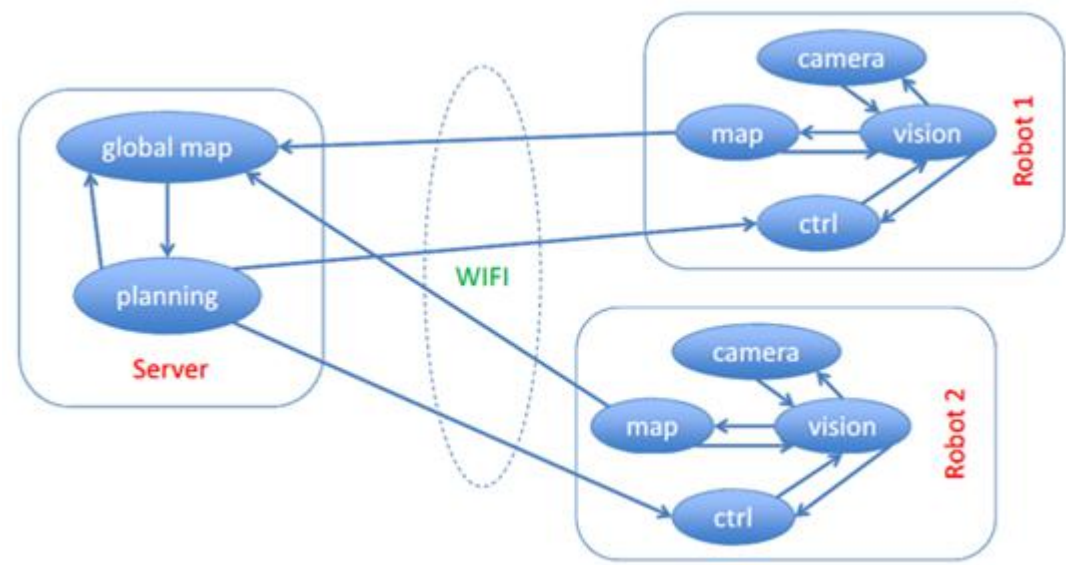


为了支持实现算法发布、代码重用等分享协作功能，ROS具有如下特点：

1) 分布式架构

一个使用ROS的系统包括一系列进程，这些进程存在于多个不同的主机并且在运行过程中通过端对端的拓扑结构进行联系。虽然基于中心服务器的那些软件框架也可以实现多进程和多主机的优势，但是在这些框架中，当各电脑通过不同的网络进行连接时，中心数据服务器就会发生问题。

ROS将每个工作进程都看作是一个节点，使用节点管理器进行统一管理。并提供了一套消息传递机制。可以分散由计算机视觉和语音识别等功能带来的实时计算压力，能够适应多机器人遇到的挑战。



2) 多语言支持

由于所有节点的通信都是通过网络套字节来实现的,这意味着只要能够提供套字节的接口,节点程序就可以使用任何编程语言来实现。ROS不依赖任何特定的编程语言,现在已经支持多种不同的语言实现,例如C++、Python、Octave和LISP,也包含其他语言的多种接口实现。ROS现在支持许多种不同的语言,例如C++、Python、Octave和LISP,也包含其他语言的多种接口实现。ROS采用了一种独立于编程语言的接口定义语言(IDL),并实现了多种编程语言对IDL的封装,使得各个不同编程语言编写的“节点”程序也能够透明的进行消息传递。

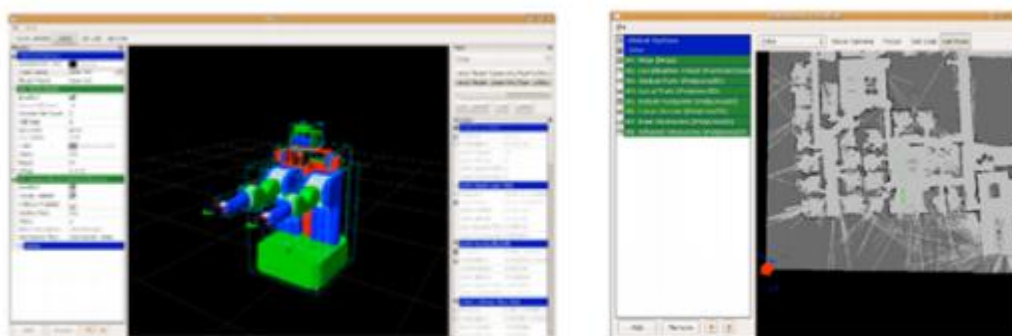


3) 良好的伸缩性

使用ROS进行机器人研发,既可以简单的编写一两个节点单独运行,又可以通过roscat、roslaunch将很多节点组成一个更大的工程,指定它们之间的依赖关系及运行时的组织形式。

4) 丰富的工具包

为了管理复杂的ROS软件框架,通过大量的小工具去编译和运行多种多样的ROS组件,从而设计成了内核,而不是构建一个庞大的开发和运行环境。这些工具担任了各种各样的任务,例如,组织源代码的结构,获取和设置配置参数,形象化端对端的拓扑连接,测量频带使用宽度,生动的描绘信息数据,自动生成文档等等。

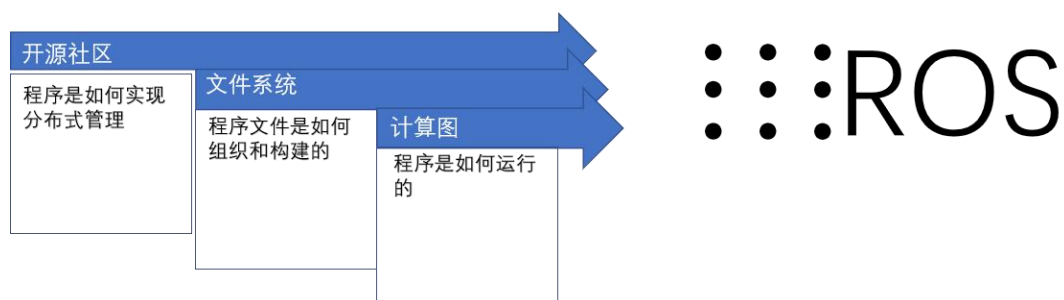


5) 免费且开源

ROS以分布式的关系遵循这BSD许可，也就是说允许各种商业和非商业的工程进行免费开发，这也是ROS得到广泛认可的原因之一。

2.2 ROS架构分析

ROS系统的架构主要被设计和划分为三部分，每一部分代表一个层级的概念：文件系统级(The filesystem level)、计算图级(The computation graph level)、开源社区级(The community level)。



- 文件系统级:用于描述可以在硬盘上查找到的代码及可执行程序。
- 计算图级：体现的是进程与进程、进程与系统之间的通信。
- 开源社区级：主要包括在开发人员之间如何共享知识、算法和代码。

1、文件系统级

在这一级，我们会使用一组概念来解释ROS的内部构成、文件结构、以及工作所需的核心文件。

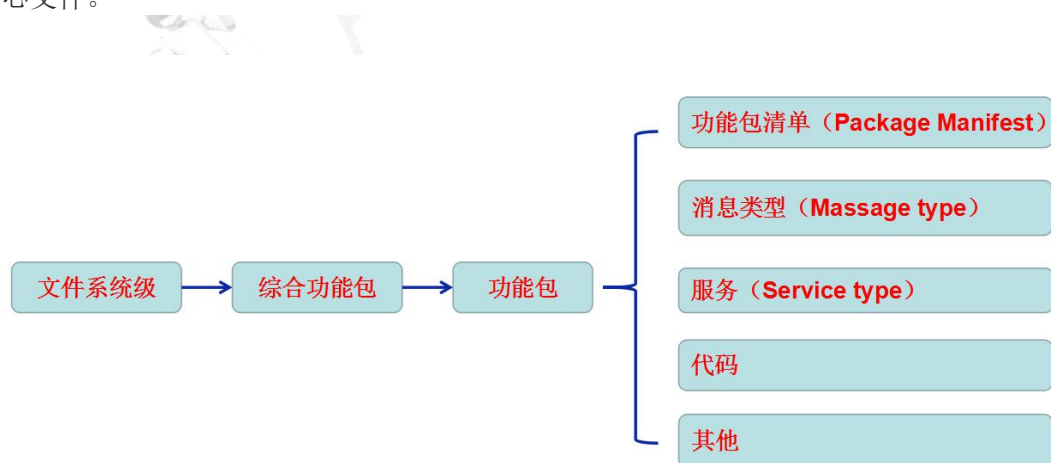


图 2-1 ROS文件系统结构

◆ **功能包(Package):** 功能包是ROS中软件组织的基本形式。一个功能包具有用于创建ROS程序的最小结构和最少内容。它可以包含ROS运行的进程（节点）、配置文件等；

◆ **功能包清单(Package Manifest):** 功能包清单提供关于功能包、许可信息、依赖关系、编译标志等的信息。一个包的清单有一个名为package.xml的文件管理；

◆ **综合功能包(Metapackage):** 将几个具有某些功能的功能包组织在一起，就获得了一个综合功能包(Stack)。

◆ **消息类型(Message type):** 消息是一个进程发送到其他进程的信息。ROS系统中有许多标准的消息类型，消息类型说明存储在my_package/msg/MyMessageType.msg中，即是在对应的功能包msg文件下。

◆ **服务类型 (Service type):** 服务类型定义了ROS中由每个进程提供服务请求和相应的数据结构。服务类型说明保存在对应功能包的srv文件夹下。

如下图所示为turtlesim功能包。

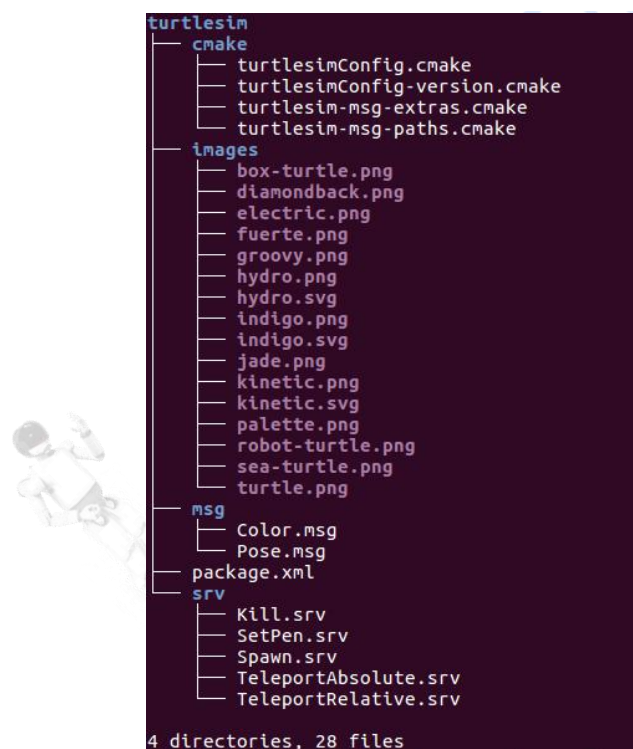


图 2-2 turtlesim功能包结构

2、ROS计算图级

这一级，体现的是进程和系统之间的通信，我们将学习ROS的各个概念和功能，包括建立系统、处理各类进程、与多台计算机通信等。

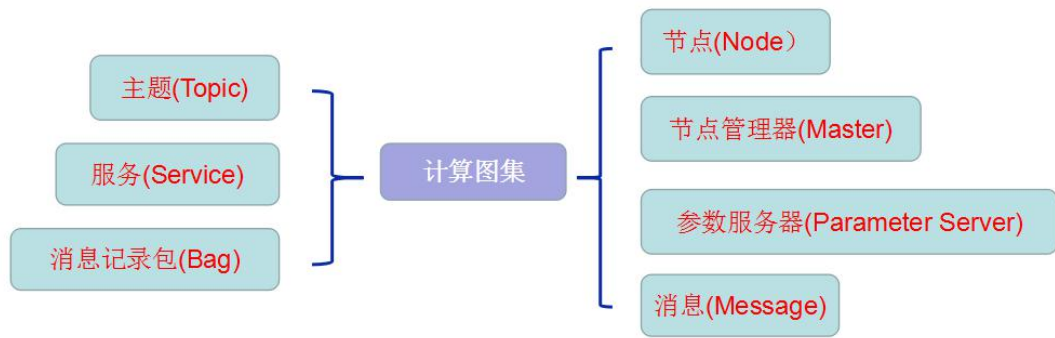


图 2-2 ROS计算图级

ROS会创建一个连接到所有进程的网络。在系统中的任何节点都可以访问此网络，并通过该网络与其他节点交互，获取其他节点发布的信息，并且将自身数据发布到网络上。如上图所示：

- ◆ **节点(Node)**：一个节点即为一个可执行文件，它可以通过ROS与其它节点进行通信；
- ◆ **节点管理器(Master)**：节点管理器用于节点的名称注册、查找、设置节点间通信等；
- ◆ **参数服务器(Parameter Server)**：参数服务器能够使数据通过一个关键词存储在一个系统的核心位置；

- ◆ **消息(Message)**：消息是节点之间沟通的一种方式，用于订阅或发布到一个话题；
- ◆ **话题(Topic)**：每个消息都必须有一个名称来被ROS网络路由。每一条消息都要发布到相应的话题。当一个节点发送数据时，我们就说该节点正在向话题发布消息。节点可以通过订阅某个话题，接受来自其他节点的消息。一个节点可以订阅一个话题，而不需要该节点同时发布该话题。这就保证了消息的发布者和订阅者间解耦。话题的名称是独一无二的，否则在同名话题之间的消息路由就会发生错误。简单说，节点可以发布消息到话题，也可以订阅话题以接收消息。

- ◆ **服务(Service)**：如果需要直接与某个节点进行交互request/response，此时不能通过Topic，而需要服务。服务必须有唯一的名称，当一个节点提供某个服务时，所有的节点都可以通过使用ROS客户端边写的代码与它通信。

- ◆ **消息记录包(Bag)**：消息记录包是一种用于保存和回放ROS消息数据的文件格式。消息记录包是一种用于存储数据的主要机制，它能够获取并记录各种难以收集的传感器数据。可以通过消息记录包反复获取实验数据，进行必要的开发和算法测试。

3、开源社区级

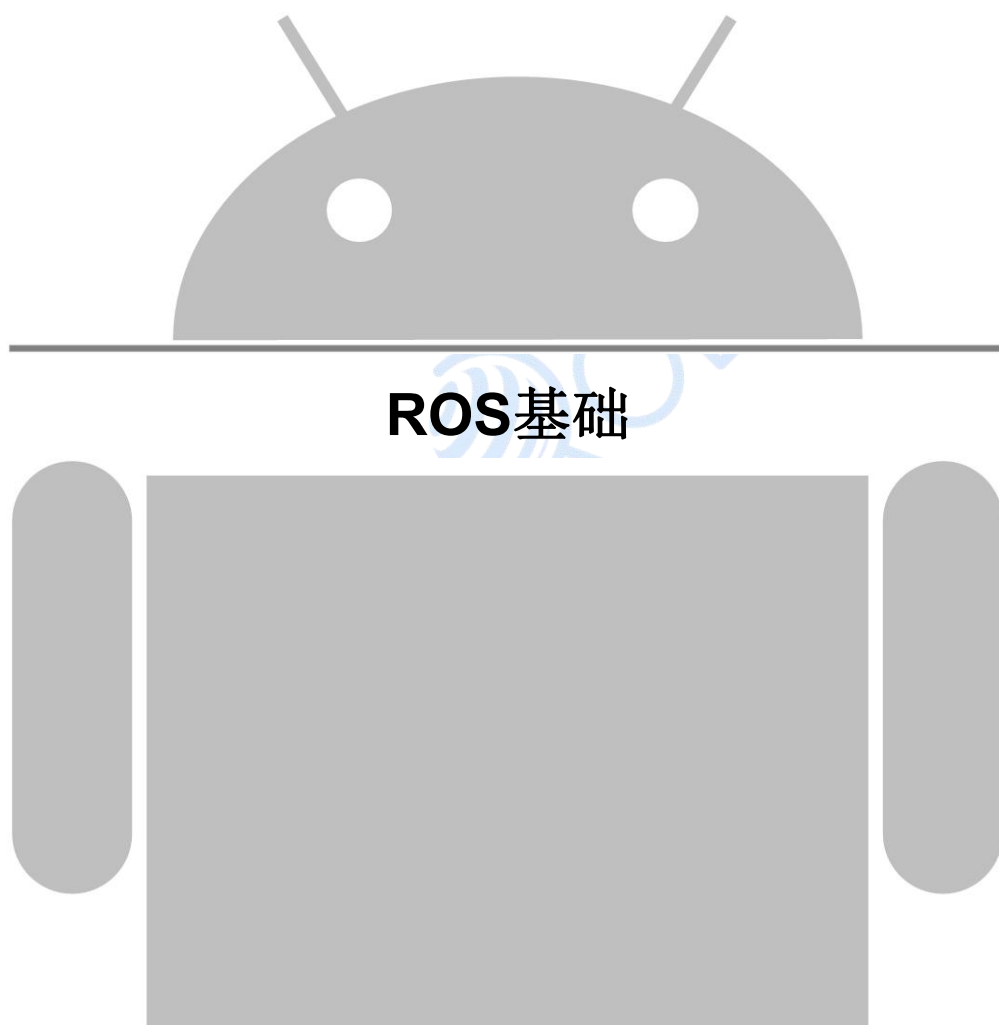
这一级将解释一系列的工具和概念，其中包括在开发人员之间如何共享知识、算法、代码。这个层级非常重要，正是由于开源社区的大力支持，ROS系统才得以快速发展。

这一级主要包括：发行版（Distribution）、软件库（Repository）、ROS维基（ROS Wiki）、右键列表（MAiling list）、ROS问答（ROS Answer）、博客（Blog）--www.ros.org/news。

2.3 ROS通信机制



Chapter 2



上一章对ROS进行一个简单的概述，基本了解ROS的一个框架结构，本章将深入去学习ROS操作系统的基础内容，包括ROS系统环境搭建，Turtlesim实例的使用，节点和功能包的创建与编译，ROS的通行机制等。

2.1 搭建ROS操作系统

我们使用一台已经安装了Ubuntu 16.04的电脑(或者虚拟机)来安装ROS，对于Ubuntu的操作就需要你具有一定的Linux基础，例如常用命令、文件操作、vim等。

2.1.1 ROS 版本的选择

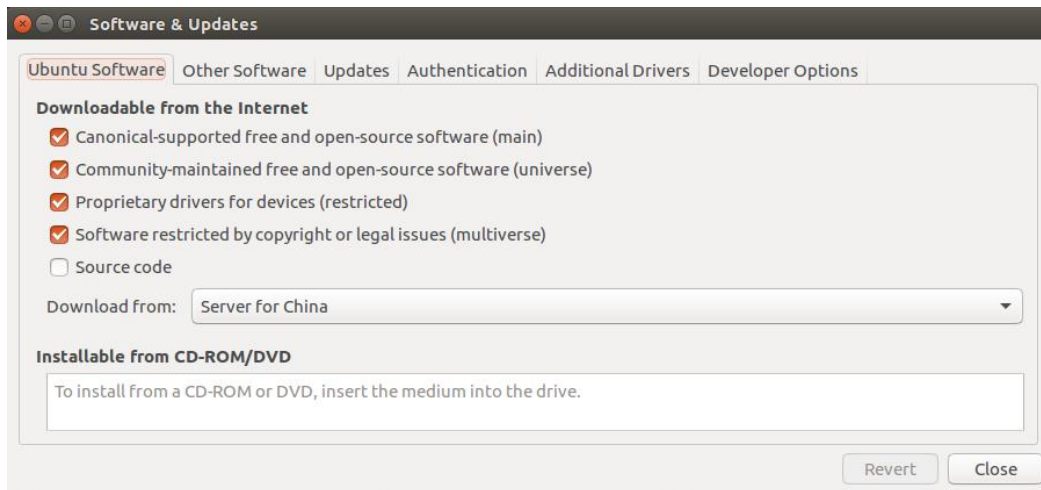
ROS操作系统基于Unix平台运行。ROS的软件主要在Ubuntu和Mac OS X 系统上测试，同时ROS社区仍持续支持Fedora, Gentoo, Arch Linux和其它Linux平台。到目前为止ROS 兼容性最好的当属 Ubuntu 操作系统了。

Ubuntu 和 ROS 都是存在不同的版本的，而且ROS各个版本之间互不兼容，所以每一个 ROS 版本都对应着一个或两个对应的 Ubuntu 版本。

ROS发布日期	ROS版本	对应Ubuntu版本
2016.3	ROS Kinetic Kame	Ubuntu 16.04
2015.3	ROS Jade Turtle	Ubuntu14.04
2014.3	ROS Indigo Igloo	Ubuntu14.04
2013.9	ROS Hydro Medusa	Ubuntu12.04
2012.12	ROS Groovy Galapagos	Ubuntu12.04

2.1.2 配置 Ubuntu 软件仓库

配置你的 Ubuntu 软件仓库(repositories) 以允许 "restricted"、"universe" 和 "multiverse"这三种安装模式。



2.1.3 添加 sources.list

<1> 配置你的电脑使其能够安装来自 packages.ros.org 的软件包。打开一个控制台(Ctrl + Alt + T)，输入如下指令：

```
$ sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
```

更多的源可以参考以下链接：

<http://wiki.ros.org/ROS/Installation/UbuntuMirrors>

建议使用对应地区的镜像源，这样可以大大提高安装下载速度。

<2> 添加秘钥：

```
$ sudo apt-key adv --keyserver hkp://pool.sks-keyservers.net --recv-key 421C365BD9FF1F717815A3895523BAEEB01FA116
```

2.1.4 更新软件包

打开一个控制台(Ctrl + Alt + T)，输入如下指令：

```
$ sudo apt-get update
```

ROS中有很多各种函数库和工具，我们提供了四种默认安装方式，你也可以单独安装某个特定软件包。

◆ 桌面完整版安装：（推荐） 包含ROS、rqt、rviz、通用机器人函数库、2D/3D仿真器、导航以及2D/3D感知功能。

```
$ sudo apt-get install ros-kinetic-desktop-full
```

- ◆ 桌面版安装： 包含ROS、rqt、rviz以及通用机器人函数库。

```
$ sudo apt-get install ros-kinetic-desktop
```

- ◆ 基础版安装： 包含ROS核心软件包、构建工具以及通信相关的程序库，无GUI工具。

```
$ sudo apt-get install ros-kinetic-ros-base
```

- ◆ 单个软件包安装： 你也可以安装某个指定的ROS软件包（使用软件包名称替换掉下面的PACKAGE）：

```
$ sudo apt-get install ros-kinetic-PACKAGE
```

我们选择安装桌面完整版。

2.1.5 初始化 rosdep

在开始使用ROS之前你还需要初始化rosdep。rosdep可以方便在你需要编译某些源码的时候为其安装一些系统依赖，同时也是某些ROS核心功能组件所必需用到的工具。

```
$ sudo rosdep init
$ rosdep update
```

2.1.6 环境配置

ROS安装到系统中，其默认路径在 /opt下，为了使用更加方便，需要将ROS环境变量自动配置好（即添加到bash会话中），这样每次打开一个新的终端时，ROS环境变量都能够直接使用，配置命令如下：

```
$ echo "source /opt/ros/kinetic/setup.bash" >> ~/.bashrc
$ source ~/.bashrc
```

2.1.7 构建工厂依赖

此时ROS系统已经基本安装完成，为了创建和管理ROS工作区，ROS中提供很多工具。其中 rosinstall 就是 ROS 中常用的一个命令行工具，它主要是方便为ROS软件包下载和安装其依赖的源码树。

要在ubuntu上安装这个工具，请运行：

```
$ sudo apt-get install python-rosinstall python-rosinstall-generator python-wstool build-essential
```

至此一个完整的ROS就安装完成了。

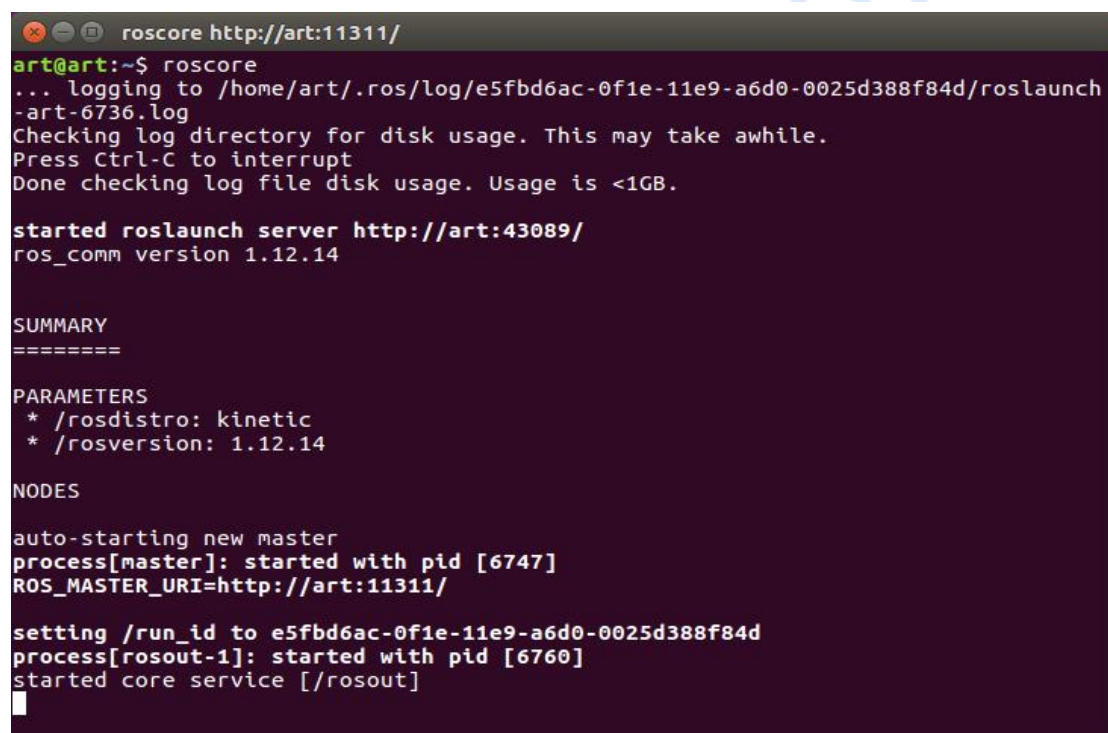
2.2 第一个ROS例程

第一节我们已经安装了ROS系统，下面我们将通过运行一个例子测试ROS是否安装正确。此例子是用ROS系统提供的turtlesim功能包在屏幕上显示ROS系统自带的小乌龟，并通过键盘控制它的一个节点，实现小乌龟的移动。本节中会使用许多ROS术语，如节点、功能包和roscore等，我们将在随后的章节中依次介绍。

下面我们将演示turtlesim功能包的功能，确保ROS安装没有问题。

<1> 启动ROS环境：

```
$ roscore
```



```
art@art:~$ roscore
... logging to /home/art/.ros/log/e5fbd6ac-0f1e-11e9-a6d0-0025d388f84d/roslaunch-art-6736.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://art:43089/
ros_comm version 1.12.14

SUMMARY
=====
PARAMETERS
* /rostdistro: kinetic
* /rosversion: 1.12.14

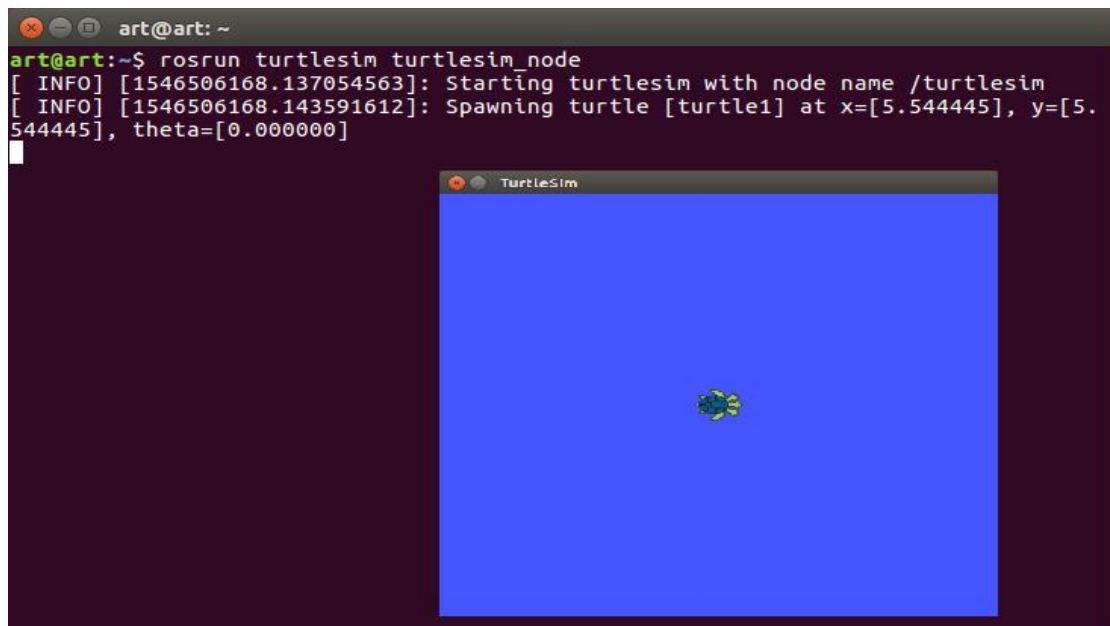
NODES

auto-starting new master
process[master]: started with pid [6747]
ROS_MASTER_URI=http://art:11311/

setting /run_id to e5fbd6ac-0f1e-11e9-a6d0-0025d388f84d
process[rosout-1]: started with pid [6760]
started core service [/rosout]
```

<2> 打开第二个终端，启动turtlesim：

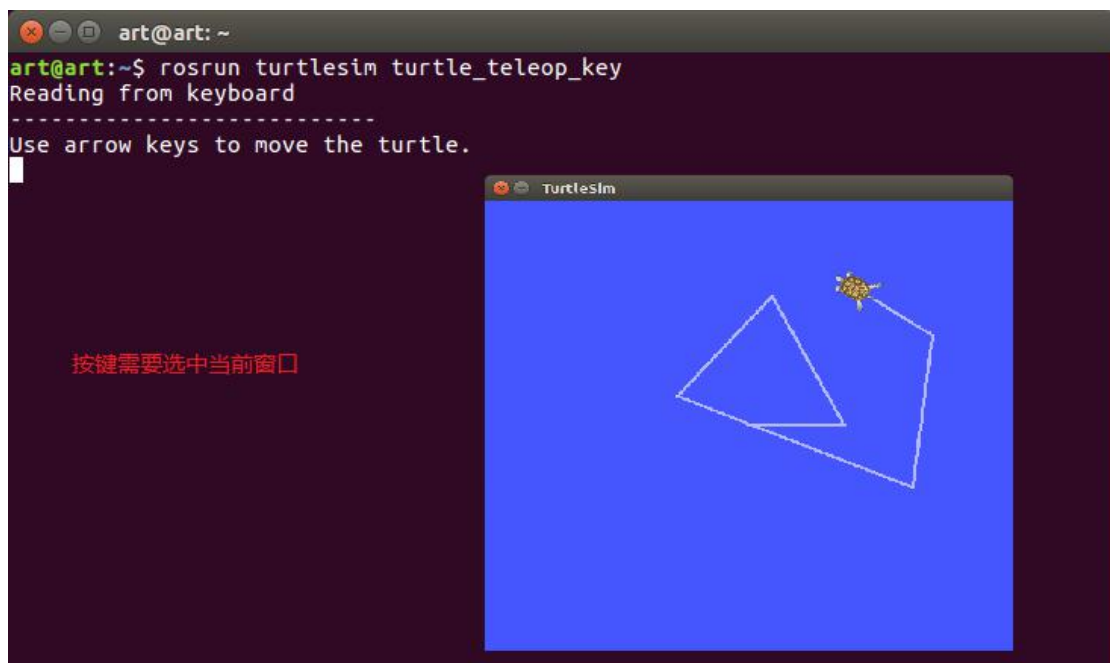
```
$ rosrun turtlesim turtlesim_node
```



如果一切正常，就能出现小乌龟画面了。

<3> 打开第三个终端，通过键盘控制turtle:

```
$ roslaunch turtlesim turtle_teleop_key
```



用鼠标激活第三个终端，按下键盘的上下、左右按键移动和转动小海龟。如果不能控制，请选中turtle_teleop_key所在的终端窗口以确保你的按键输入能够被捕获。如果成功，则基本上说明ros-kinetic安装成功了！

2.3 文件系统工具

程序代码是分布在众多ROS软件包当中，当使用命令行工具（比如ls和cd）来浏览时会非常繁琐，因此ROS提供了专门的命令工具来简化这些操作。

2.3.1 使用rospack

当ROS安装包越来越多时，该命令就会非常有用，尤其是当安装包有上百个之多时，根本无法准确记清各个软件包的路径或是否存在某软件包等。

rospack允许你获取软件包的有关信息，例如，rospack的find参数选项，可直接返回软件包的路径信息。用法：

```
$ rospack find [包名称]
```

示例：

```
$ rospack find roscpp
/opt/ros/kinetic/share/roscpp
```

上面示例可以看到rospack对我们的使用提供了很大的便捷。如果要查询rospack下的所有参数，可运行rospack help 命令。

```
USAGE: rospack <command> [options] [package]
Allowed commands:
  help
  cflags-only-I      [--deps-only] [package]
  cflags-only-other  [--deps-only] [package]
  depends            [package] (alias: deps)
  depends-indent     [package] (alias: deps-indent)
  depends-manifests  [package] (alias: deps-manifests)
  depends-msgsrv     [package] (alias: deps-msgsrv)
  depends-on         [package]
  depends-on1        [package]
  depends-why --target=<target> [package] (alias: deps-why)
  depends1           [package] (alias: deps1)
  export [--deps-only] --lang=<lang> --attrib=<attrib> [package]
  find [package]  查询软件包的路径信息
  langs
  libs-only-L        [--deps-only] [package]
  libs-only-l        [--deps-only] [package]
  libs-only-other    [--deps-only] [package]
  list  列出当前系统中安装的软件包名及其路径信息
  list-duplicates
  list-names  只列出当前系统中安装的软件包名称
  plugins --attrib=<attrib> [--top=<toppkg>] [package]
  profile [--length=<length>] [--zombie-only]
  rosdep [package] (alias: rosdeps)
  rosdep0 [package] (alias: rosdeps0)
  vcs [package]
  vcs0 [package]
Extra options:
  -q      Quiets error reports.

If [package] is omitted, the current working directory
is used (if it contains a package.xml or manifest.xml).
```

图 2-2 ROS计算图级

我们常使用的有find、list、list-names三个命令，大家可以分别尝试运行体验。

2.3.2 使用 roscd

roscd是rosbash命令集中的一部分，它允许你直接切换(cd)工作目录到某个软件包或者软件包集中。相当于ros + cd。用法：

```
$ roscd [本地包名称[/子目录]]
```

示例：

```
$ roscd roscpp
/opt/ros/kinetic/share/roscpp$
```

注意：roscd只能切换到那些路径已经包含在ROSPACKAGEPATH环境变量中的软件包，要查看ROSPACKAGEPATH中包含的路径可以使用命令：

```
$ echo $ROS_PACKAGE_PATH
/home/silver/catkin_ws/src:/opt/ros/kinetic/share
```

ROSPACKAGEPATH环境变量应该包含那些保存有ROS软件包的路径，并且每个路径之间用冒号分隔开来。

2.3.3 使用 rosls

rosls是rosbash命令集中的一部分，它允许你直接按软件包的名称而不是绝对路径执行ls命令（罗列目录）。用法：

```
$ rosls [本地包名称[/子目录]]
```

示例：

```
$ rosls roscpp_tutorials
cmake launch package.xml srv
```

2.3.4 使用 rosed

rosed是rosbash命令集中的一部分，它允许你直接编辑某个软件包中的可编辑文件相当于ros + vim。用法：

```
$ rosed [本地包名称[/子目录]]
```

示例：

```
$ rosls speed.cpp
```

2.3.5 使用 roscp

roscd是roscd命令集中的一部分，它允许你直接复制某个文件到指定目录下。相当于ros + cp。用法：

```
$ roscp [本地包名称]
```

示例：

```
$ roscp speed.cpp /src
```

2.4 创建工作空间

工作空间就是一个包含功能包、可编辑源文件、或编译包的文件夹。当你想同时编译不同的功能包时非常有用，并且可以用来保存本地开发包。

2.4.1 创建工作空间

<1> 将要创建的工作空间文件夹是在~/catkin_ws/src/中。若新创建则使用下面命令。

```
$ mkdir -p ~/catkin_ws/src/  
$ cd ~/catkin_ws/src/  
$ catkin_init_workspace #初始化工作空间
```

<2> 创建完工作空间文件夹后，里面并没有问的文件夹，只有CMakeList.txt。下一步是编译工作空间，使用下面的命令：

```
$ cd ~/catkin_ws  
$ catkin_make
```

编译完成后，查看catkin_ws文件目录，可以看到上面的编译命令创建了build和devel文件夹。

<3> 使用命令，完成配置

```
~/catkin_ws$ source devel/setup.bash
```

<4> 添加程序包到全局路径并使之生效（如果已经添加过则忽略此步骤）

```
$ echo "source ~/catkin_ws/devel/setup.bash" >> ~/.bashrc
$ source ~/.bashrc
```

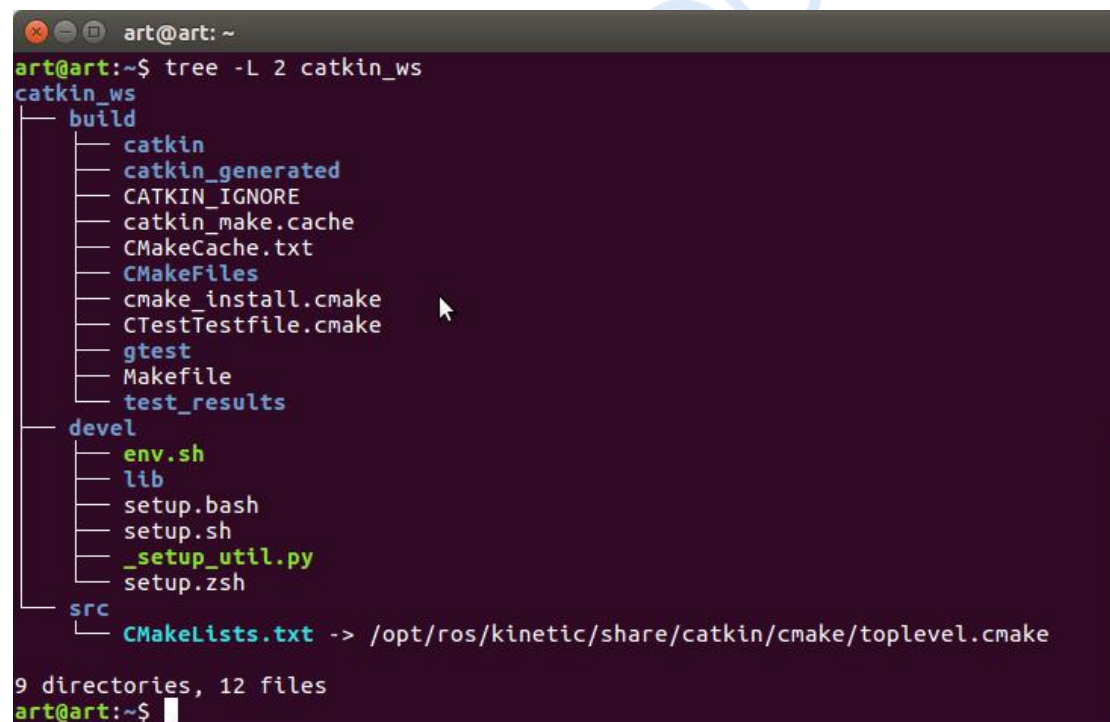
要想保证工作空间已配置正确需确保ROS_PACKAGE_PATH环境变量包含你的工作空间目录，采用以下命令查看：

```
$ echo $ROS_PACKAGE_PATH
/home/<youruser>/catkin_ws/src:/opt/ros/kinetic/share
```

到此工作环境已经搭建完成，接下来就可以创建功能包了。

2.5.2 工作空间目录解析

使用linux下的tree树状目录结构打开我们创建的catkin_ws工作空间（如下图所示），可以看到该工作空间下包含build、devel和src三个文件夹，有时候编译也会出现更多的信息，但是这三个文件夹是catkin编译系统默认的，每个文件夹都是一个具有不同功能的空间。



```
art@art: ~
art@art:~$ tree -L 2 catkin_ws
catkin_ws
├── build
│   ├── catkin
│   ├── catkin_generated
│   ├── CATKIN_IGNORE
│   ├── catkin_make.cache
│   ├── CMakeCache.txt
│   ├── CMakeFiles
│   ├── cmake_install.cmake
│   ├── CTestTestfile.cmake
│   ├── gtest
│   ├── Makefile
│   └── test_results
├── devel
│   ├── env.sh
│   ├── lib
│   ├── setup.bash
│   ├── setup.sh
│   ├── _setup_util.py
│   └── setup.zsh
└── src
    └── CMakeLists.txt -> /opt/ros/kinetic/share/catkin/cmake/toplevel.cmake

9 directories, 12 files
art@art:~$
```

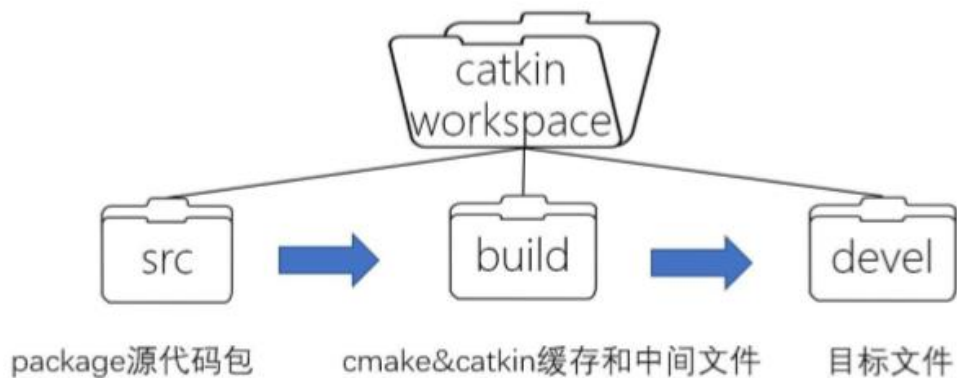
图 2-2 111

◆ **源文件空间（The Source space）**：在源空间（src 文件夹）放置了功能包、项目、克隆包等，这个空间最重要的是 CMakeLists.txt。当你在工作空间中配置功能包时，src 文件夹 CMakeList.txt 调用 make。这个文件是通过 `catkininitworkspace` 命令创建的。

◆ **编译空间 (The Build space)**：在build文件夹里，CMake和catkin为功能包和项目保存缓存信息、配置、和其他中间文件。

◆ **开发空间 (The Development space)**：devel 文件夹用来保存编译后的程序，编译生成的目标文件包括头文件，动态链接库，静态链接库，可执行文件等环境变量，这些无需安装就能用来测试的程序。一旦通过测试，就可以安装或者导出功能包与其他开发人员分享。

其编译流程如下：



catkin编译包有两个选项，一个是使用标准的CMake工作流程，通过此方法编译一个包：

```
$ cmake packageToBuild/  
$ make
```

如果要编译所有的包，可以使用catkin_make命令行：

```
$ cd workspace(工作空间目录)  
$ catkin_make
```

以上就是catkin工作空间的基本结构，package是catkin工作空间的基本单元，在做ROS开发时，代码完成后，只需要catkin_make，系统就会完成所有编译构建的工作。

2.5 功能包的创建与编译

开发catkin程序包的一个推荐方法是使用catkin工作空间，但是你也可以单独开发(standalone)catkin 软件包。本部分教程将演示如何使用catkin_create_pkg命令来创建一个新的catkin 程序包以及创建之后都能做些什么。

2.6.1 catkin功能包的组成

一个程序包要想称为catkin程序包必须符合以下要求：

- 1、该程序包必须包含package.xml文件，这个文件提供有关程序包的元信息。
- 2、程序包必须包含一个catkin版本的CMakeLists.txt文件。
- 3、每个目录下只能有一个程序包。

2.6.2 创建程序包

首先切换到之前通过创建catkin工作空间教程创建的catkin工作空间中的src目录下：

```
art@art:~$ cd ~/catkin_ws/src/  
art@art:~/catkin_ws/src$ ls  
CMakeLists.txt
```

现在使用catkin_create_pkg命令来创建一个名为'test_tutorials'的新程序包，这个程序包依赖于std_msgs、roscpp和rospy：

```
art@art:~/catkin_ws/src$ catkin_create_pkg test_tutorials std_msgs roscpp rospy  
Created file test_tutorials/CMakeLists.txt  
Created file test_tutorials/package.xml  
Created folder test_tutorials/include/test_tutorials  
Created folder test_tutorials/src  
Successfully created files in /home/art/catkin_ws/src/test_tutorials. Please adjust the values in  
package.xml.  
art@art:~/catkin_ws/src$ ls  
CMakeLists.txt  test_tutorials
```

由此可见，创建程序包的命令格式为：

```
catkin_create_pkg <package_name> [depend1] [depend2] [depend3]
```

catkin_create_pkg命令默认必须输入package_name，同时可以添加该程序包用到的依赖项，通过catkin_create_pkg命令创建test_tutorials文件夹里面包含一个package.xml文件和一个CMakeLists.txt文件，这两个文件都已经自动包含了部分你在执行catkin_create_pkg命令时提供的信息。

以下是创建程序包的几个关键信息：

std_msgs ： 包含了常见消息类型，表示基本数据类型和其他基本的消息构造类型

roscpp :使用c++实现了ROS的各种功能。

rospy：使用python实现ROS的各种功能。

package.xml：清单文件，关于'软件包'相关信息的描述，用于定义软件包相关元信息之间的依赖关系，这些信息包括版本、维护者和许可协议，编译依赖和运行依赖等。

CMakeLists.txt：编译配置文件，使用cmake进行程序编译的时候，会根据CMakeLists.txt这个文件进行一步一步的处理，然后形成一个MakeFile文件，系统再通过这个文件的设置进行程序的编译。

2.6.3 程序包依赖关系

1、一级依赖

在使用catkin_create_pkg命令时提供了几个程序包作为依赖包，现在可以使用rospack命令工具来查看一级依赖包。

```
$ rospack depends1 test_tutorials
roscpp
rospy
std_msgs
```

rospack列出了在运行catkin_create_pkg命令时作为参数的依赖包，这些依赖包随后保存在package.xml文件中。

2、间接依赖

一个程序包还可以有好几个间接的依赖包，使用rospack可以递归检测出所有的依赖包。

```
$ rospack depends test_tutorials
cpp_common
rostime
roscpp_traits
roscpp_serialization
std_msgs
rosgraph_msgs
.....(省略)
roscpp
rosgraph
catkin
rospack
roslib
rospy
```

2.6.4 程序包命名规范

ROS 包的命名遵循一个命名规范，只允许使用小写字母、数字和下划线，而且首字符必须是一个小写字母。一些 ROS 工具，包括 `catkin`，不支持那些不遵循此命名规范的包。

```
~/catkin_ws/src$ catkin_create_pkg Test_Pack
WARNING: Catkin package name "Test_Pack" does not follow the naming conventions. It should start
with a lo, digits, and underscores.
Created file Test_Pack/CMakeLists.txt
Created file Test_Pack/package.xml
Successfully created files in /home/silver/catkin_ws/src/Test_Pack. Please adjust the values in
package.xml
```

可以看到虽然成功的创建了 `Test_Pack` 软件包，但是在创建的过程中发生了警告信息（`WARNING`），这样做既不符合规范，也可能增加不必要的兼容性错误。

2.6.5 删除程序包

ROS 是运行在 `linux` 之上的操作做系统，所以他适用 `linux` 的基本命令，想要删除工作空间里的软件包，可采用 `linux` 删除命令 `rm -rf [软件包目录地址]` 即可，然后重新编译整个工作空间。

```
~/catkin_ws/src$ ls
CMakeLists.txt  Test_Pack  test_tutorials
~/catkin_ws/src$ rm -rf Test_Pack/
~/catkin_ws/src$ ls
CMakeLists.txt  test_tutorials
```

2.6.6 编译功能包

一旦安装了所需的系统依赖项，我们就可以开始编译刚才创建的程序包了。如果是通过 `apt` 或者其它软件包管理工具来安装 ROS 的，那么系统已经默认安装好所有依赖项。

`catkin_make` 是一个命令行工具，它简化了 `catkin` 的标准工作流程。使用 `catkin_make` 来编译功能包如下操作：

```
$ cd ~/catkin_ws/
$ catkin_make
```


为了演示如何正确的编译软件包，我们在软件包test_tutorials下的src目录下编写一个测试代码来演示如何编译软件包。

<1> 编辑测试代码，实现hello的连续打印：

```
#include "ros/ros.h"
#include "std_msgs/String.h"
#include <sstream>
int main(int argc, char **argv)
{
    ros::init(argc, argv, "talker");
    ros::NodeHandle n;
    ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000);
    ros::Rate loop_rate(10);
    int count = 0;
    while (ros::ok())
    {
        std_msgs::String msg;
        std::stringstream ss;
        ss << "hello" << count;
        msg.data = ss.str();
        ROS_INFO("%s", msg.data.c_str());
        chatter_pub.publish(msg);
        ros::spinOnce();
        loop_rate.sleep();
        ++count;
    }
    return 0;
}
```

<2> 修改CMakeLists.txt配置文件

找到如下代码，去掉前面的#号。

```
#add_executable(${PROJECT_NAME}_node src/test.cpp)
#target_link_libraries(${PROJECT_NAME}_node
#  ${catkin_LIBRARIES}
# )
```

按照节点名称，程序文件名称修改：

```
add_executable(test_tutorials_node src/test.cpp)
target_link_libraries(test_tutorials_node ${catkin_LIBRARIES})
add_dependencies(test_tutorials_node test_tutorials_generate_messages_cpp)
```

解析：

◆ **add_executable()** 指定要编译的可执行文件，例如：

```
add_executable(myNode src/main.cpp src/file1.cpp src/file2.cpp)
```

上面的语句将调用src/main.cpp，src/file1.cpp和src/file2.cpp生成名为myNode的目标可执行文件，生成的可执行文件就是我们说的节点。**\${PROJECT_NAME}**也可以不修改，因为该文件第二行已经指定**project(test_tutorials)**。

◆ **target_link_libraries()** 指定所生成的可执行文件所链接的库文件

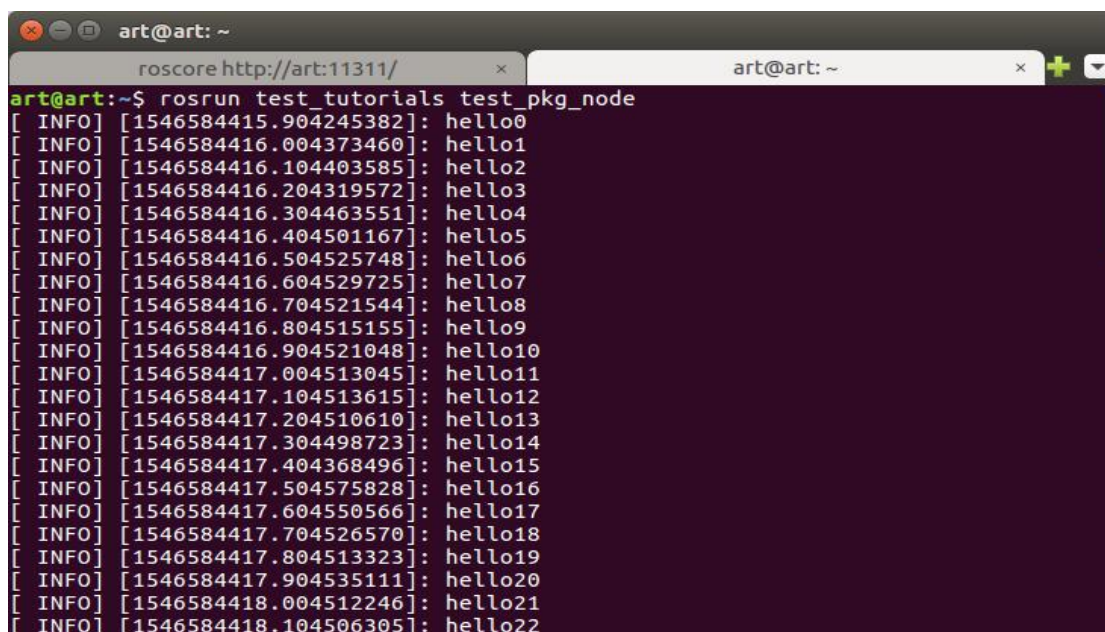
通常写在**add_executable()**之后。一般来说，要生成一个ros节点，必须添加**catkin_LIBRARIES**，例如：

```
target_link_libraries(myNode ${catkin_LIBRARIES})
```

<3> 在工作空间根目录下使用**catkin_make**进行编译工作空间

```
$ catkin_make
.....(省略)
-- ~~~~~
-- ~~ traversing 2 packages in topological order:
-- ~~ - test_tutorials
-- ~~~~~
-- +++ processing catkin package: 'test_tutorials'
-- ==> add_subdirectory(test_tutorials)
-- +++ processing catkin package: 'test_tutorials'
-- ==> add_subdirectory(test_tutorials)
-- Configuring done
-- Generating done
-- Build files have been written to: /home/silver/catkin_ws/build
####
#### Running command: "make -j2 -l2" in "/home/silver/catkin_ws/build"
####
[100%] Built target test_tutorials_node
```

<4> 启动roscore，然后新打开一个命令窗口运行roslaunch命令

A terminal window titled 'art@art: ~' showing the output of the command 'roslaunch test_tutorials test_pkg_node'. The output consists of 23 lines, each starting with '[INFO]' followed by a timestamp and the message 'hello' followed by a number from 0 to 22. The terminal has a dark background with light-colored text. The window title bar shows 'art@art: ~' and a URL 'http://art:11311/'.

2.6 ROS节点

一个节点其实只不过是ROS程序包中的一个可执行文件。ROS节点可以使用ROS客户库与其他节点通信。节点可以发布或接收一个话题。节点也可以提供或使用某种服务。

ROS有另一种类型的节点，称为nodelet，这类特殊节点可以在单个进程中运行多个节点，其中每个nodelet为一个轻量级的线程。这样，可以在不使用ROS网络的情况下与其他节点通信，节点的通信效率更高，避免网络拥塞。对于摄像头类和3D传感器这类数据传输量非常大的设备十分有用。

2.7.1 启动roscore

roscore 是运行所有ROS程序前必须要运行的命令。

```
$ roscore
```

Roscore启动成功后终端会显示：

```
... logging to /home/art/.ros/log/e5fbd6ac-0f1e-11e9-a6d0-0025d388f84d/roslaunch-art-6736.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.
started roslaunch server http://art:43089/
```

```
ros_comm version 1.12.14
```

SUMMARY

=====

PARAMETERS

* /roscdistro: kinetic

* /rosversion: 1.12.14

NODES

auto-starting new master

process[roscdistro]: started with pid [6747]

ROS_MASTER_URI=http://art:11311/

setting /run_id to e5fbd6ac-0f1e-11e9-a6d0-0025d388f84d

process[roscdistro-1]: started with pid [6760]

started core service [/roscdistro]

如果 roscore 运行后无法正常初始化，则需要查看是否网络配置出了问题

2.7.2 使用roscdistro

ROS提供了处理节点的工具，例如roscdistro，运行该命令，可以获得一个命令清单，该清单列出了roscdistro的命令功能。

```
~/catkin_ws/src$ roscdistro
```

roscdistro is a command-line tool for printing information about ROS Nodes.

Commands:

roscdistro ping	test connectivity to node
roscdistro list	list active nodes
roscdistro info	print information about node
roscdistro machine	list nodes running on a particular machine or list machines
roscdistro killkill	kill a running node
roscdistro cleanup	purge registration information of unreachable nodes

Type roscdistro <command> -h for more detailed usage, e.g. 'roscdistro ping -h'

roscdistro list 可以列出当前活跃的节点名称，例如：

```
$ rosnodet list
/roout
/talker
```

其中 `/roout`总是随着`roscore`的启动而出现的，`/talker`是我们在上一节中编写的测试节点。可以通过`rosnodet info`来进一步查看节点的详细信息，例如：

```
$ rosnodet info /talker
-----
Node [/talker]
Publications:
  * /chatter [std_msgs/String]
  * /roout [rosgaph_msgs/Log]
Subscriptions: None
Services:
  * /talker/get_loggers
  * /talker/set_logger_level
```

2.8 ROS话题

话题是节点用来传输数据的总线。通过话题进行消息路由不需要节点之间直接连接，这就意味着发布者和订阅者之间不需要知道彼此的存在。同一个话题可以有多个订阅者，也可以有多个发布者。话题默认采用TCP/IP长连接的方式传输。

话题之间的通信是通过在节点之间发送ROS消息实现的。对于发布者(`turtleteleopkey`)和订阅器(`turtlesim_node`)之间的通信，发布器和订阅器之间必须发送和接收相同类型的消息。这意味着话题的类型是由发布在它上面的消息类型决定的。使用`rostopic type`命令可以查看发布在某个话题上的消息类型。

2.8.1 控制turtle移动

通过第一个ROS例程，我们已经能够自如的控制小乌龟的移动。下面回顾一下：

<1> 启动ROS

`roscore` 是你在运行所有ROS程序前首先要运行的命令。

```
$ roscore
```

<2> 打开新的命令窗口，启动turtlesim_node节点

```
$ rosrn turtlesim turtlesim_node
```

<3> 打开新的命令窗口，启动turtle_teleop_key节点

```
$ rosrn turtlesim turtle_teleop_key
```

当命令窗口界面出现Use arrow keys to move the turtle时，我们就可通过键盘得方向键来控制小乌龟的移动了。如下图所示：



能控制小乌龟移动只是第一步，接下来需要做的就是了解ROS系统对小乌龟做了些什么，通过小乌龟的例程的学习，扩展到对真实机器人节点的控制。

2.8.2 查看节点信息

通过小乌龟移动的实例可以预想到，turtlesim_node节点和turtle_teleop_key节点之间是通过一个ROS话题来互相通信的。turtle_teleop_key在一个话题上发布按键输入消息，而turtlesim_node则订阅该话题以接收该消息，实现小乌龟的移动。我们来查看节点列表。

```
$ rosnod list
/rosout
/teleop_turtle
/turtlesim
```

下一步我们分别查看teleop_turtle和turtlesim节点的信息，

```
$ rosnode info /teleop_turtle

-----

Node [/teleop_turtle]
Publications:
* /rosout [roscpp_msgs/Log]
* /turtle1/cmd_vel [geometry_msgs/Twist]

Subscriptions: None
Services:
* /teleop_turtle/get_loggers
* /teleop_turtle/set_logger_level
```

```
art@art:~$ rosnode info /turtlesim

-----

Node [/turtlesim]
Publications:
* /rosout [roscpp_msgs/Log]
* /turtle1/color_sensor [turtlesim/Color]
* /turtle1/pose [turtlesim/Pose]

Subscriptions:
* /turtle1/cmd_vel [geometry_msgs/Twist]
```

这说明teleop_turtle节点发布了/turtle1/cmd_vel [geometry_msgs/Twist]这样的一个话题，而turtlesim 节点则订阅了这样的一个话题。

2.8.3 查看话题信息

使用rostopic list可以查看当前的话题清单：

```
$ rostopic list

/rosout
/rosout_agg
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

使用echo参数可以查看使用键盘方向键之后节点发出的信息。

```
$ rostopic echo /turtle1/cmd_vel
linear:
  x: -2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

注意：调用rostopic echo命令后，通过键盘控制移动小乌龟方可看到具体消息（只有在\$ rosrn turtlesim turtle_teleop_key 这个窗口下键盘消息才能生效），ctrl + z 停止。

使用rostopic type 可以查看消息的类型。

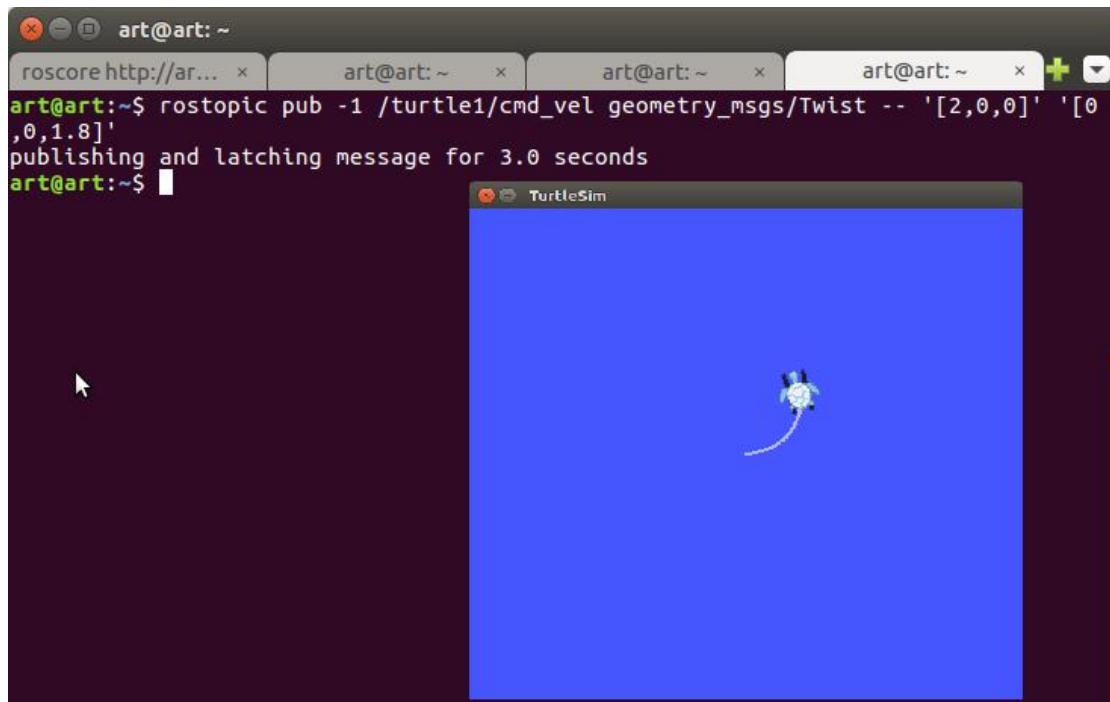
```
$ rostopic type /turtle1/cmd_vel
geometry_msgs/Twist
```

如果想要看到消息字段，可以使用下命令：

```
$ rosmmsg show geometry_msgs/Twist
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

在了解话题与节点的交互原理之后，我们可以使用rostopic pub [topic] [msg_type]命令来直接发布话题：

```
$ rostopic pub -1 /turtle1/cmd_vel geometry_msgs/Twist -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 1.8]'
```

以上命令会发送一条消息给turtlesim，告诉小乌龟以2.0大小的线速度和1.8大小的角速度开始移动。

`rostopic pub` 这条命令将会发布消息到某个给定的话题。

`-1`（单个破折号）：这个参数选项使rostopic发布一条消息后马上退出。

`/turtle1/cmd_vel`：这是消息所发布到的话题名称。

`geometry_msgs/Twist`：这是所发布消息的类型。

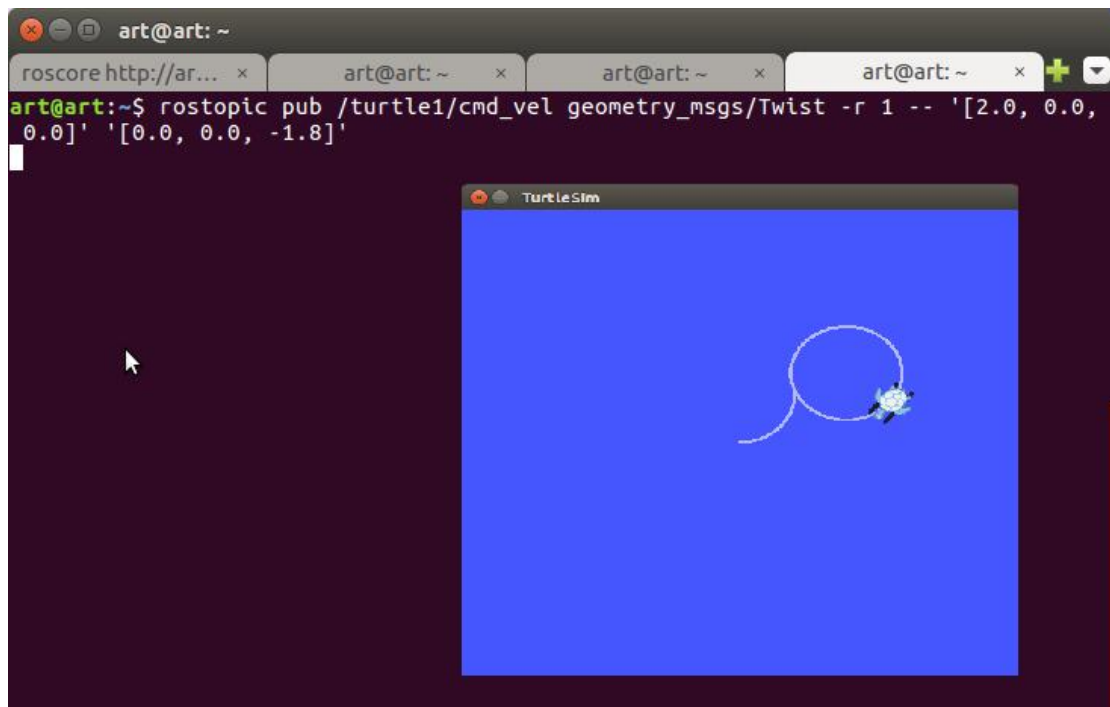
`--`（双破折号）：这会告诉命令选项解析器接下来的参数部分都不是命令选项。这在参数里面包含有破折号-（比如负号）时是必须要添加的。

`'[2.0, 0.0, 0.0]'` `'[0.0, 0.0, 1.8]'`：正如之前提到的，在一个turtlesim/Velocity消息里面包含有两个浮点型元素：`linear`和`angular`。在本例中，2.0是`linear`的值，1.8是`angular`的值。这些参数其实是按照YAML语法格式编写的，这在YAML文档中有更多的描述。

此时，turtle已经停止移动了。这是因为turtle需要一个稳定的频率为1Hz的命令流来保持移动状态。

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1 -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, -1.8]'
```

这条命令以1Hz的频率发布速度命令到速度话题上。发布完成后再来看turtle的移动轨迹。



分析上面两个小乌龟的运动状态可知：

命令	功能
rostopic pub	发布消息到话题的指令
-1	命令只发送一次
-r	表示重复发送命令
/turtle1/cmd_vel	发送的话题
geometry_msgs/Twist	发送消息类型
--	表示非命令选项
'[2, 0, 0]' '[0, 0, 1.8]'	发送消息的内容

2.8.4 使用 rqt_graph

rqt_graph是rqt程序包中的一部分，它可以创建一个显示当前系统运行情况的动态图形。

如果你没有安装，请通过以下命令来安装：

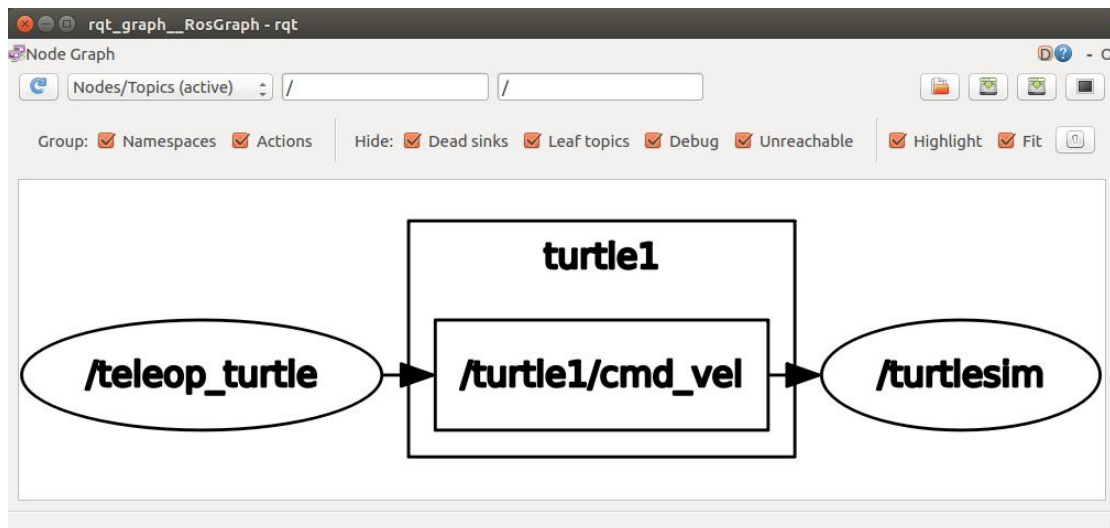
```
$ sudo apt-get install ros-kinetic (版本)-rqt
```

```
$ sudo apt-get install ros-kinetic (版本)-rqt-common-plugins
```

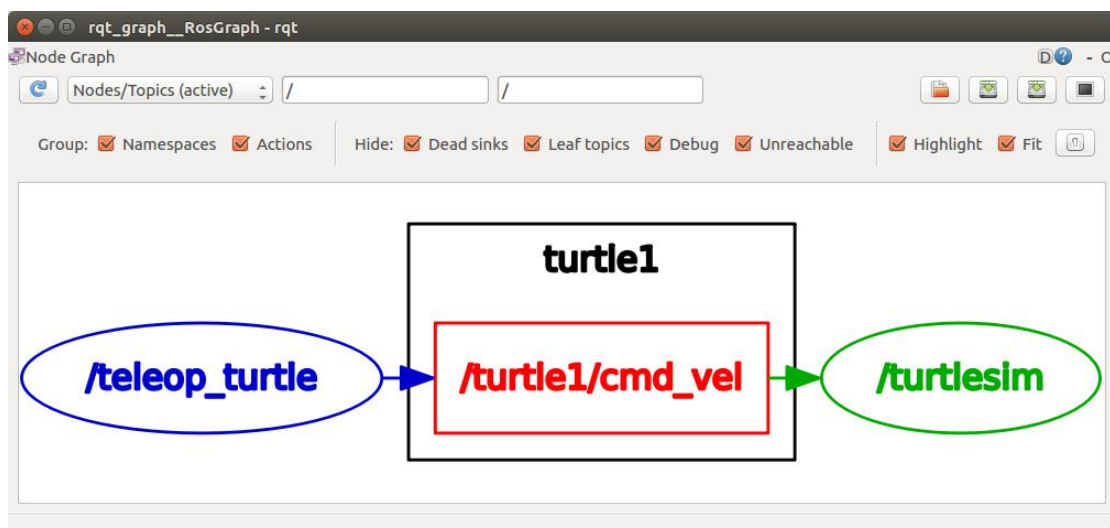
打开一个新命令终端，运行代码：

```
$ rosrun rqt_graph rqt_graph
```

你会看到类似下图所示的图形：



如果你将鼠标放在/turtle1/cmd_vel上方，相应的ROS节点（蓝色和绿色）和话题（红色）就会高亮显示。正如你所看到的，turtlesim和teleop_turtle节点正通过一个名为/turtle1/cmd_vel的话题来互相通信。



2.8.5 使用 rqt_plot

rqt_plot命令可以实时显示一个发布到某个话题上的数据变化图形。这里我们将使用rqt_plot命令来绘制正在发布到/turtle1/pose话题上的数据变化图形。首先，在一个新终端中运行

rqt_plot命令:

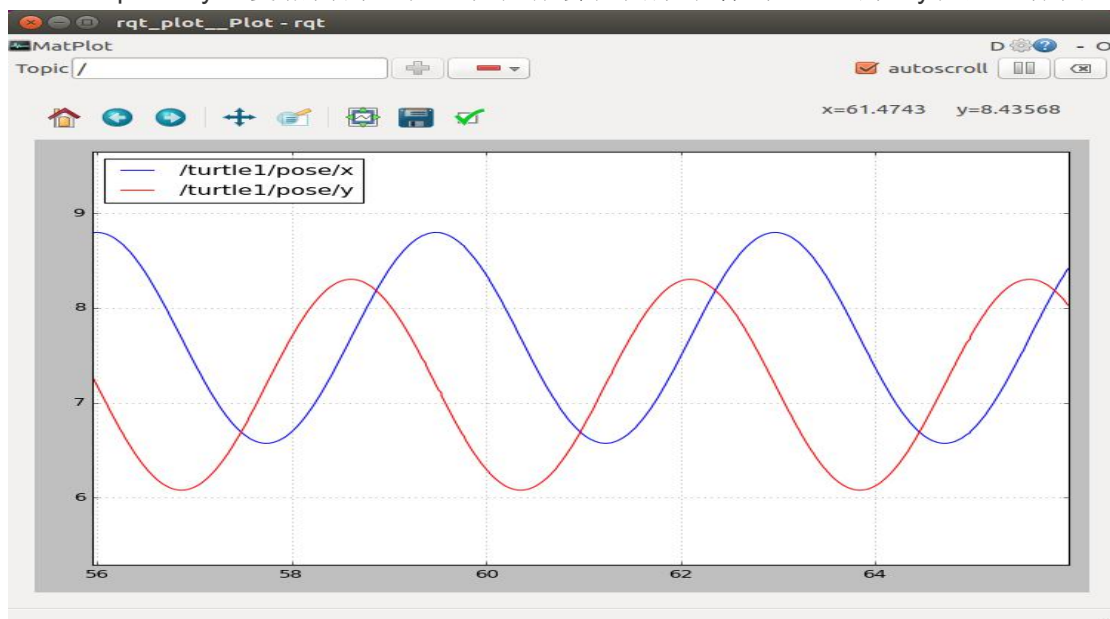
```
$ rosrun rqt_plot rqt_plot
```

这会弹出一个新窗口，在窗口左上角的一个文本框里面你可以添加需要绘制的话题。

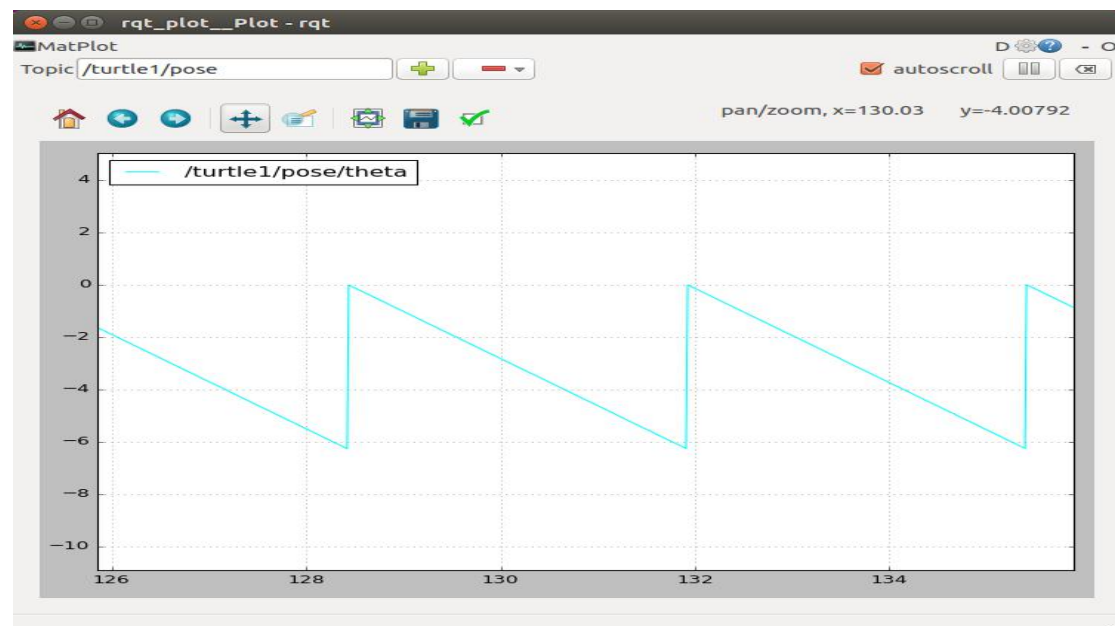


在里面输入/turtle1/pose/x后之前处于禁用状态的加号按钮将会被使能变亮。按一下该按钮，并对/turtle1/pose/y重复相同的过程。现在你会在图形中看到turtle的x-y位置坐标图。

这会弹出一个新窗口，在窗口左上角的一个文本框里面你可以添加需要绘制的话题。在里面输入/turtle1/pose/x后之前处于禁用状态的加号按钮将会被使能变亮。按一下该按钮，并对/turtle1/pose/y重复相同的过程。现在你会在图形中看到turtle的x-y位置坐标图。



按下减号按钮会显示一组菜单让你隐藏图形中指定的话题。现在隐藏掉你刚才添加的话题并添加/turtle1/pose/theta，你会看到如下图所示的图形：



2.9 ROS服务和参数

当需要直接与节点通信并获得应答时，将无法通过话题实现，而需要使用服务。

服务（**services**）是节点之间通讯的另一种方式。服务允许节点发送请求（**request**）并获得一个响应（**response**）。

2.9.1 rosservice

rosservice 可以很轻松的使用 ROS 客户端/服务器框架提供的服务。rosservice 提供了很多可以在 topic 上使用的命令，如下所示：

```
$ rosservice
```

Commands:

rosservice args	print service arguments
rosservice call	call the service with the provided args
rosservice find	find services by service type
rosservice info	print information about service
rosservice list	list active services
rosservice type	print service type
rosservice uri	print service ROSRPC uri

2.9.2 使用rosservice list

```
$ rosservice list  
  
/clear  
  
/kill  
  
/reset  
  
/rosout/get_loggers  
  
/rosout/set_logger_level  
  
/spawn  
  
.....(省略)
```

list 命令显示turtlesim节点提供了多个服务：重置（reset），清除（clear），再生（spawn），终止（kill），turtle1/set_pen，/turtle1/teleport_absolute，/turtle1/teleport_relative，turtlesim/get_loggers，和 turtlesim/set_logger_level，同时还有另外两个rosout节点提供的服务：/rosout/getloggers 和 /rosout/setlogger_level。

2.9.3 使用rosservice type

我们可以使用rosservice 来查看服务的类型，格式为：

```
$ rosservice type [service]
```

首先说clear服务的类型：

```
$ rosservice type clear  
std_srvs/Empty
```

服务的类型为空（empty），这表明在调用这个服务是不需要参数（比如，请求不需要发送数据，响应也没有数据）。

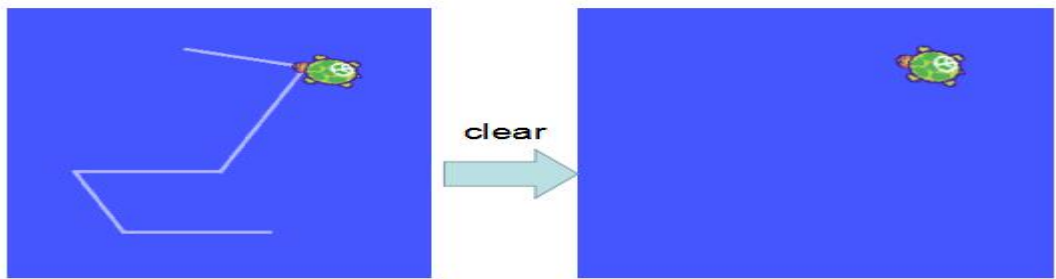
下面我们使用rosservice call命令调用clear服务。

使用方法：

```
$ rosservice call [service] [args]
```

我们来调用 clear的服务，因为服务类型是空，所以进行无参数调用：

```
$ rosservice call clear
```



再次打开turtlesim，就可以看到clear服务清除了turtlesim_node的背景上的轨迹。

再生（spawn）服务的信息，了解带参数的服务：

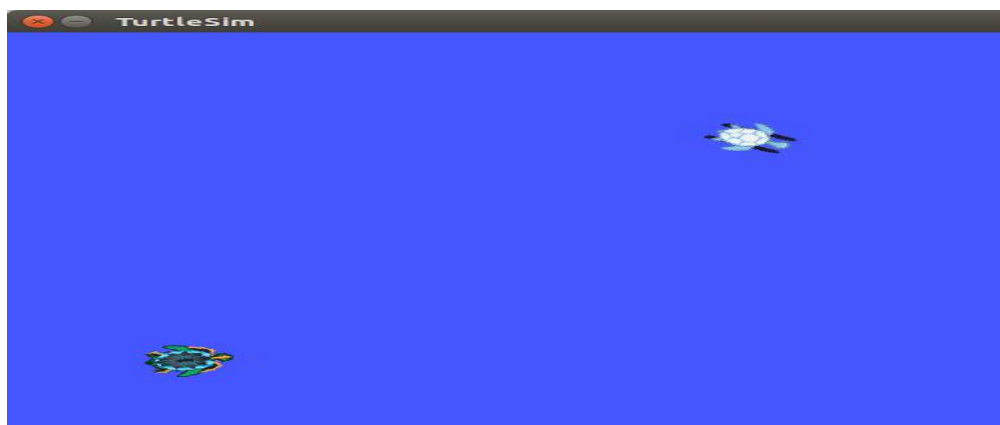
```
$ rosservice type spawn| rossrv show
float32 x
float32 y
float32 theta
string name
---
string name
```

这个服务使得我们可以在给定的位置和角度生成一只新的乌龟。名字参数是可选的，这里我们不设具体的名字，让turtlesim自动创建一个。

```
$ rosservice call spawn 2 2 0.2 ""
name: turtle2
```

服务返回了新产生的乌龟的名字，如果我们名字参数设置具体的名字如："art"，则系统会创建一个名字为"art"的小乌龟。

```
$ rosservice call spawn 2 2 0.2 "art"
name: "art"
```



2.9.4 rosparam

rosparam使得我们能够存储并操作ROS参数服务器（Parameter Server）上的共享数据。参数服务器能够存储整型、浮点、布尔、字符串、字典和列表等数据类型。rosparam使用YAML标记语言的语法。一般而言，YAML的表述很自然：1 是整型，1.0 是浮点型，one是字符串，true是布尔，[1, 2, 3]是整型列表，{a: b, c: d}是字典。rosparam 有很多指令可以用来操作参数，如下所示：

```
$ rosparam
rosparam is a command-line tool for getting, setting, and deleting parameters from the ROS
Parameter Server.
Commands:
    rosparam set          set parameter  设置参数
    rosparam get          get parameter  获取参数
    rosparam load         load parameters from file 从文件读取参数
    rosparam dump         dump parameters to file 向文件中写入参数
    rosparam delete       delete parameter 删除参数
    rosparam list         list parameter names 列出参数名
```

2.9.5 使用rosparam list

我们来看看现在参数服务器上都有哪些参数：

```
$ rosparam list
/background_b
/background_g
/background_r
/rosdistro
/roslaunch/uris/host_ubuntun__40153
/rosversion
/run_id
```

上面得背景三基色是用来设置turtlesim节点的参数。这些参数可以改变窗体的颜色。

2.9.6 使用rosparam set和get

可以用过rosparam get、 rosparam set 获取和设置参数，例如：

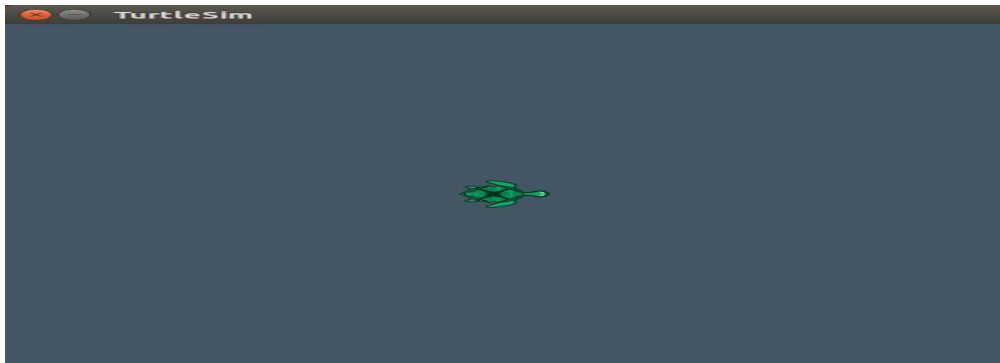
```
$ rosparam get /background_b
255
```



```
$ rosparam set /background_b 100
$ rosparam get /background_b
100
```

上述指令修改了参数的值，现在我们调用清除服务使得修改后的参数生效：

```
$ rosservice call clear
```



可以看到小乌龟的背景已经变成灰色了。

2.9.7 使用 rosparam dump和load

<1> 使用rosparam get /来显示参数服务器上的所有内容：

```
$ rosparam get /
background_b: 100
background_g: 86
background_r: 69
rostdistro: 'kinetic'

roslaunch:
  uris: {host_art__37221: 'http://art:37221/'}
  rosversion: '1.12.14'

run_id: e2fd2cee-1220-11e9-b128-00e45c680efeart
```

如果我们希望存储这些信息以备日后重新读取。这通过rosparam很容易就可以实现，使用方法：

```
$ rosparam dump [file_name]
$ rosparam load [file_name] [namespace]
```

<2> 使用 `rosparam dump` 将所有的参数写入 `art.yaml` 文件:

```
$ rosparam dump art.yaml
```

<3> 使用 `rosparam load` 向参数服务器加载新的数据文件:

```
$ rosparam load params.yaml namespace
```

示例: 将 `yaml` 文件重载入新的命名空间, 比如说 `art` 空间:

```
$ rosparam load params.yaml art
$ rosparam get art/background_b
100
```

2.10 创建ROS消息和ROS服务

前面的章节对ROS相关知识做了详细介绍, 本次我们介绍如何创建并编译ROS消息和服务, 以及 `rosmmsg`, `rossrv` 和 `roscpp` 命令行工具的使用。

2.10.1 消息(msg)和服务(srv)介绍

<1> 定义:

◆ 消息(msg): `msg` 文件就是一个描述ROS中所使用消息类型的简单文本。它们会被用来生成不同语言的源代码。

◆ 服务(srv): 一个 `srv` 文件描述一项服务。它包含两个部分: 请求和响应。

<2> 存放位置:

`msg` 文件存放在 `package` 的 `msg` 目录下, `srv` 文件则存放在 `srv` 目录下。

<3> `msg` 本质:

`msg` 文件实际上就是每行声明一个数据类型和变量名。可以使用的数据类型如下:

```
int8, int16, int32, int64 (plus uint*)
float32, float64
string
time, duration
other msg files
variable-length array[] and fixed-length array[C]
```

在ROS中有一个特殊的数据类型：**Header**，它含有时间戳和坐标系信息。在msg文件的第一行经常可以看到Header header的声明。下面是一个msg文件的样例，它使用了Header，string，和其他另外两个消息类型。

```
Header header
string child_frame_id
geometry_msgs/PoseWithCovariance pose
geometry_msgs/TwistWithCovariance twist
```

srv文件分为请求和响应两部分，由'---'分隔。下面是srv的一个样例：

```
int64 A
int64 B
---
int64 Sum
```

其中A和B是请求，而Sum是响应。

2.10.2 使用 msg

<1> 我们将在之前创建的package里定义新的消息info.msg

```
$ cd ~/catkin_ws/src/test_tutorials/
$ mkdir msg
$ cd msg/
$ touch info.msg
```

进入info.msg编辑内容如下：

```
string first_name
string last_name
uint8 age
uint32 score
```

<2> 编辑package.xml

要确保msg文件被转换成为C++，Python和其他语言的源代码，就必须确保package.xml包含以下两条语句。

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
```

在package.xml中找到该两行，取消注释。如果没有，添加进去。在构建的时候，我们需要"message_generation"。然而，在运行的时候，我们需要"message_runtime"。

<3> 编辑CMakeLists.txt文件

在 CMakeLists.txt文件中，利用find_package函数，增加对message_generation的依赖，这样就可以生成消息了。

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
```

<4> 找到add_message_files行，取消注释，添加消息名称info.msg。

```
## Generate messages in the 'msg' folder
add_message_files(
  FILES
  info.msg
)
```

<5> 找到generate_message行，取消注释。

```
## Generate added messages and services with any dependencies listed here
generate_messages(
  DEPENDENCIES
  std_msgs
)
```

<6> 使用一下命令编译。

```
$ cd ~
$ cd catkin_ws/
$ catkin_make
```

编译成功后，通过rosmmsg show命令，检查ROS是否能够识消息，如下图所示，即表示消息创建成功。

```
art@art:~$ rosmmsg show test_tutorials/info
string first_name
```

```
string last_name
uint8 age
uint32 score
```

2.10.3 使用 `srv`

<1> 在`test_tutorials`中创建一个服务：

```
$ roscd test_tutorials
$ mkdir srv
$ cd srv
$ touch data.srv
```

<2> 进入 `data.srv`编辑内容如下：

```
int32 A
int32 B
int32 C
---
int32 sum
```

<3> 使用命令从`test_tutorials`功能包中打开`CMakeList.xml`文件

```
$ roscd test_tutorials CMakeLists.txt
```

找到`catkin_package`添加如下依赖。

```
catkin_package(
#  INCLUDE_DIRS include
#  LIBRARIES test_tutorials
#  CATKIN_DEPENDS roscpp rospy std_msgs
#  DEPENDS system_lib
  CATKIN_DEPENDS message_runtime
)
```

<4> 找到`add_service_files`，取消注释，添加服务名称。

```
add_service_files(
  FILES
  data.srv
)
```

<5> 添加完成后，保存，回到命令窗口，使用以下命令编译package。

```
$ cd catkin_ws/  
$ catkin_make
```

<6> 编译成功后，调用rossrv show指令查看编译是否成功

```
art@art:~/catkin_ws$ rossrv show data.srv
```

```
[test_tutorials/data]:  
int32 A  
int32 B  
int32 C  
---  
int32 sum
```

注意：增加了新的消息、服务，都要需要重新编译我们的 package:

2.11 编写发布器与订阅器

话题之间的通信是通过在节点间发送消息实现的。对于发布器和订阅器之间的通信，发布器和订阅器之间必须发送和接收相同类型的消息。这意味着话题的类型是由发布在它上面的消息类型决定的。使用 `rostopic type` 命令可以查看发布在某个话题上的消息类型。

本节中，我们创建两个节点并通过话题来实现通信。

2.11.1 定义msg消息

1、我们之前已经创建好一个test_tutorials包，现在进入msg路径中定义新的消息num.

msg

```
$ cd ~/catkin_ws/src/test_tutorials/msg  
$ touch num.msg
```

进入num.msg编辑内容如下：

```
int64 num
```

2、编辑package.xml

要确保msg文件被转换成为C++/Python和其他语言的源代码，就必须确保package.xml包含以下两条语句。

```
<build_depend>message_generation</build_depend>
```

```
<exec_depend>message_runtime</exec_depend>
```

在构建的时候，我们需要"messagegeneration"。然而，在运行的时候，我们需要"message_runtime"。

3、编辑CMakeLists.txt文件

在 CMakeLists.txt文件中，在find_package中，增加对message_generation的依赖，这样就可以生成消息了。

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
```

4、找到add_message_files行，取消注释，添加消息名称num.msg。

```
## Generate messages in the 'msg' folder
add_message_files(
  FILES
  num.msg
)
```

5、找到generate_message行，取消注释。

```
## Generate added messages and services with any dependencies listed here

generate_messages(
  DEPENDENCIES
  std_msgs
)
```

6、使用一下命令编译。

```
$ cd ~
$ cd catkin_ws/
$ catkin_make
```

编译成功后，通过rosmmsg show命令，检查ROS是否能够识别消息。

2.11.2 创建发布器节点

创建发布器节点通常使用python或者C++语言。下面我们对两种语言创建发布器节点进行介绍：

python创建发布器节点：

1、进入test_tutorials包下，src目录。

```
/catkin_ws/src/test_tutorials$ mkdir scripts  
/catkin_ws/src/test_tutorials$ cd scripts/
```

2、新建talker.py文件，设置为可执行权限

```
$ touch talker.py  
$ chmod +x talker.py
```

3、编辑代码如下

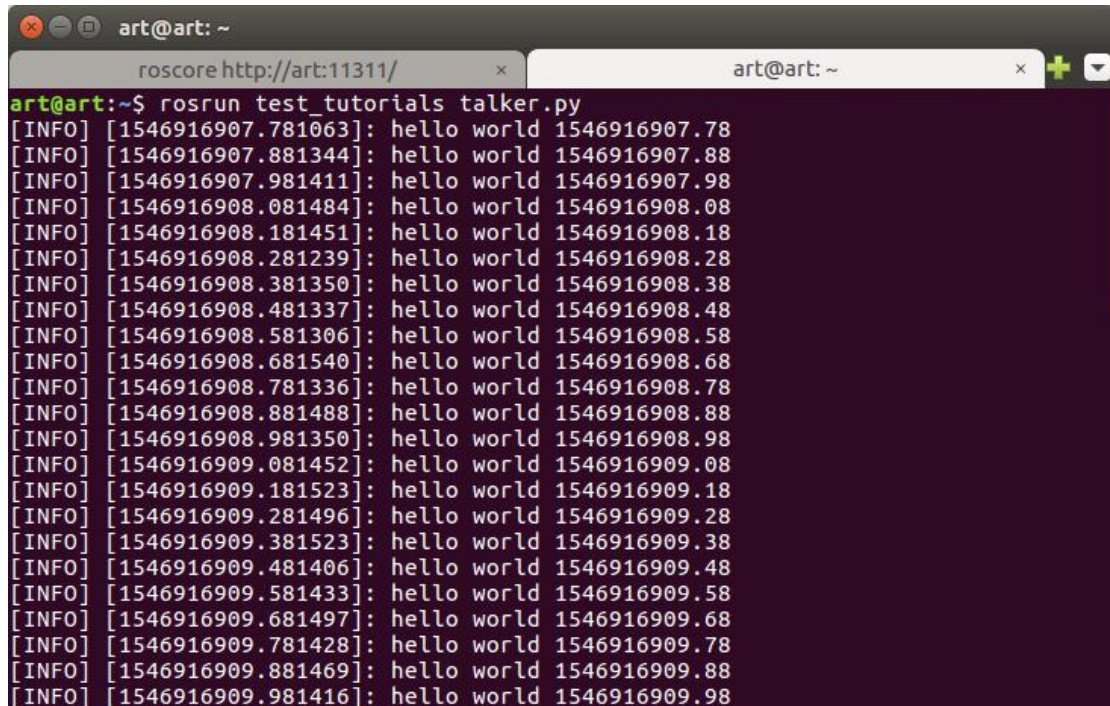
```
#!/usr/bin/env python  
# license removed for brevity  
import rospy  
from std_msgs.msg import String  
  
def talker():  
    pub = rospy.Publisher('chatter', String, queue_size=10)  
    rospy.init_node('talker', anonymous=True)  
    rate = rospy.Rate(10) # 10hz  
    while not rospy.is_shutdown():  
        hello_str = "hello world %s" % rospy.get_time()  
        rospy.loginfo(hello_str)  
        pub.publish(hello_str)  
        rate.sleep()  
  
if __name__ == '__main__':  
    try:  
        talker()  
    except rospy.ROSInterruptException:  
        pass
```


4、编译工作空间，并启动roscore

```
$ cd ~/catkin_ws  
$ catkin_make  
$ roscore
```

5、打开新的终端，启动talker.py

```
$ rosrun test_tutorials talker.py
```



```
art@art: ~  
roscore http://art:11311/  
art@art:~$ rosrun test_tutorials talker.py  
[INFO] [1546916907.781063]: hello world 1546916907.78  
[INFO] [1546916907.881344]: hello world 1546916907.88  
[INFO] [1546916907.981411]: hello world 1546916907.98  
[INFO] [1546916908.081484]: hello world 1546916908.08  
[INFO] [1546916908.181451]: hello world 1546916908.18  
[INFO] [1546916908.281239]: hello world 1546916908.28  
[INFO] [1546916908.381350]: hello world 1546916908.38  
[INFO] [1546916908.481337]: hello world 1546916908.48  
[INFO] [1546916908.581306]: hello world 1546916908.58  
[INFO] [1546916908.681540]: hello world 1546916908.68  
[INFO] [1546916908.781336]: hello world 1546916908.78  
[INFO] [1546916908.881488]: hello world 1546916908.88  
[INFO] [1546916908.981350]: hello world 1546916908.98  
[INFO] [1546916909.081452]: hello world 1546916909.08  
[INFO] [1546916909.181523]: hello world 1546916909.18  
[INFO] [1546916909.281496]: hello world 1546916909.28  
[INFO] [1546916909.381523]: hello world 1546916909.38  
[INFO] [1546916909.481406]: hello world 1546916909.48  
[INFO] [1546916909.581433]: hello world 1546916909.58  
[INFO] [1546916909.681497]: hello world 1546916909.68  
[INFO] [1546916909.781428]: hello world 1546916909.78  
[INFO] [1546916909.881469]: hello world 1546916909.88  
[INFO] [1546916909.981416]: hello world 1546916909.98
```

6、代码解析：

代码：#!/usr/bin/env python

分析：指定通过python解释代码

代码：import rospy

分析：导入rospy包，rospy是ROS的python客户端。

代码：from std_msgs.msg import String

分析：导入python的标准字符处理库

代码：def talker()

分析：定义talker函数。

代码：pub = rospy.Publisher('chatter', String, queue_size=10)

分析：定义发布的话题名称chatter， 消息类型String，实质是std_msgs.msg.String，设置队列条目个数为10。

代码: `rospy.init_node('talker', anonymous=True)`

分析: 初始化节点, 节点名称为talker, `anonymous=True`, 要求每个节点都有唯一的名称, 避免冲突。这样可以运行多个talker.py。

代码: `rate = rospy.Rate(10)`

分析: 设置发布的频率, 单位是每秒次数, 这是每秒10次的频率发布话题。

代码: `rospy.is_shutdown()`

分析: 用于检测程序是否退出, 是否按Ctrl-C 或其他

代码: `rospy.loginfo`

分析: 在屏幕输出日志信息, 写入到rosout节点, 也可以通过rqt_console来查看。

代码: `pub.publish(hello_str)`

分析: 发布信息到话题, 参考pub.publish API

代码: `rate.sleep()`

分析: 睡眠一定持续时间, 如果参数为负数, 睡眠会立即返回。参考 sleep api。

2.11.3 创建订阅器节点

python创建订阅器节点:

1、/src目录下新建listener.py文件, 设置为可执行权限

```
$ touch listener.py
$ chmod +x listener.py
```

2、编辑代码如下

```
#!/usr/bin/env python
from std_msgs.msg import String

def callback(data):
    rospy.loginfo(rospy.get_caller_id() + 'I heard %s', data.data)

def listener():
    rospy.init_node('listener', anonymous=True)
    rospy.Subscriber('chatter', String, callback)
    rospy.spin()

if __name__ == '__main__':
```

```
listener()
```

3、编译工作空间，并启动roscore

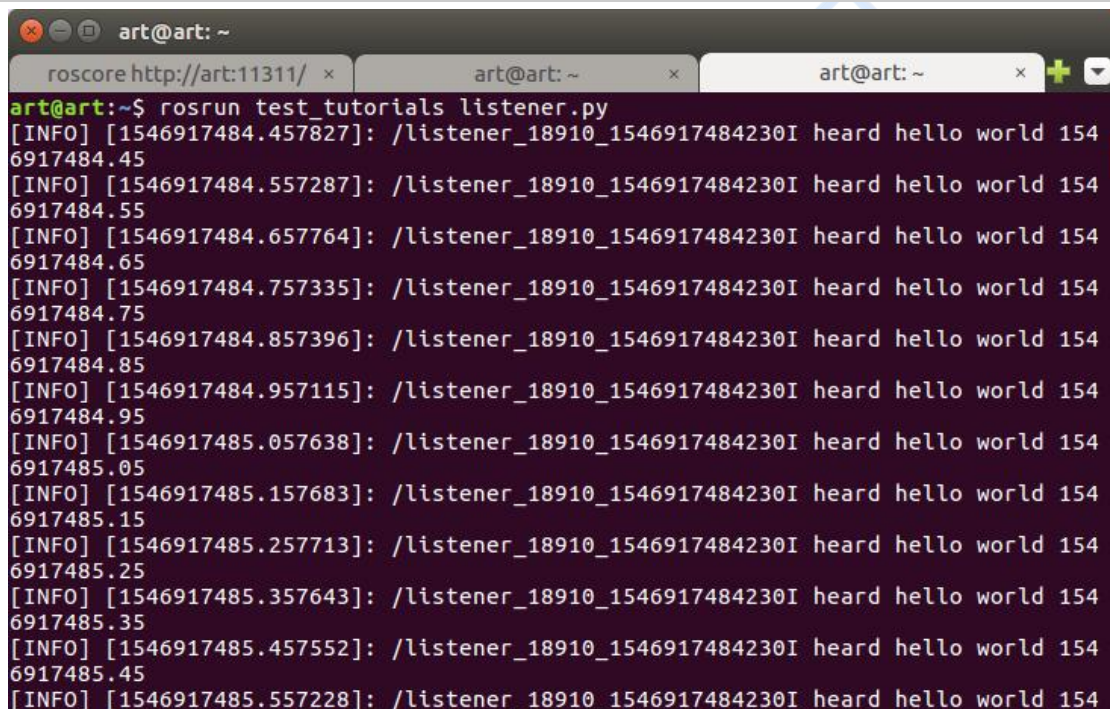
```
$ cd ~/catkin_ws  
$ catkin_make  
$ roscore
```

4、打开新的终端，启动talker.py

```
$ rosrun test_tutorials talker.py
```

5、打开新的终端，启动listener.py

```
$ rosrun test_tutorials listener.py
```



```
art@art:~$ rosrun test_tutorials listener.py  
[INFO] [1546917484.457827]: /listener_18910_1546917484230I heard hello world 154  
6917484.45  
[INFO] [1546917484.557287]: /listener_18910_1546917484230I heard hello world 154  
6917484.55  
[INFO] [1546917484.657764]: /listener_18910_1546917484230I heard hello world 154  
6917484.65  
[INFO] [1546917484.757335]: /listener_18910_1546917484230I heard hello world 154  
6917484.75  
[INFO] [1546917484.857396]: /listener_18910_1546917484230I heard hello world 154  
6917484.85  
[INFO] [1546917484.957115]: /listener_18910_1546917484230I heard hello world 154  
6917484.95  
[INFO] [1546917485.057638]: /listener_18910_1546917484230I heard hello world 154  
6917485.05  
[INFO] [1546917485.157683]: /listener_18910_1546917484230I heard hello world 154  
6917485.15  
[INFO] [1546917485.257713]: /listener_18910_1546917484230I heard hello world 154  
6917485.25  
[INFO] [1546917485.357643]: /listener_18910_1546917484230I heard hello world 154  
6917485.35  
[INFO] [1546917485.457552]: /listener_18910_1546917484230I heard hello world 154  
6917485.45  
[INFO] [1546917485.557228]: /listener_18910_1546917484230I heard hello world 154
```

6、代码分析

代码: `rospy.init_node('listener', anonymous=True)`

分析: 初始化节点, 节点名称为talker, `anonymous=True`, 要求每个节点都有唯一的名称, 避免冲突。这样可以运行多个listener.py

代码: `rospy.Subscriber("chatter", String, callback)`

分析: 订阅函数, 订阅chatter话题, 内容类型是std_msgs.msgs.String。当有新内容, 调用callback函数处理。接受到的话题内容作为参数传递给callback。

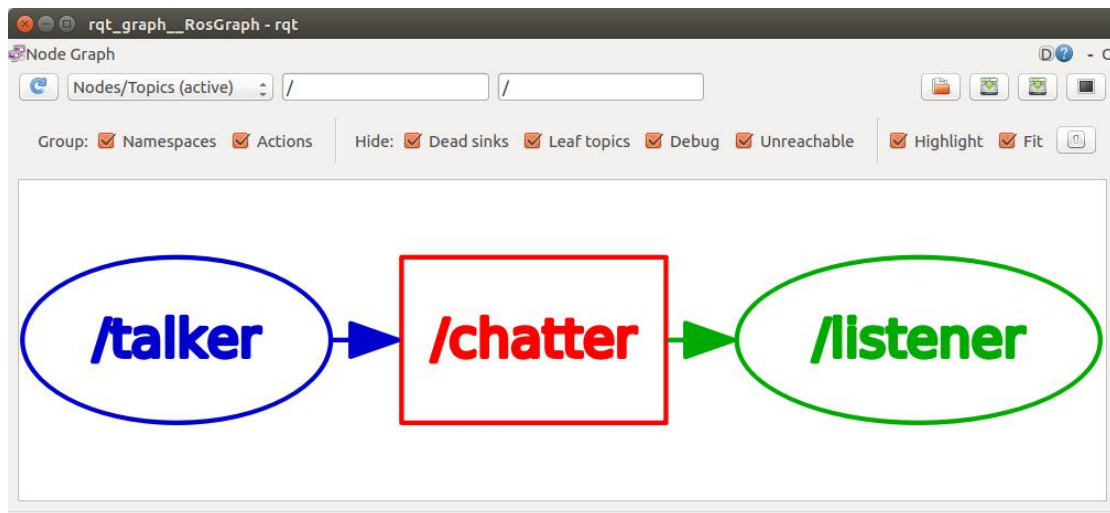
代码: `rospy.spin()`

分析：保持节点运行，直到节点关闭。不像roscpp，rospy.spin不影响订阅的回调函数，因为回调函数有自己的线程。

2.11.4 启动rqt查看节点信息

```
$ rqt_graph
```

在下图中可以看到，节点talker与节点listener通过话题/chatter进行交互，这是节点之间重要的一种交互方式。



2.12 编写服务端和客户端

当需要直接与节点通信并获得应答时，将无法通过话题实现，而需要使用服务。可以说，服务（services）是节点之间通讯的另一种方式。服务允许节点发送请求（request） 并获得一个响应（response）。

本节我们将学习创建节点，一个是服务器，另一个是客户端。

2.12.1 定义srv服务

1、在test_tutorials包里定义新的消息AddTwoInts.srv

```
$ cd ~/catkin_ws/src/test_tutorials/srv
$ touch AddTwoInts.srv
```

进入AddTwoInts.srv，srv文件分为请求和响应两部分，由'---'分隔。手工输入代码：

```
int64 a
```

```
int64 b
---
int64 sum
```

2、编辑package.xml

要确保msg文件被转换成为C++/Python和其他语言的源代码，就必须确保package.xml包含以下两条语句。

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
```

在构建的时候，我们需要"messagegeneration"。然而，在运行的时候，我们需要"message_runtime"。

3、编辑CMakeLists.txt文件

在 CMakeLists.txt文件中，在find_package中，增加对message_generation的依赖，这样就可以生成消息了。

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
```

4、在CMakeLists.txt文件，增加服务文件，取消#，并修改为

```
add_service_files(
  FILES
  AddTwoInts.srv
)
```

5、找到generate_message行，取消注释。

```
## Generate added messages and services with any dependencies listed here
generate_messages(
  DEPENDENCIES
  std_msgs
)
```

6、编辑test_tutorials 中的CMakeLists.txt文件，末尾加入。

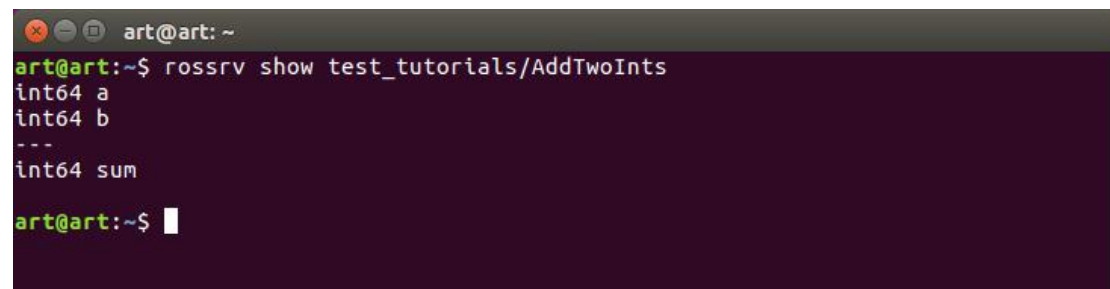
```
add_executable(add_two_ints_server src/add_two_ints_server.cpp)
target_link_libraries(add_two_ints_server ${catkin_LIBRARIES})
add_dependencies(add_two_ints_server test_tutorials_gencpp)

add_executable(add_two_ints_client src/add_two_ints_client.cpp)
target_link_libraries(add_two_ints_client ${catkin_LIBRARIES})
add_dependencies(add_two_ints_client test_tutorials_gencpp)
```

7、使用一下命令编译。

```
$ cd ~
$ cd catkin_ws/
$ catkin_make
```

编译成功后，通过rossrv show命令，检查ROS是否能够识服务，如下图所示，即表示消息创建成功。



```
art@art: ~
art@art:~$ rossrv show test_tutorials/AddTwoInts
int64 a
int64 b
---
int64 sum
art@art:~$
```

2.12.2 编写服务端

python创建服务器：

我们将创建一个简单的service节点("add_two_ints_server")，该节点将接收到两个整形数字，并返回它们的和。

1、在/src目录新建add_two_ints_server.py文件

```
$ touch add_two_ints_server.py
$ chmod +x add_two_ints_server.py
$ roscd test_tutorials add_two_ints_server.py
```

2、编辑代码如下

```
#!/usr/bin/env python
```



```

NAME = 'add_two_ints_server'
# import the AddTwoInts service
from test_tutorials.srv import *
import rospy

def add_two_ints(req):
    print("Returning [%s + %s = %s]" % (req.a, req.b, (req.a + req.b)))
    sum = req.a + req.b
    return AddTwoIntsResponse(sum)

def add_two_ints_server():
    rospy.init_node(NAME)
    s = rospy.Service('add_two_ints', AddTwoInts, add_two_ints)
    print "Ready to add Two Ints"
    # spin() keeps Python from exiting until node is shutdown
    rospy.spin()

if __name__ == "__main__":
    add_two_ints_server()

```

3、编译工作空间，并启动roscore

```

$ cd ~/catkin_ws
$ catkin_make
$ roscore

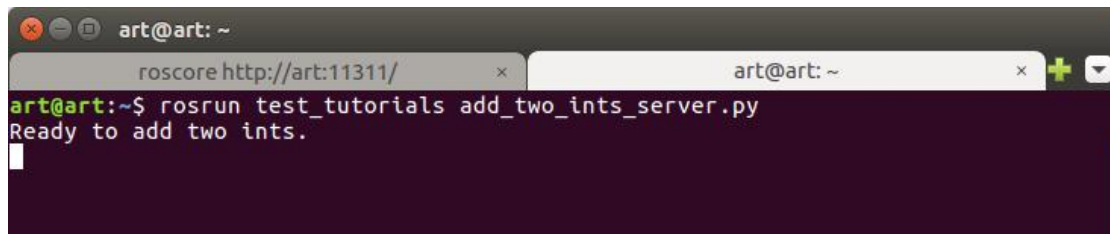
```

4、打开新的终端，启动add_two_ints_server.py

```

$ rosrunc test_tutorials add_two_ints_server.py
Ready to add Two Ints

```



5、查看服务列表，检验服务是否启动

```

art@art:~$ rosservice list

```

```
/add_two_ints
/add_two_ints_server/get_loggers
/add_two_ints_server/set_logger_level
/rosout/get_loggers
/rosout/set_logger_level
/talker_17812_1546916546785/get_loggers
/talker_17812_1546916546785/set_logger_level
```

可以看到 /add_two_ints服务已经启动，我们来检验该服务所需输入的参数。

```
$ rosservice args /add_two_ints
a b
```

也就是说我们在调用服务的时候需要输入两个参数 a b。

6、打开新的终端，调用服务add_two_ints

```
$ rosservice call /add_two_ints 1 2

sum: 3
```

7、代码分析

代码：from test_tutorials.srv import *

分析：导入定义的服务

代码：s = rospy.Service('add_two_ints', AddTwoInts, add_two_ints)

分析：定义服务节点名称，服务的类型，处理函数。

代码：return AddTwoIntsResponse(sum)

分析：AddTwoIntsResponse由服务生成的返回函数

2.12.3 编写客户端代码

python创建客户端：

1、在/src目录新建add_two_ints_client.py文件

```
$ touch add_two_ints_client.py
$ chmod +x add_two_ints_client.py
```

2、进入add_two_ints_client.py输入代码


```

#!/usr/bin/env python

import sys
import os
import rospy

# imports the AddTwoInts service
from test_tutorials.srv import *

## add two numbers using the add_two_ints service
def add_two_ints_client(x, y):
    # NOTE: you don't have to call rospy.init_node() to make calls against
    # a service. This is because service clients do not have to be nodes.
    # block until the add_two_ints service is available
    # you can optionally specify a timeout

    rospy.wait_for_service('add_two_ints')
    try:
        # create a handle to the add_two_ints service
        add_two_ints = rospy.ServiceProxy('add_two_ints', AddTwoInts)
        print "Requesting %s+%s"%(x, y)
        # simplified style
        resp1 = add_two_ints(x, y)
        # formal style
        resp2 = add_two_ints.call(AddTwoIntsRequest(x, y))
        if not resp1.Sum == (x + y):
            raise Exception("test failure, returned sum was %s"%resp1.sum)
        if not resp2.Sum == (x + y):
            raise Exception("test failure, returned sum was %s"%resp2.sum)
        return resp1.Sum
    except rospy.ServiceException, e:
        print "Service call failed: %s"%e

def usage():
    return "%s [x y]"%sys.argv[0]

if __name__ == "__main__":
    argv = rospy.myargv()
    if len(argv) == 1:

```

```

import random

x = random.randint(-50000, 50000)
y = random.randint(-50000, 50000)

elif len(argv) == 3:
    try:
        x = int(argv[1])
        y = int(argv[2])
    except:
        print usage()
        sys.exit(1)
else:
    print usage()
    sys.exit(1)

print "%s + %s = %s"%(x, y, add_two_ints_client(x, y))

```

3、编译工作空间

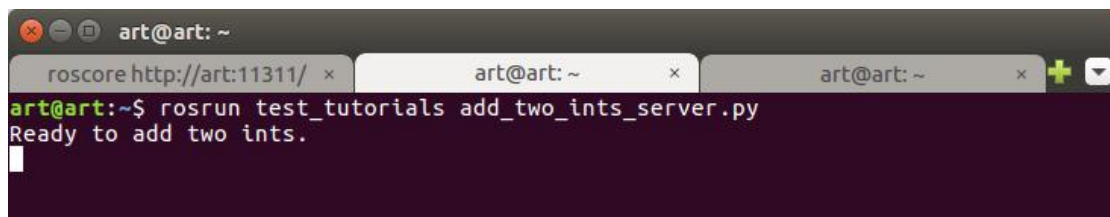
```

$ cd ~/catkin_ws/
$ catkin_make

```

4、先启动service代码，然后运行client代码

```
$ rosrn test_tutorials add_two_ints_server.py
```

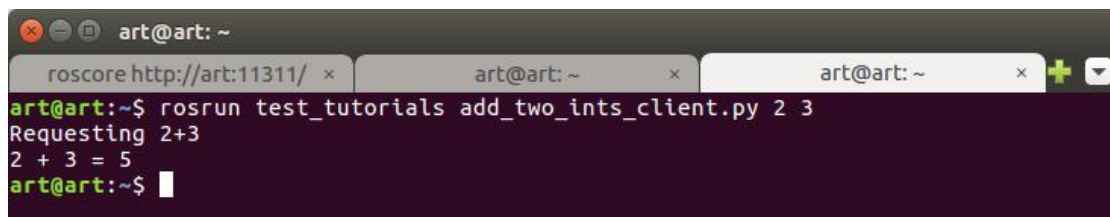


A terminal window titled 'art@art: ~' with three tabs. The first tab shows 'roscore http://art:11311/'. The second tab shows the command 'art@art:~\$ rosrn test_tutorials add_two_ints_server.py' and the output 'Ready to add two ints.' followed by a cursor.

```

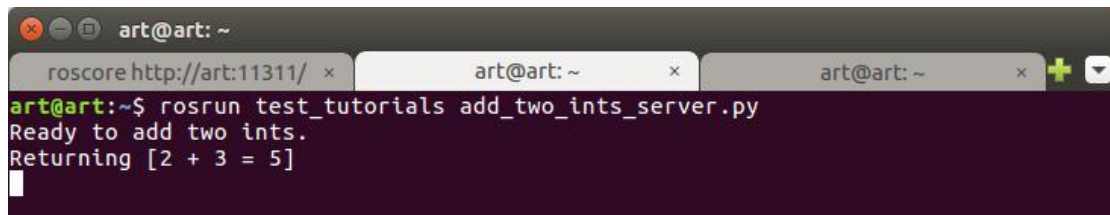
$ rosrn test_tutorials add_two_ints_client.py 2 3
Requesting 2+3
2 + 3 = 5

```



A terminal window titled 'art@art: ~' with three tabs. The first tab shows 'roscore http://art:11311/'. The second tab shows the command 'art@art:~\$ rosrn test_tutorials add_two_ints_client.py 2 3' and the output 'Requesting 2+3' and '2 + 3 = 5'. The third tab shows the prompt 'art@art:~\$' followed by a cursor.

同时add_two_ints_server.py的界面出现反馈:

A terminal window with a dark background and light text. The prompt is 'art@art: ~'. The command 'roscore http://art:11311/' is in the title bar. The terminal shows the command 'rosrun test_tutorials add_two_ints_server.py' being executed. The output is 'Ready to add two ints.' followed by 'Returning [2 + 3 = 5]' on the next line.

```
art@art: ~$ rosrun test_tutorials add_two_ints_server.py
Ready to add two ints.
Returning [2 + 3 = 5]
```

可以看到client已经完成和service的通信，实现了对两个数的求和。

5、代码分析

代码：`rospy.wait_for_service('add_two_ints')`

分析：等待接入服务节点，也就是运行client之前启动service

代码：`add_two_ints = rospy.ServiceProxy('add_two_ints', AddTwoInts)`

分析：创建服务的处理函数，参考rospy.ServiceProxy api

2.13 编写launch文件

机器人是一个复杂的结构,通常让一个机器人完成一项任务时候,需要开启很多个节点,这并不是一个好的操作系统,所以,ROS加入了launch file,这样,我们并不需要每个节点依次进行roscore, ROS就可以使用一个命令一次性启动master和多个node的操作方式。即启动文件(launch file),在 ROS 功能包中,启动文件的使用是非常普遍的。任何包含两个或两个以上节点的系统都可以利用启动文件来指定和配置需要使用的节点。其基本思想是在一个XML格式的文件内将需要同时启动的一组节点罗列出来。

2.13.1 roslaunch使用方法

roslaunch的使用方法为:

```
$ roslaunch pkg-name launch-file-name
```

下面以一个典型的launch文件举例说明:

```

<launch>

  <!-- these are the arguments you can pass this launch file, for example paused:=true -->
  <arg name="debug" default="true"/>

  <!-- We resume the logic in empty_world.launch, changing only the name of the world to be launched -->
  <include file="$(find gazebo_ros)/launch/empty_world.launch">
    <arg name="debug" value="$(arg debug)" />
  </include>

  <!-- Load the URDF into the ROS Parameter Server -->
  <arg name="model" />
  <param name="robot_description"
    command="$(find xacro)/xacro.py $(arg model)" />

  <!-- Run a python script to the send a service call to gazebo_ros to spawn a URDF robot -->
  <node name="urdf_spawner" pkg="gazebo_ros" type="spawn_model" respawn="false" output="screen"
    args="-urdf -model robot1 -param robot_description -z 0.05"/>

</launch>

```

1、launch

每个launch文件都必须且只能包含一个根元素。根元素由一对launch标签定义，其他所有元素标签都应该包含在这两个标签之内。

```

<launch>
...
</launch>

```

2、arg

roslaunch支持启动参数arg，可以通过设置arg来改变程序的运行。name为启动参数的名称，default为该参数的默认值，value为该参数的参数值。在launch文件中，要声明一个参数的存在，我们使用arg元素：

```
<arg name="arg-name" />
```

声明里面只有一个 name 是起不上什么作用的，这就像是你在程序中定义了一个 int 类型的变量，但 是你并没有使用它一样。(你至少还需要给 arg 元素分配 default 属性或 value 属性)

在launch文件中使用的每一个argument必须给它分配一个 value（赋值）。有以下两种实现方法：

在命令行中你可以这样做：给 roslaunch 提供一个 value：

```
roslaunch package-name launch-file-name arg-name:=arg-value
```

另外，在launch文件中，你可以提供一个 value（赋值） 作为 arg 声明的一部分，使用下面的两种语法之一就可以：

```
< arg name="arg-name" default="arg-value" />
< arg name="arg-name" value="arg-value" />
```

这两种语法的唯一不同是：命令行可以覆盖default的值，但是不能覆盖 value 。在例子launch文件中，use_sim3 节点的 default 值为 0，所以它可以通过命令行改变值。如果强行修改value的值系统就会报错。

\$() 这个符号出现的任何地方，roslaunch命令都将会把它替换成给定argument 的值(value)。使用以下命令可以获取argument的值。

```
$(arg arg-name)
```

在argument的传递上有一个限制，就是argument不能传递给 include 元素里包含的子launch文件使用。这个问题非常重要，因为这个 argument 就像是一个局部变量，它不能被包含的launch文件 所 “继承” 。

解决这个问题的方法：在 include 元素中插入 arg 元素作为 include 元素的子类(children)，就像是这样：

```
< include file="path-to-launch-file">
<arg name="arg-name" value="arg-value"/>
...
< /include>
```

注意，这里的 arg 元素不同于我们已经知道的 arg 声明，在 include 标签内的arguments是给包含 (included) 的launch文件提供的arguments，不是为本launch文件提供的。一种常见的情况是，被包含(include)的launch文件和本launch文件会有共同的参数。

在这种情况下，我们希望这些值(values)永远不变。像这样的元素，在这两个地方使用相同的argument name （参数名），要这样做：

```
< arg name="arg-name" value="$(arg arg-name)" />
```

在这种情况下，第一个 arg-name 和往常一样。第二个 arg-name 是launch文件中提供的。结果是，这两个launch文件中给定的argument具有相同的值(value)。

3、node

任何一个launch文件的重点都是：节点(node)元素的集合。节点的形式为：

```
<node name="node-name" pkg="pkg-name" type="executable-name" />
```

node的三个属性分别为节点名字、程序包名字和 可执行文件的名字。 **name**属性给节点指派了名称，它将覆盖任何通过调用 **ros::init**来赋予节点的名称。另外**node**标签内也可以用**arg**设置节点参数值。如果**node**标签有**children**标签，就需要显式标签来定义。即末尾为**/node>**。

- ◆ 设置**output = "screen"**：将标准输出信息显示在终端(console)上。
- ◆ 设置**respawn=" true"**：启动完所有请求启动的节点之后，**roslaunch** 监测每一个节点，让它们保持正常的运行状态。对于每一个节点，当它终止时，我们可以要求 **roslaunch** 重新启动它。
- ◆ 设置**required=" true"**：当一个必需的节点终止时，**roslaunch**会做出响应，终止其他所有的节点并退出它自己。

4、remap标签

在**launch**文件中重新命名：使用 **remap** 元素：

```
<remap from="original-name" to="new-name" />
```

如果这个 **remap** 是 **launch** 元素的一个**child**（子类），与 **node** 元素同一层级，并在 **launch** 元素内的最顶层。那么这个 **remapping** 将会作用于后续所有的节点。

5、include标签

在**launch**文件中复用其他**launch**文件可以减少代码编写的工作量，提高文件的简洁性。使用包含元素**include**在**launch**文件中可包含其他**launch**文件中所有的节点和参数。

```
<include file="$(find pkg-name)/launch/launch-file-name">  
  <arg name="arg_name" value="set-value"/>  
</include>
```

6、Param

在ROS中**parameter**和**argument** 是不同的，虽然翻译一样。**parameter**是运行中的ROS系统使用的数值，存储在参数服务器（**parameter server**）中，每个活跃的节点都可以通过 **ros::param::get** 函数来获取**parameter**的值，用户也可以通过**rosparam**来获得**parameter**的值，而**argument**只在启动文件内才有意义他们的值是不能被节点直接获取的。

在**launch**文件中设置**parameter**，使用**param**标签：

```
<param name="param-name" value="param-value" />
```

在launch文件中也支持等同与rosparam load 功能的rosparam标签，用于一次性加载大量的参数，其中file是.yaml类型。

```
<rosparam command="load" file="path-to-param-file" />
```

2.13.2 编写launch文件

此处通过两个实例详细说明launch文件的编写：

实例一：编写talker-and-listener.launch

以发布器与订阅器章节的talker和listener节点为例，来说明如何编写launch文件。

1、在test_tutorials目录下新建launch目录

```
$ roscd test_tutorials
$ mkdir launch
```

2、进入launch，新建talker-and-listener.launch

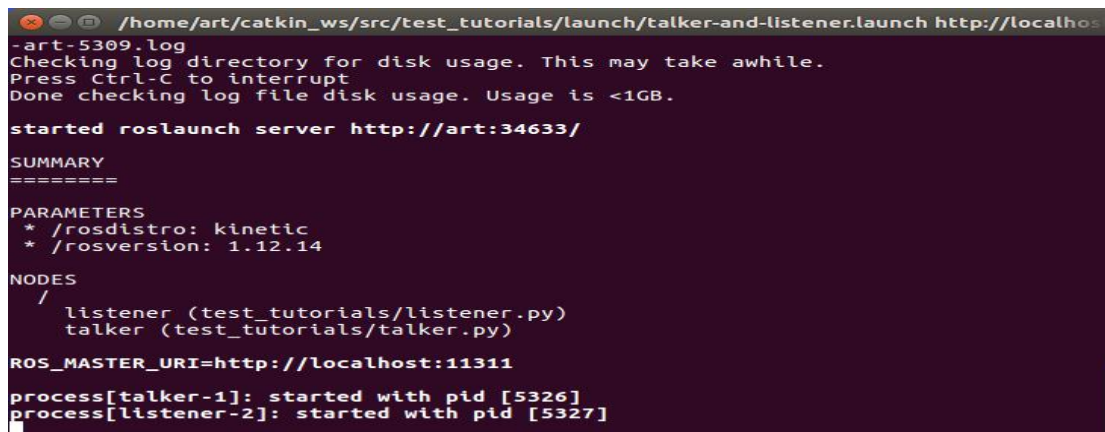
```
$ cd launch
$ touch talker-and-listener.launch
```

3、输入代码：

```
<launch>
  <node name="talker" pkg="test_tutorials" type="talker.py" />
  <node name="listener" pkg="test_tutorials" type="listener.py" />
</launch>
```

4、运行launch文件

```
$ roslaunch test_tutorials talker-and-listener.launch
```



```
/home/art/catkin_ws/src/test_tutorials/launch/talker-and-listener.launch http://localhost:11311
- art-5309.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://art:34633/

SUMMARY
=====
PARAMETERS
* /rostdistro: kinetic
* /rosversion: 1.12.14
NODES
/
  listener (test_tutorials/listener.py)
  talker (test_tutorials/talker.py)
ROS_MASTER_URI=http://localhost:11311
process[talker-1]: started with pid [5326]
process[listener-2]: started with pid [5327]
```

实例二：编写启动小乌龟launch

首先切换到test_tutorials文件下：

```
$ cd ~/catkin_ws  
$ source devel/setup.bash  
$ roscd test_tutorials
```

创建launch文件夹，切换到该文件夹下：

```
$ cd launch
```

注意:launch文件是在相应的功能包文件里面。

然后在这个文件夹下新建一个launch文件，命名为turtlemimic.launch，编辑文件：

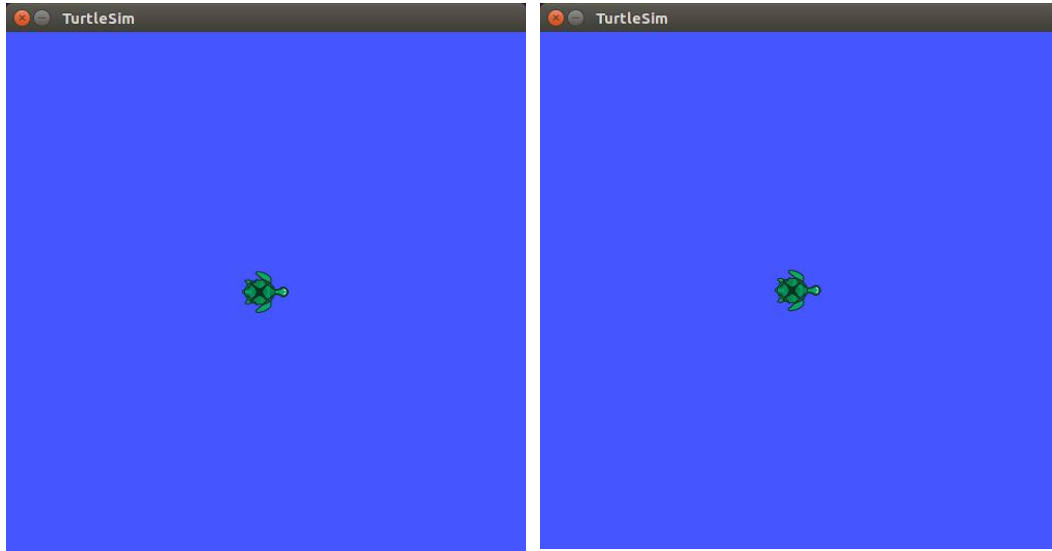
```
$ gedit turtlemimic.launch
```

输入以下内容：

```
<launch>  
  
  <group ns="turtlesim1">  
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>  
  </group>  
  
  <group ns="turtlesim2">  
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>  
  </group>  
  
  <node pkg="turtlesim" name="mimic" type="mimic">  
    <remap from="input" to="turtlesim1/turtle1"/>  
    <remap from="output" to="turtlesim2/turtle1"/>  
  </node>  
  
</launch>
```

保存退出。使用如下命令启动launch文件：

```
$ roslaunch test_tutorials turtlemimic.launch
```

这时就能看到两只小乌龟，但是这时用`rostopic list`查询，能看到新增三个节点。

```
art@art:~$ rostopic list
/mimic
/rosout
/rostopic_5463_1547005740563
/turtlesim1/sim
/turtlesim2/sim
```

在一个新打开的终端中输入：

```
$ rostopic pub /turtlesim1/turtle1/cmd_vel geometry_msgs/Twist -r 1 -- '[2.0 , 0.0 , 0.0]' '[0.0 , 0.0 , -1.8]'
```



就可以看到两只乌龟都在转圈。

下面来解释一下原因，分析一下刚才的那个`launch`文件：

<launch>标签用来识别这是一个launch文件。

```
<group ns="turtlesim1">
  <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
</group>

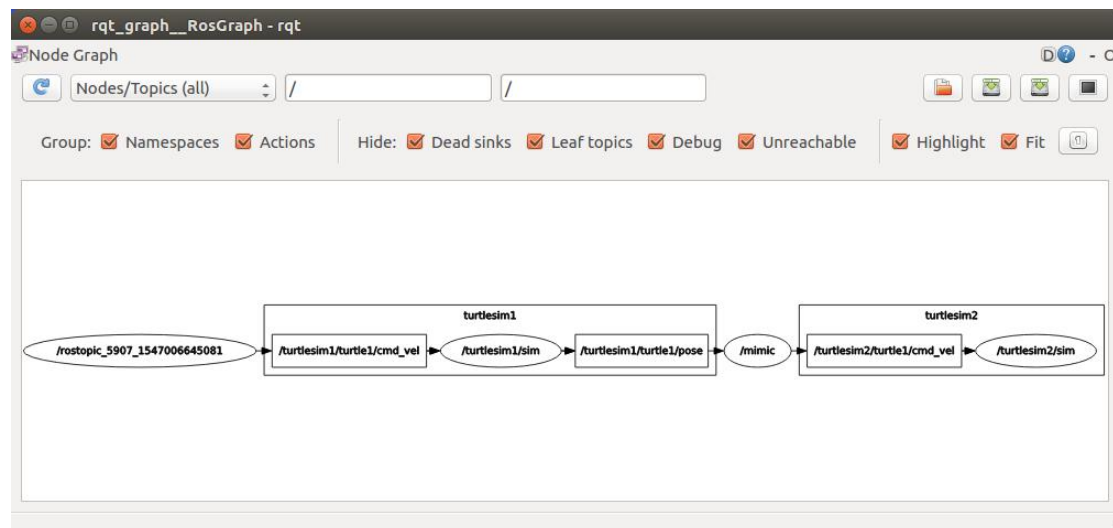
<group ns="turtlesim2">
  <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
</group>
```

此处代码表示我们启动了两个乌龟节点，名字都是sim，由于他们在不同的命名空间中，所以不会有相同名字的冲突。

```
<node pkg="turtlesim" name="mimic" type="mimic">
  <remap from="input" to="turtlesim1/turtle1"/>
  <remap from="output" to="turtlesim2/turtle1"/>
</node>
```

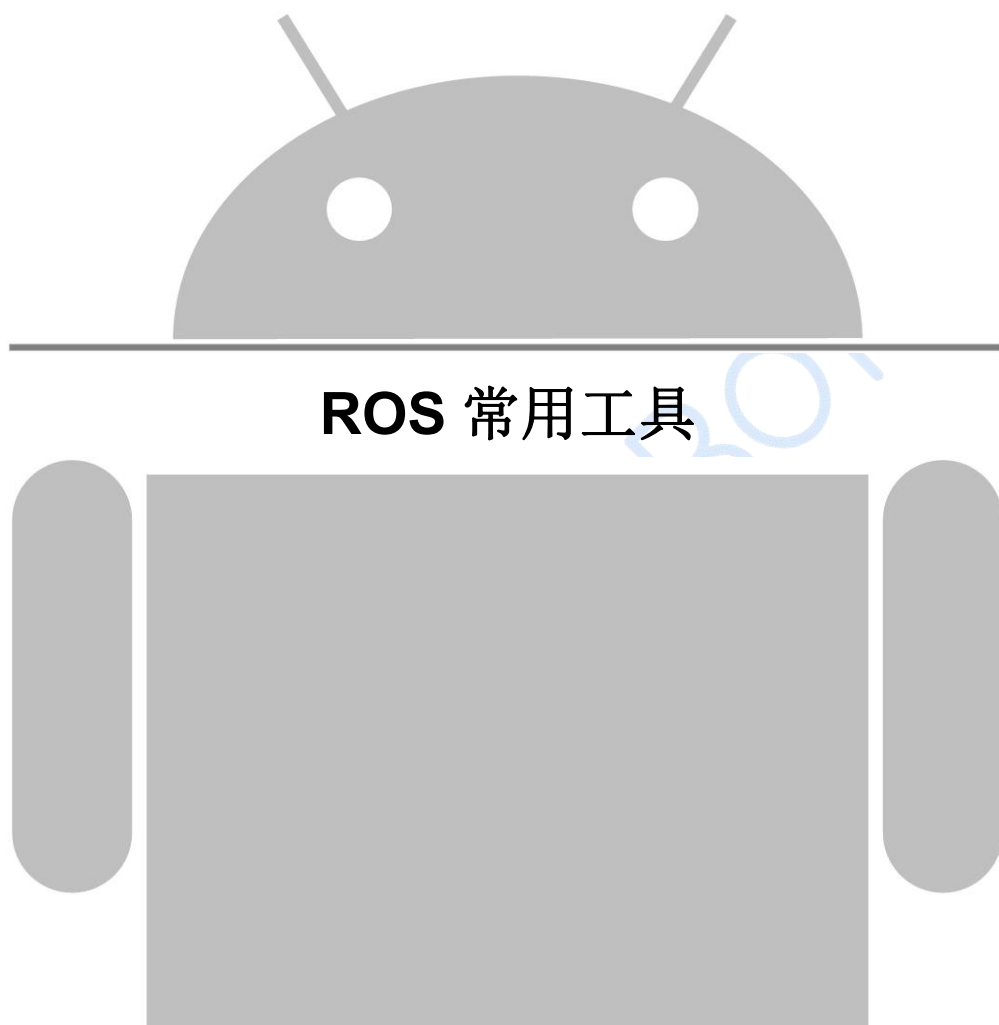
此处代码启动了mimic节点，同时把这个节点的input和output话题重命名为turtle1和turtle2。

这时通过rqt_graph显示的信息：



rqt显示turtlesim2模仿turtlesim1，所以虽然我们只向turtlesim1发布了话题，turtlesim2也做出了相应的反应。

Chapter 3



3.1 Rviz

RViz是ROS的三维可视化工具。它的主要目的是以三维方式显示ROS消息，可以将数据进行可视化表达。例如，可以无需编程就能表达激光测距仪（LRF）传感器中的传感器到障碍物的距离，RealSense、Kinect或Xtion等三维距离传感器的点云数据（PCD，Point Cloud Data），从相机获取的图像值等。

另外，利用用户指定的多边形（polygon）支持各种表现形式，交互标记可以表达接收来自用户节点的命令和数据并互交的过程。在 ROS中，机器人以URDF（Unified Robot Description Format，统一机器人描述格式）描述，它可以表示为三维模型，并且每个模型可以根据自由度进行移动或驱动，因此可以用于仿真或控制。

3.1.1 RViz 安装和使用

安装方法（完整版ROS安装包已经集成了rviz系统）：

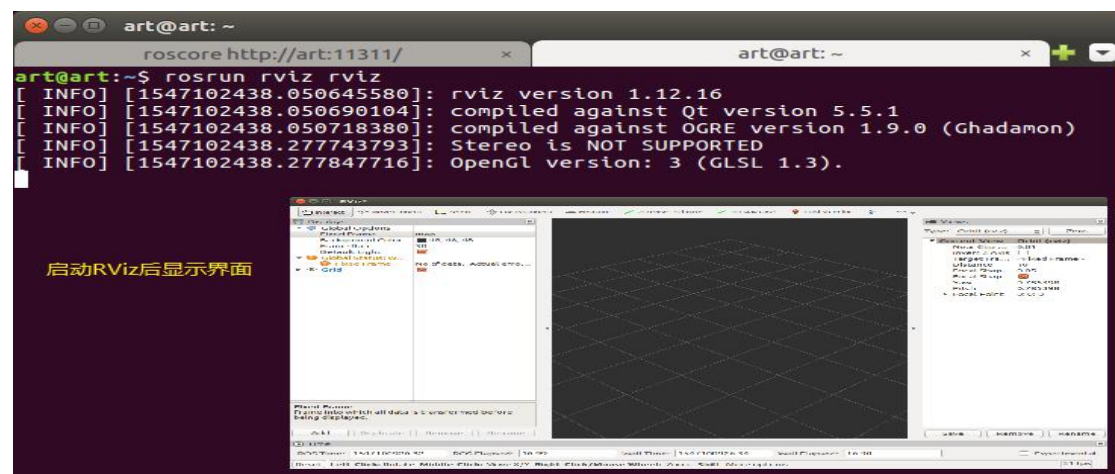
```
$ sudo apt-get install ros-kinetic-rviz
```

环境检测、安装：

```
$ rosdep install rviz
$ rosmake rviz
$ source /opt/ros/kinetic/setup.bash
```

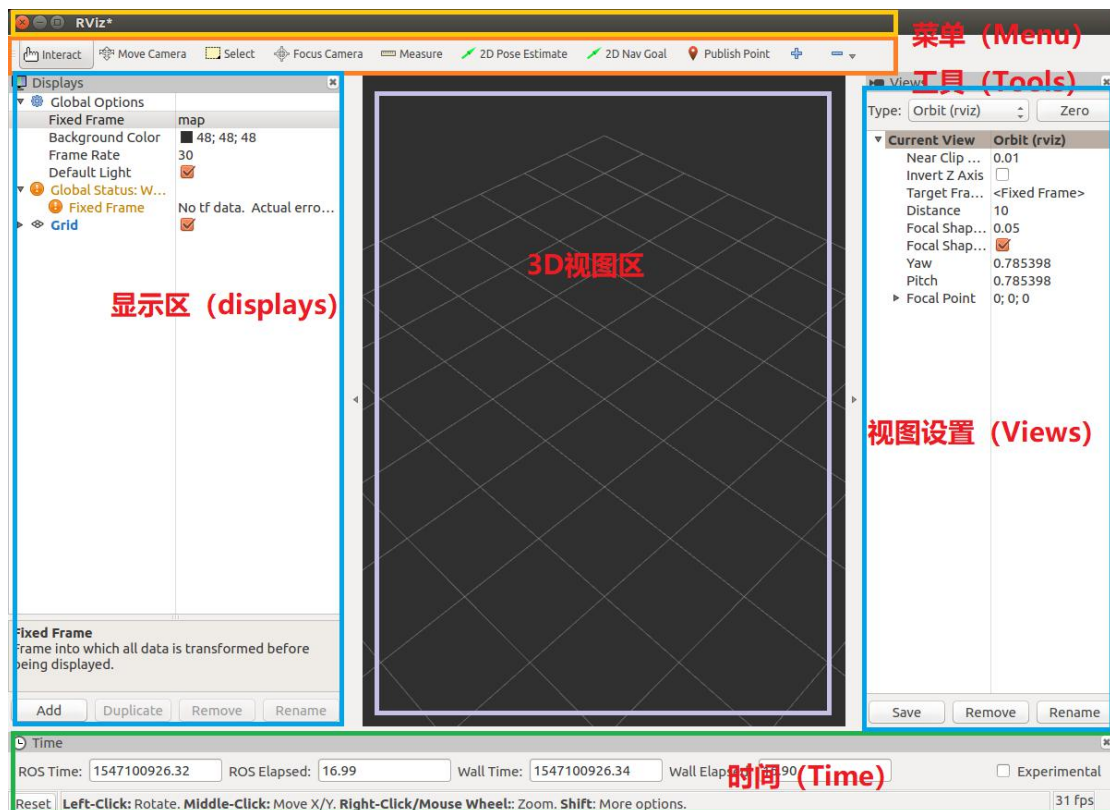
启动RViz平台：

```
$ roscore
$ rosrn rviz rviz
```



3.2.1 RViz 界面显示

RViz整体界面如下图所示：



由上到下分别是：

- 1、菜单（Menu）：菜单位于顶部。用户可以选择保存或读取显示屏状态的命令，还可以选择各种面板。
- 2、工具（Tools）：工具是位于菜单下方的按钮，允许用户用各种功能按键选择多种功能的工具，例如 Interact、Move Camera、Select, Focus Camera、Measure、2D Pose Estimate、2D Navigation Goal 以及 Publish Point 等。
- 3、显示（Displays）：左侧的显示屏是从各种话题当中选择用户所需的数据的视图的区域。如果单击屏幕4 左下方的[Add]，选择屏幕将如图所示。目前有大约30种不同的显示屏可供选择，我们将在下面的描述中详细介绍。
- 4、3D视图区（3D view）：指屏幕的黑色部分。它是可以用三维方式查看各种数据的主屏幕。3D视图的背景颜色、固定框架、网格等可以在左侧显示的全局选项（Global Options）和网格（Grid）项目中进行详细设置。
- 5、视图设置（Views）：设定三维视图的视点

◆ **Orbit**: 以指定的视点（在这里称为**Focus**）为中心旋转。这是默认情况下最常用的基本视图。

◆ **FPS**（第一人称）：显示第一人称视点所看到的画面。

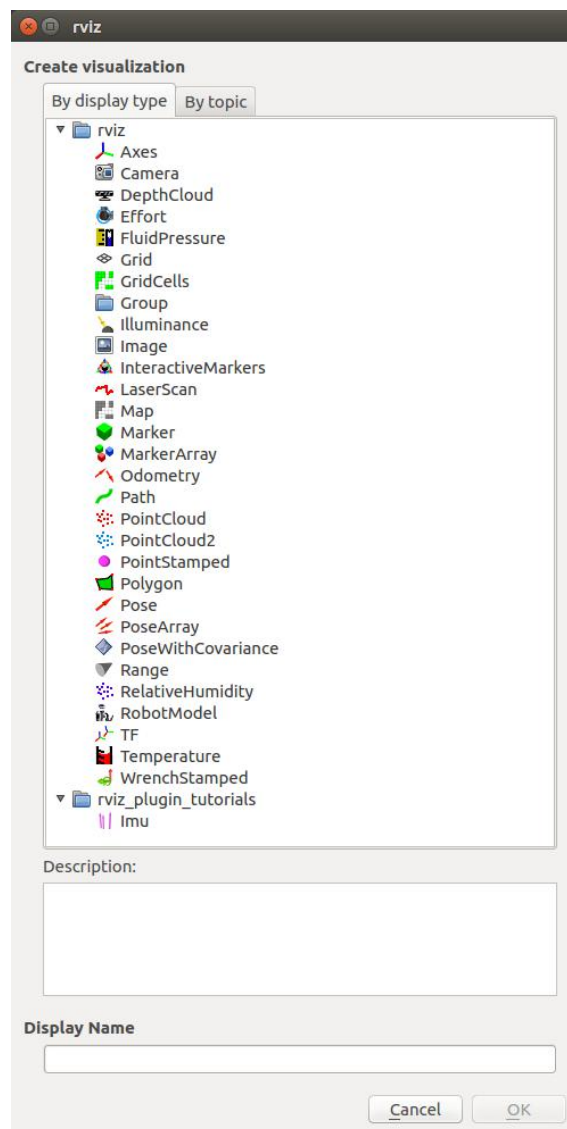
◆ **ThirdPersonFollower**: 显示以第三人称的视点尾追特定目标的视图。

◆ **TopDownOrtho**: 这是Z轴的视图，与其他视图不同，以直射视图显示，而非透视法。

◆ **XYOrbit**: 类似于Orbit的默认值，但焦点固定在Z轴值为0的XY平面上。

6、**时间（Time）**：显示当前时刻（**wall time**）、**ROS Time**以及他们各自经过的时间。这主要用于仿真，如果需要重新启动，请点击底部的**[Reset]**按钮。

使用RViz的过程中最常用的菜单应该是**displays**菜单。该**displays**菜单用于选择三维视图（**3D View**）画面所显示的信息，RViz displays的选择画面如下：



其中各项目的解析通过<http://wiki.ros.org/rviz/DisplayTypes>详细查看。

3.2 Rqt

rqt是一个基于qt开发的可视化工具，拥有扩展性好、灵活易用、跨平台等特点，主要作用和 RViz一致都是可视化，但是和RViz相比，rqt要高级一个层次。

ROS机器操作系统为机器人开发提供各种GUI工具，除了RViz和Gazebo还有一个将每个节点的层次结构显示为图形，且显示当前节点和话题状态的graph；将消息显示为二维图形的plot等功能的一个工具——rqt。从ROS Fuerte版本开始，这些GUI开发工具被称为rqt，它集成了30多种工具，可以作为一个综合的GUI工具来使用。另外，RViz也被集成到rqt的插件中，这使rqt成为ROS的一个不可缺少的GUI工具。

3.1.1 rqt 安装和使用

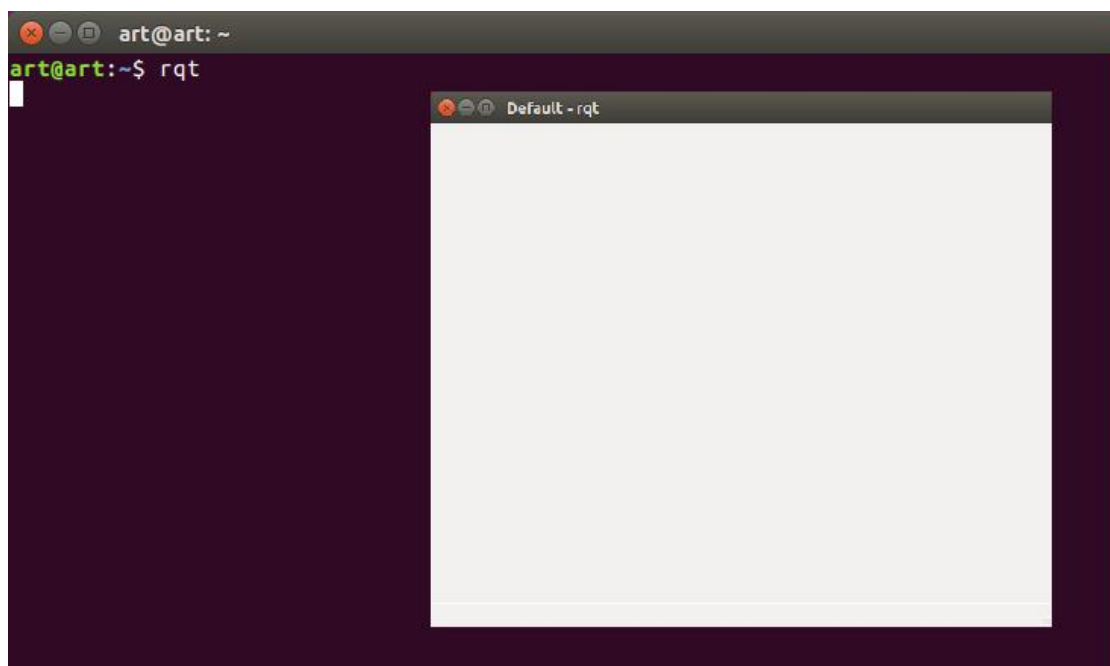
安装方法（完整版ROS安装包已经集成了rqt系统）：

```
$ sudo apt-get install ros-kinetic-rqt*
```

运行rqt的命令如下。只需键入rqt。

```
$ rqt
```

运行rqt将显示rqt的GUI界面，如下图所示。如果是第一次，它将只显示一个纯净界面，没有任何内容。这是因为还没有指定rqt直接运行的插件程序。



同时会有下拉菜单显示在界面左上端，其各个菜单内容如下：

- 1、文件（File）：只有一个退出rqt的子菜单。

- 2、插件（Plugins）：有30多个插件。可以选择并使用它。
- 3、动作（Running）：显示当前运行的插件，在不需要的时候可以停止。
- 4、全景（Perspectives）：用于保存当前运行的插件组，并在下次运行相同的插件组。

3.1.2 rqt 主要插件

rqt插件大约有30个，rqt中部分默认插件是非常有用的，在我们做开发时候具有非常有效的功能。而且可以将非官方的插件添加进来，满足特定用户的需求，所以如果有需要，用户完全可以添加自己开发的rqt插件。下面我们介绍以下几个常用的rqt插件：

1、rqt_graph

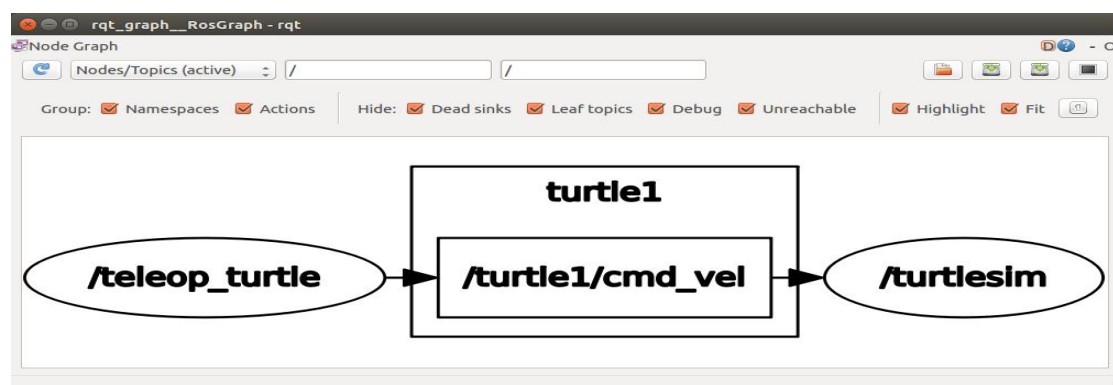
rqt_graph是用图形表示当前活动中的节点与在ROS网络上传输的消息之间的相关性的工具。这对了解当前ROS网络情况非常有用。用法很简单，我们通过查看turtlesim功能包中的turtlesim_node和turtle_teleop_key节点的关联情况，将他们分别在不同的终端中运行，然后通过rqt查看。

```
$ roscore
$ rosrun turtlesim turtlesim_node
$ rosrun turtlesim turtle_teleop_key
```

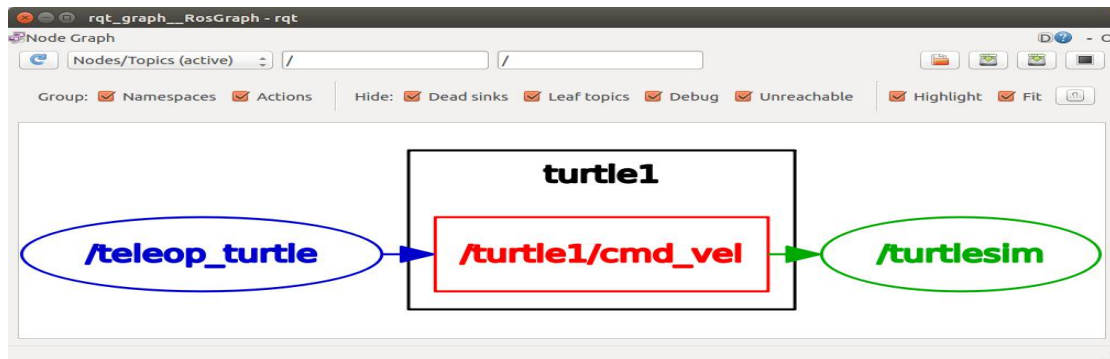
打开一个新命令终端，运行代码：

```
$ rosrun rqt_graph rqt_graph
```

你会看到类似下图所示的图形：



如果你将鼠标放在/turtle1/cmd_vel上方，相应的ROS节点（蓝色和绿色）和话题（红色）就会高亮显示。正如你所看到的，turtlesim和teleop_turtle节点正通过一个名为/turtle1/cmd_vel的话题来互相通信。



在图中，椭圆表示节点（/teleop_turtle、/turtlesim），方块（/turtle1/cmd_vel）表示话题消息。箭头表示发送和接收消息。此例子中，运行turtle_teleop_key时，会运行teleop_turtle节点；运行turtlesim_node节点时，会运行turtlesim节点。可以看到，这两个节点正在以平移速度和旋转速度的消息类型（话题名称：/turtle1/ cmd_vel）发送和接收键盘的方向键值。

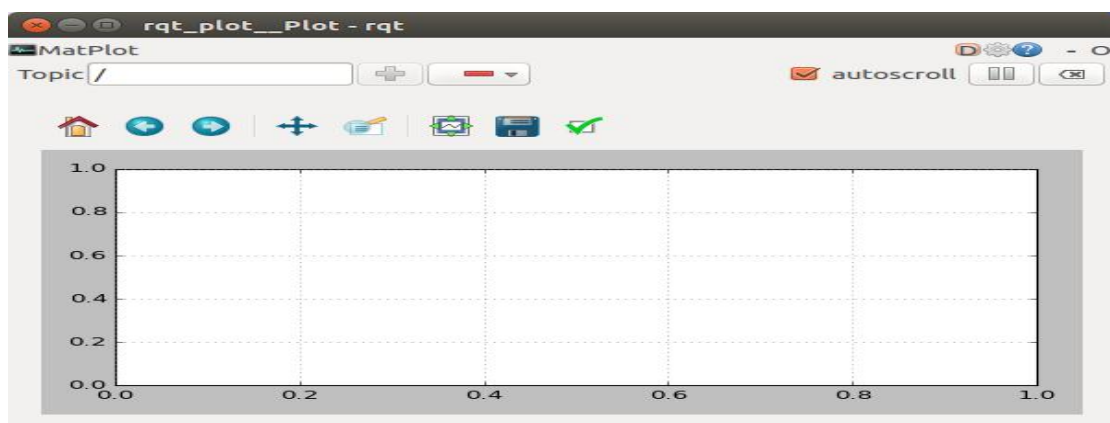
2、rqt_plot

rqt_plot是一个二维数据绘图工具。绘图意味着绘制坐标。换句话说，它接收到ROS消息并将其撒在坐标系上。例如，假设要标记turtlesim节点pose消息的x和y坐标。首先，运行turtlesim功能包中的turtlesim_node。

rqt_plot命令可以实时显示一个发布到某个话题上的数据变化图形。这里我们将使用rqt_plot命令来绘制正在发布到/turtle1/pose话题上的数据变化图形。在一个新终端中运行rqt_plot命令：

```
$ rosrun rqt_plot rqt_plot
```

这会弹出一个新窗口，在窗口左上角的一个文本框里面你可以添加需要绘制的话题。

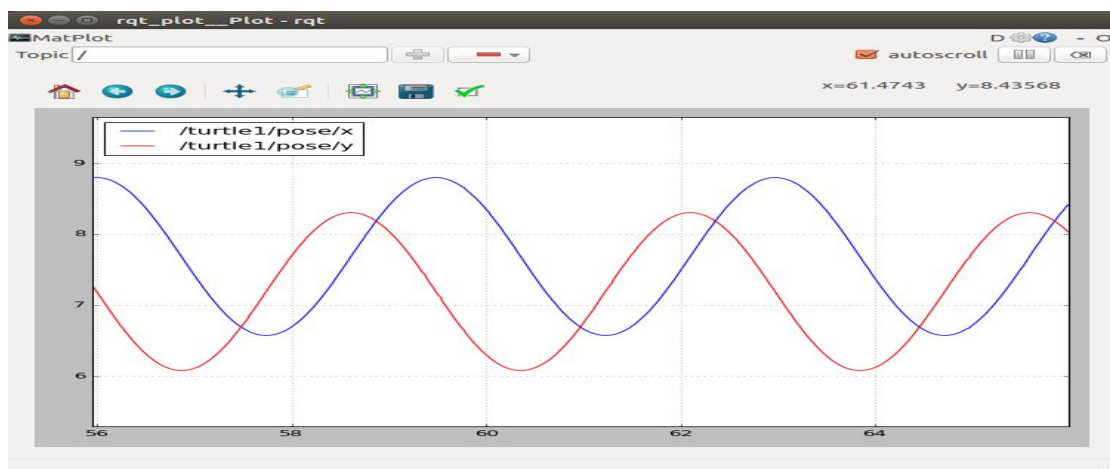


接下来，运行turtlesim功能包中的turtle_teleop_key来移动屏幕上的乌龟。

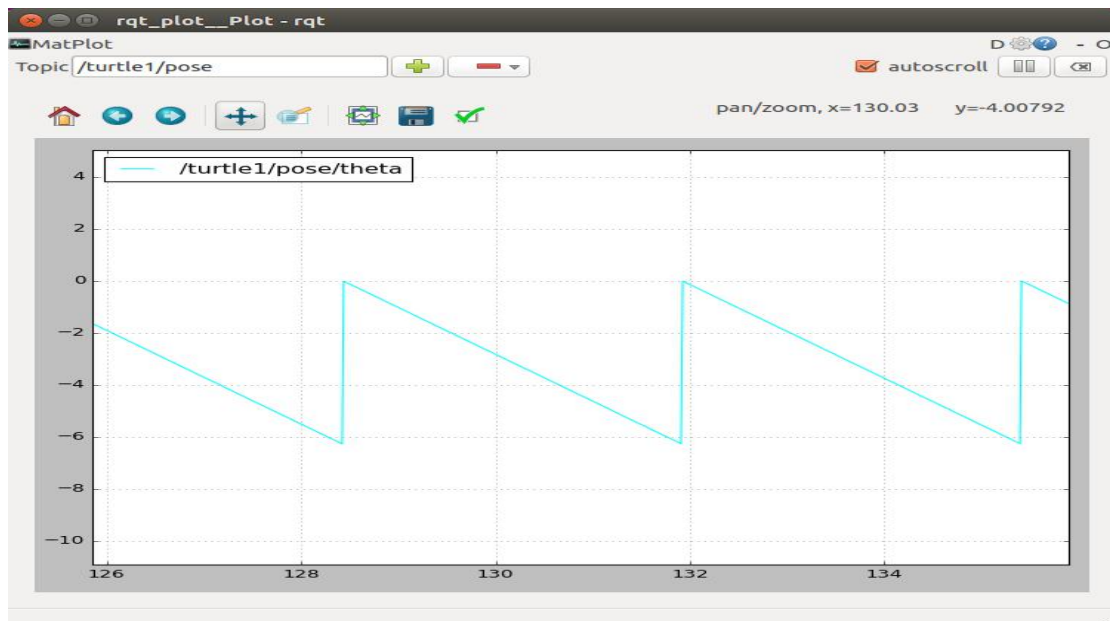
```
$ rosrn turtlesim turtle_teleop_key
```

在rqt界面中输入/turtle1/pose/x后之前处于禁用状态的加号按钮将会被使能变亮。按一下该按钮，并对/turtle1/pose/y重复相同的过程。现在你会在图形中看到turtle的x-y位置坐标图。

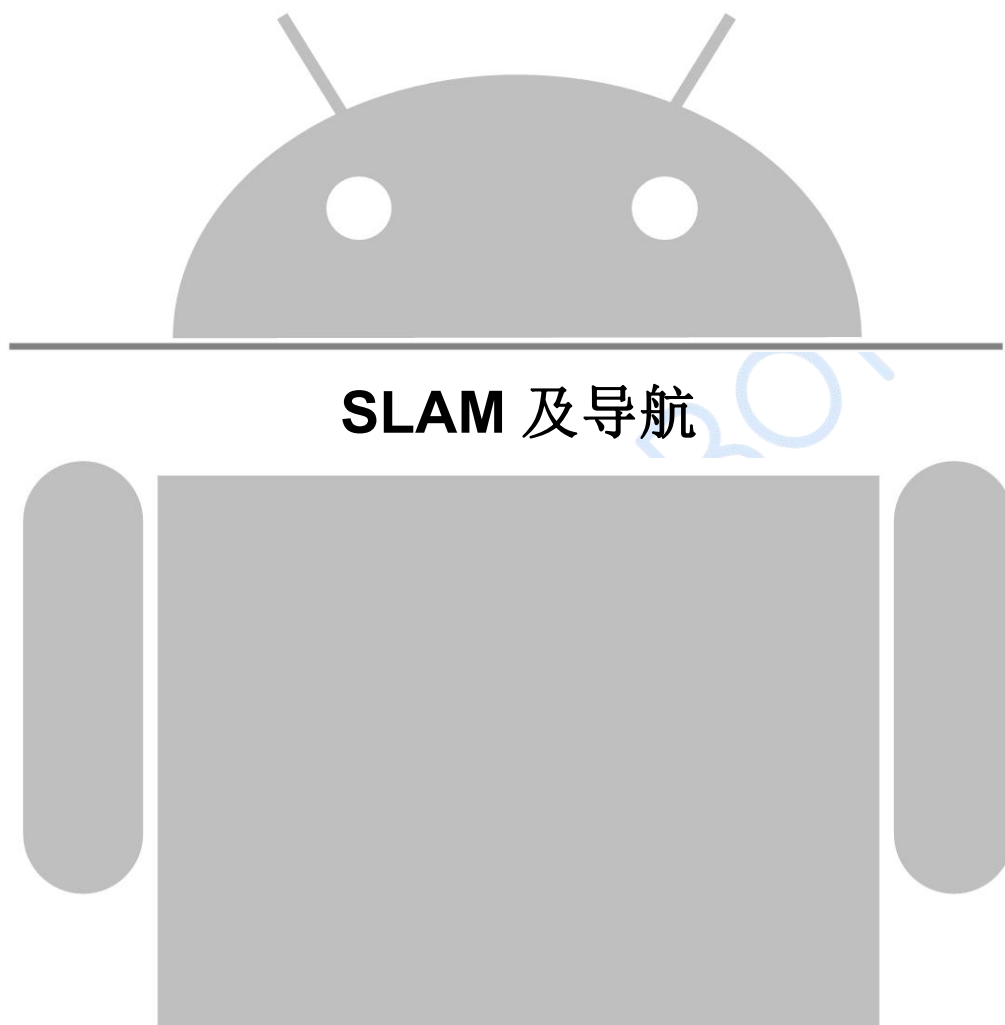
这会弹出一个新窗口，在窗口左上角的一个文本框里面你可以添加需要绘制的话题。在里面输入/turtle1/pose/x后之前处于禁用状态的加号按钮将会被使能变亮。按一下该按钮，并对/turtle1/pose/y重复相同的过程。现在你会在图形中看到turtle的x-y位置坐标图。



按下减号按钮会显示一组菜单让你隐藏图形中指定的话题。现在隐藏掉你刚才添加的话题并添加/turtle1/pose/theta，你会看到如下图所示的图形：



Chapter 4



4.1 SLAM简介

SLAM是Simultaneous Localization and Mapping的缩写，意为“同时定位与建图”。可以描述为：机器人在位置环境中的一个未知位置开始移动，并在移动过程中根据传感器的信息，一边计算自身位置，一边构建环境地图，从而实现机器人的自主定位和导航。目前，SLAM的应用领域主要有机器人、虚拟现实和增强现实。其用途包括传感器自身的定位，以及后续的路径规划、场景理解。

SLAM研究自1988年提出以来，已经过了近三十年。早期SLAM研究侧重于使用滤波器理论，最小化运动体位姿和地图的路标点的噪声。21世纪之后，学者们开始借鉴SfM(Structure from Motion)中的方式，以优化理论为基础求解SLAM问题。这种方式取得了一定的成就，并且在视觉SLAM领域中取得了主导地位。

随着传感器种类和安装方式的不同，SLAM的实现方式和难度会有很大差异。按传感器来分，SLAM主要分为激光、视觉两大类。其中，激光SLAM研究较早，理论和工程均比较成熟。视觉方案目前大部分还处于实验室研究阶段，实际产品应用相对较少。

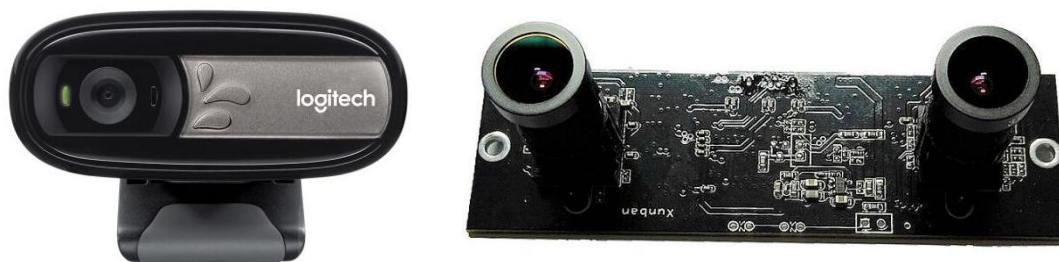
(1) 激光雷达

激光雷达是研究最多、使用最成熟的深度传感器，可以直接获得相对于环境的直接距离信息，从而实现直接相对定位，很多常见的扫地机器人都有装配。激光雷达的优点是精度高，响应快，数据量小；缺点是成本高。现阶段，国内的很多企业也在专注研发高性价比的激光雷达，也有不少优秀的产品已经推向市场。



(2) 摄像头

SLAM所用到的摄像头分为两种：一种是单目摄像头，就是用一个摄像头完成SLAM。这种方案的传感器相对简单，适用性强，但单目摄像头无法再静止状态测量距离，只有在动态状态下才能根据三角测量等原理感知距离。另一种是双目摄像头，与单目摄像头相比，双目摄像头可以感知距离信息，但是标定相对复杂，大量的图像数据会导致运算量过大。



(3) RGB-D摄像头

RGB-D摄像头是一种新兴起的新型传感器，它不仅可以获取环境的RGB信息，也可以通过红外结构光、Time-of-Flight等原理获取每一个像素点的深度信息。由于多了一组深度信息的数据，使得RGB-D摄像头不仅可以用于SLAM，还可以用于图像处理、物体识别等多个方面。但同时也存在视野窄、盲区大、噪声大等缺点需要解决。



4.1.1 SLAM分类

SLAM主要分为两大类，一类是动态环境，另一类是静态环境。在动态环境又分为高动态环境和低动态环境；如果机器人在构建地图时，在视野范围内有正在移动的物体，这种情况定义为高动态环境；如果机器人在构建地图时，在视野范围外有移动的物体，这种情况定义为低动态环境。在静态环境中，根据输出地图的类型可以分为三种：尺度地图、拓扑地图、混合地图。拓扑地图主要是在环境较大的情况下使用，它可以将环境简化成一个个点，如果两点相连我们再连一条边，这条边的长度就是连个点之间的距离。尺度地图就是一个1:1的比例尺地图，无失真。混合地图就是在一个拓扑地图的每一个点上都依附一个尺度地图。而构建尺度地图的方法主要有两种，一种是基于贝叶斯滤波器的构建方法，另一种是基于图优化的构建方法。



化的构建方法。

4.1.2 激光SLAM框架

➤ 传感器信息处理（Sensor information processing）

对于激光SLAM来说，可获取的传感器信息一般有里程计、激光雷达数据以及IMU数据，对于里程计数据，最常见的有两轮差速运动模型和三轮全向底盘运动模型；而激光雷达数据则会由于传感器的移动而导致的采集回的数据产生畸变，需要通过估计或者里程计辅助等方法对畸变进行矫正。

➤ 前端配准方法（Visual Odometry）

在实现VO需要考虑特征点，根据输入激光数据提取特征，之后再对两帧数据的特征点位置进行匹配。当然我们仅仅凭借两帧的估计往往是不够的，我们会把特征点缓存成一个小地图，计算当前帧与地图之间的位置关系。这样才能算的上是一个完整的前端配准方法。

➤ 后端优化 (Optimization)

在后端优化上，我们通常需要考虑更长一段时间或者所有时间上的状态估计问题，不仅仅使用过去的信息更新自己的状态，也会用未来的信息来更新自己。在SLAM的过程中由于可以用运动方程和观测方程描述，那么我们可以列出运动观测方程：

$$\begin{cases} x_k = f(x_{k-1}, v_k) + w_k \\ z_{k,j} = h(y_i, x_k) + v_{k,j} \end{cases} \quad k = 1, \dots, N, j = 1, \dots, M$$

➤ 回环检测 (Loop Closing)

如果只像VO那样考虑相邻时间上的关联，那么之前产生的误差将不可避免地累积到下一个时刻，使得整个SLAM出现累积误差，无法长期估计。而在后端虽然能估计最大后验误差，但只有相邻关键数据时，我们能做的事情并不是很多，无法消除累积误差。回环检测对SLAM系统的意义重大，他关系到我们估计的轨迹和地图在长时间下的正确性，当然也提供了当前数据与所有历史数据的关联，在跟踪算法求职后，我们还可以利用会还检测进行重定位。因此，回环检测对整个SLAM系统精度与稳健性的提升是非常明显的。

➤ 建图 (Mapping)

在SLAM模型中，我们的地图就是所有路标点的集合，一旦确定了各个路标点的位置，那么就可以说我们完成了建图。但即使是地图，也会有许多不同的需求，可大致分为：定位、导航、避障、重现、交互这五个方面。

4.2 激光雷达数据的采集和处理

LiDAR(Light Detection and Ranging)，是激光探测及测距系统的简称，另外也称Laser Radar或LADAR(Laser Detection and Ranging)。用激光器作为发射光源，采用光电探测技术手段的主激光雷达激光雷达遥感设备。激光雷达是激光技术与现代光电探测技术结合的先进探测方式。由发射系统、接收系统、信息处理等部分组成。发射系统是各种形式的激光

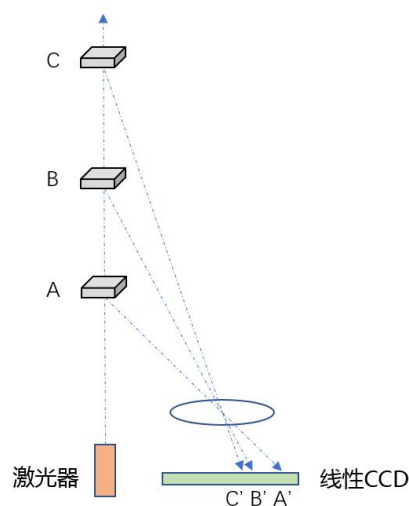
器，如二氧化碳激光器、掺钕钇铝石榴石激光器、半导体激光器及波长可调谐的固体激光器以及光学扩束单元等组成；接收系统采用望远镜和各种形式的光电探测器，如光电倍增管、半导体光电二极管、雪崩光电二极管、红外和可见光多元探测器件等组合。

4.2.1 激光雷达测距的基本原理

对于不同的激光雷达，使用的测距原理并不一样，目前主要分为两种测距方法，一种是三角测距；另一种是飞行时间测距（TOF）。

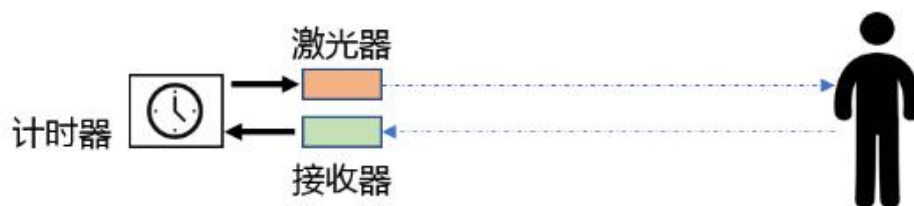
➤ 三角测距

三角测距是一借由测量目标点与固定基准线的已知端点的角度，测量目标距离的方法。而不是直接测量特定位置的距离（三边量测法）。当已知一个边长及两个观测角度时，观测目标点可以被标定为一个三角形的第三个点。三角法的原理如下图所示，激光器发射激光，在照射到物体后，反射光由线性CCD接收，由于激光器和探测器间隔了一段距离，所以依照光学路径，不同距离的物体将会成像在CCD上不同的位置。按照三角公式进行计算，就能推导出被测物体的距离。



➤ 飞行时间（TOF）

TOF的相对于三角测量法原理更加简单。激光器发射一个激光脉冲，并由计时器记录下出射的时间，回返光经接收器接收，并由计时器记录下回返的时间。两个时间相减即得到了光的“飞行时间”，而光速是一定的，因此在已知速度和时间后很容易就可以计算出距离。



➤ 对比

(1) 测距距离

从测量原理上来说，TOF的测距距离更远一些，三角测距雷达测距距离相对较近是由于他本身原理上的限制，通过三角测距的图示，我们可以看出如果三角测距雷达测量的物体距离越远的话，那他返回到CCD上的位置差别也就越小，如果超过最远的限制，那么CCD就几乎无法分辨。而且三角测距雷达的信噪比没有TOF雷达的高，由于TOF雷达采用脉冲激光的方式，可以严格控制视野中外界环境的影响

(2) 采样率

在转速一定的情况下，采样率决定了每一帧图像的点云数目以及角分辨。如果角分辨率越高，点云数量就会越多，那么对周围环境的精度也就越高。在市场上的三角测距雷达的采样率一般都在20kHz一下，而TOF雷达会更高。分析其原因是由于三角测距雷达的运算量更大导致的采样率较低。

(3) 精度

由于测距原理的不同，三角测距法在近距离下的测量精度会很高，但是距离越远，测量精度就会逐渐下降，由于距离的增加，角度差异就会越来越小导致的测量精度下降。而TOF测距依赖的是飞行时间，时间精度并不会随着距离的变化而变化。

通过对比，三角测距雷达对于中近距离的测量精度较高，但是对于远距离测量精度较差，容易受到干扰，然而价格相对便宜，一般可以满足室内使用的需求；而飞行时间法的激光雷达传感器测距范围广，精度高，抗干扰能力强，但是价格昂贵，在室内外都可以使用。

4.2.2 运动畸变去除

由于我们的激光雷达传感器的频率有限，如果搭载激光雷达车辆移动，而且激光点的数据不能实时获得，就可能产生畸变。例如我们的激光雷达以 $90^\circ/\text{s}$ 的速度旋转，如果 5Hz 的雷达，那么每 200ms 可以获得1帧，每帧数据相差 18° ，由于我们认为每帧数据再同一时刻获得的，但实际情况是激光雷达一直在运动，这样就导致每一个激光点都有不同的基准位置，这样就会产生运动畸变。

➤ 纯估计方法——VICP

有速度估计的ICP，在ICP匹配中，我们没考虑到激光的运动畸变和当前激光数据是有错误的，而作为ICP的变种，VICP在考虑到了机器人运动的同时也匹配了估计机器人的速度。在VICP中，我们认为他在每一帧之间是匀速运动的，我们设 X^i, X^{i-1} 表示的第 i 帧和第 $i-1$ 帧的数据， T_i, T_{i-1} 表示对应的位置矩阵，则我们的到机器人的速度是：

$$V_i = \frac{1}{\Delta t} T_{i-1}^{-1} T_i。$$

设第 i 帧的时刻表：

$$(t_i - n\Delta t, t_i - (n-1)\Delta t, \dots, t_i - \Delta t, t_i)$$

则第 i 个节点的位置矩阵为：

$$T(t_i - (n-j)\Delta t) = T_i e^{(n-j)\Delta t(-V_i)}$$

最终可以得到矫正过程为：

$$\bar{X}^i = \{e^{(n-j)\Delta t(-V_i)} X_j | j = 0, \dots, n\}$$

➤ 里程计辅助方法

如果激光雷达的帧率过低，那么匀速运动的假设就不成立，不能真实的反正机器人的情况，数据预处理和状态估计过程耦合。所以我们需要传感器进行辅助（odom/IMU），这样我们就可以有极高的更新频率（ 200Hz ），也可以比较准确的反应运动情况，有较高精度的局部位置估计，这样我们就可以实现状态估计与预处理解耦。

(1) 惯性测量单元 (IMU)

惯性测量单元可以直接测量角速度和线加速度，可以到的较高的角速度和测量精度，测量频率较高，可以达到1kHz以上，但是线加速度精度太差，在二次积分的局部精度也很差。

(2) 轮式里程计 (odom)

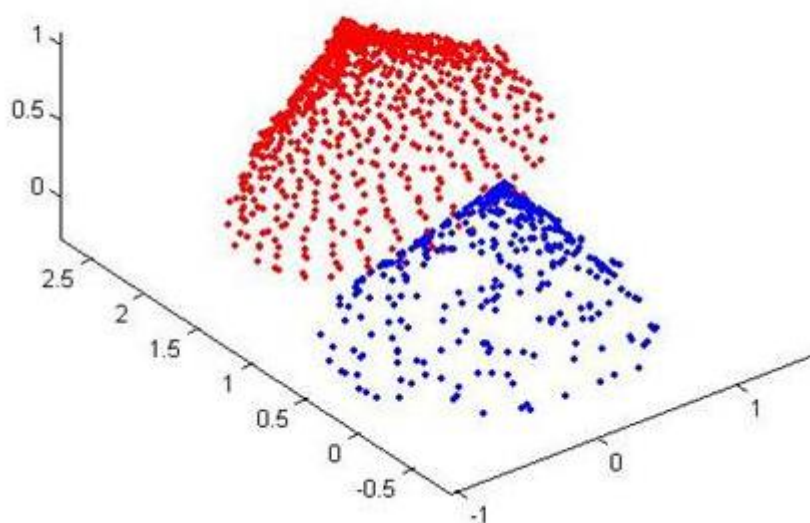
轮式里程计可以直接测量机器人的位移和角度，具有较高的局部角度测量精度和局部位置测量精度，更新速度虽然没有IMU高，但也能达到200Hz，可以满足我们更新频率的需求。

4.3 激光SLAM前端配准方法

前端配准的任务就是将两个或以上坐标系中的点云数据转换到统一坐标系中的数学计算过程。他的实质就是空间坐标变换。空间坐标的变换可以由三类参数确定：尺度、旋转和平移。

4.3.1 ICP匹配方法 (Point-Point)

ICP算法能够使不同的坐标下的点云数据合并到同一个坐标系中，首先是找到一个可用的变换，配准操作实际是要找到从坐标系1到坐标系2的一个刚性变换。



ICP是在点云中为了将的数据点进行匹配，我们给出两个点云集合，红色点云为 Q_i ，蓝色点云为 P_i

$$\begin{cases} Q_i = \{x_1, x_2, \dots, x_{N_x}\} \\ P_i = \{p_1, p_2, \dots, p_{N_x}\} \end{cases}$$

假如要将 Q_i 和 P_i 进行匹配，那么，我们需要找到一个旋转矩阵 R 和平移矩阵 T ，使得两个点集的对齐匹配转换，是下列目标函数最小。

$$E(R, T) = \sum_{i=1}^{N_p} \|Q_i - (RP_i + T)\|^2 = \min$$

其中 R 和 T 就是找到的待配准点云数据与参考点云数据之间的旋转参数和平移参数,使得两点集数据之间满足某种度量准则下的最优匹配。换句话说匹配的根本是选取合适的 R 和 T 矩阵，使得他们之间所有的点的欧几里得距离之和小于给定的阈值，采用最小二乘的方法可以取得最终的 R 和 T 。并将蓝色点云通过拼接旋转到红色点云附近的位置上，达到匹配的目的。

ICP配准过程：

- 计算点云 P 中的每一个点在点云 Q 中的对应最近点；
- 求得使上述对应点对平均距离最小的刚体变换，求得平移参数和旋转参数；
- 对 P 使用上一步求得的平移和旋转参数，得到变换后的点云 P' ；
- 如果新的变换后的点云与参考点集满足以上目标函数需求，就是两个点云的平均距离小于给定的阈值，那么就停止迭代计算，否则新的变换点集 P' 作为新的 P 继续进行迭代，最终达到目标函数的要求。

4.3.2 PL-ICP匹配方法（Point-Line）

与普通的ICP相比，PL-ICP是连续的。由于激光雷达的扫描出的数据是离散的，我们不能保证两次在不同位置上扫描同一地点的扫描点全部落在同一个物理点上，这就是点对点的距离产生的误差，而PL-ICP中则使用点到线的距离作为误差。由于PL-ICP是点到线的误差，所以PL-ICP为二阶收敛，比PP-ICP一阶收敛的速度更快，精度也比PP-ICP更加精准。

在PL-ICP中通过点与线进行匹配，将当前帧的数据根据初始位置投影到参考帧的坐标下，对当前帧的点 i ，在参考帧中寻找到最近的两个点 (p_1, p_2) ；再对误差进行计算，去除误差过大的点；最后的除最小化误差的函数。

现将第一次采集到的数据进行拟合，将离散的数据转变成连续的，再将洗一次需要匹配的点向连续的线上进行匹配，最后得到点到线的距离和整体的旋转角度。

4.3.3 基于优化的匹配方法

如果把要优化的目标函数看成山谷的话，可以把要优化的参数看成滚下山的石头，参数随机化为一个随机数可以看做在山谷的某个位置以0速度开始往下滚。目标函数的梯度可以看做给石头施加的力。

在给定一个目标函数时，可以把激光的帧间匹配问题转换为求解目标函数的极值问题：

$$E(T) = \arg \min_T \sum [1 - M(S_i(T))]^2$$

$$T = (T_x, T_y, T_\theta)$$

$S_i(T)$ 表示把激光数据用位姿 T 进行转换；

$M(x)$ 表示得到坐标 x 的地图占用概率。

4.4 后端优化

前端配准智能给出一个短时间内的轨迹和地图，但由于不可避免的误差累积，导致地图在长时间内并不是非常准确的。所以在前端配准的基础上，我们再构建一个尺度、规模更大的优化问题，用以考虑长时间的最优轨迹和地图。这就是后端优化。

4.4.1 基于滤波器的激光SLAM方法

机器人SLAM的过程是：先控制机器人到达一个位姿，然后再进行观测。但是问题出现在了我们是控制数据和观测数据来求机器人位姿，这就出现了一个逆向的过程即：我们是先有位姿再得到观测数据却变成了先得到观测数据再来求位姿的过程。通俗来讲就是我们需

要由结果（观测数据）来反推原因（位姿）；因此这里就引入了贝叶斯公式，贝叶斯公式的存在就是为了求解这种由结果反推原因的问题。

4.4.2 贝叶斯滤波

贝叶斯滤波估计的是概率分布，不是具体的数值，是一大类方法的统称，它是一个抽象的表达形式—对于不同问题有不同的实现方式(卡尔曼家族、粒子滤波)，迭代估计形式。贝叶斯滤波主要分为预测和修正（更正）两个过程。

首先我们先来看一下基本数学概念

贝叶斯公式：

$$p(x,y) = p(x|y)p(y) = p(y|x)p(x)$$

$$\rightarrow p(x|y) = \frac{p(y|x)p(x)}{p(y)} = \eta p(y|x)p(x)$$

条件贝叶斯贝叶斯公式：

$$p(x|y,z) = \frac{p(y|x,z)p(x|z)}{p(y|z)} = \eta p(y|x,z)p(x|z)$$

当我们给定t时刻以及之前的所有观测z和输入u，我们的目标是求得当前状态量x的概率分布（belief），即

$$bel(x_t) = p(x_t|z_{1:t}, u_{1:t})$$

首先我们需要求解t时刻的观测前的先验分别：

$$\overline{bel}(x_t) = p(x_t|z_{1:t-1}, u_{1:t})$$

之后，我们再根据t时刻的观测通过贝叶斯公式的先验分布，求得后验分布

$$\overline{bel}(x_t) = \eta p(z_t|x_t, z_{1:t-1}, u_{1:t}) \overline{bel}(x_t)$$

η 为归一化系数。

那么，t时刻的先验分布就可以根据全概率公式用t-1时刻的后验分布表示：

$$\begin{aligned}\overline{bel}(x_t) &= p(x_t|z_{1:t-1}, u_{1:t}) \\ &= \int p(x_t, x_{t-1}|z_{1:t-1}, u_{1:t}) dx_{t-1} \\ &= \int p(x_t|x_{t-1}, z_{1:t-1}, u_{1:t}) p(x_{t-1}|z_{1:t-1}, u_{1:t}) dx_{t-1}\end{aligned}$$

假设t-1时刻的状态与t时刻的输入无关（马尔科夫性），那么

$$p(x_t, x_{t-1}|z_{1:t-1}, u_{1:t}) = p(x_{t-1}|z_{1:t-1}, u_{1:t}) = \overline{bel}(x_{t-1})$$

则我们可以得到

$$\overline{bel}(x_t) = \int p(x_t|x_{t-1}, z_{1:t-1}, u_{1:t}) \overline{bel}(x_{t-1}) dx_{t-1}$$

进一步假设马尔科夫性，即t时刻的状态只和t时刻的输入以及t-1时刻的状态有关，t时刻的观测只和t时刻的状态有关，可以得到：

$$p(x_t|x_{t-1}, z_{1:t-1}, u_{1:t}) = p(x_t|x_{t-1}, u_t)$$

$$p(x_t|x_{t-1}, z_{1:t-1}, u_{1:t}) = p(z_t|x_t)$$

最终我们可以得到贝叶斯滤波器的更新方程：

$$\overline{bel}(x_t) = \int p(x_t|x_{t-1}, u_t) \overline{bel}(x_{t-1}) dx_{t-1}$$

$$bel(x_t) = \eta p(z_t|x_t) \overline{bel}(x_t)$$

一个完整的贝叶斯滤波器就是随着时间推移不断执行以上两步来完成状态量的估计。其中第一个方程称为prediction（预测）， $p(x_t|x_{t-1}, u_t)$ 称为状态转移概率。第二个方程称为correction（修正）， $p(z_t|x_t)$ 称为观测概率或测量概率。

由以上推导可知，贝叶斯滤波器对系统状态的更新进行了马尔科夫假设，两个更新方程描述了一个隐马尔科夫模型（因为状态量无法直接观测到，所以为‘隐’），这也是贝叶斯滤波器与基于图的优化器的区别，基于图的优化器没有做马尔科夫假设。

4.4.3 粒子滤波

粒子滤波通过非参数化的蒙特卡洛(Monte Carlo)模拟方法来实现递推贝叶斯滤波，适用于任何能用状态空间模型描述的非线性系统，精度可以逼近最优估计。粒子滤波器具有简单、易于实现等特点，它为分析非线性动态系统提供了一种有效的解决方法，从而引起目标跟踪、信号处理以及自动控制等领域的广泛关注。简单来说，粒子滤波法是指通过寻找一组在状态空间传播的随机样本对概率密度函数进行近似，以样本均值代替积分运算，从而获得状态最小方差分布的过程。

如果我们用 X 表示机器人的状态，则

$$X = \{(x_t^i, w_t^i) | i = 1, \dots, n\}$$

其中：

x_t^i 表示一个状态的假设 —— 机器人位姿

w_t^i 表示假设的权重 —— 跟地图的匹配度

这样我们的可以进一步得到

用粒子进行状态传播，这里表示的是每个粒子单独传播的函数

$$x_t^i \sim p(x_t | u_t, x_{t-1}^i)$$

评估每一个粒子的权重，也就是地图匹配程度（观测和矫正）：

$$w_t^i = \eta p(z_t, x_t^i)$$

根据权重进行重采样以 w_t^i 的概率接受 x_t^i ，也可理解为粒子的重新分布，通过这种方式去除低权重粒子。

如果我们的传播模型为：

$$\overline{bel}(x_t) = \int p(x_t|x_{t-1},u_t)p(x_{t-1}|z_{1:t-1},u_{1:t-1})dx_{t-1}$$

那么t-1时刻的得到一个的概率分布（粒子分布）：

$$p(x_{t-1}|z_{1:t-1},u_{1:t-1}) = \{(x_{t-1}^i, w_{t-1}^i) | i = 1, \dots, n\}$$

根据 u_t 预测t-1时刻的概率分布（具体位置）：

$$x_t^i \sim p(x_t|u_t, x_{t-1}^i) \quad i = 1, \dots, n$$

这里我们引入机器人的运动模型：

$$\begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} dx + \varepsilon_x \\ dy + \varepsilon_y \\ d\theta + \varepsilon_\theta \end{bmatrix}$$

如果我们设t-1时刻第i个粒子的位姿是：

$$x_{t-1}^i = (x_{t-1}, y_{t-1}, \theta_{t-1})$$

设t-1时刻的 $u_t = (\Delta x, \Delta y, \Delta \theta)$,

设t时刻第i个粒子的位姿 $x_t^i = (x_t, y_t, \theta_t)$

噪声均值为0的高斯分布，方差分别为 N_x, N_y, N_z

那么我们可以推导出：

$$\begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{bmatrix} + \begin{bmatrix} \cos \theta_{t-1} & -\sin \theta_{t-1} & 0 \\ \sin \theta_{t-1} & \cos \theta_{t-1} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta x + N_x \\ \Delta y + N_y \\ \Delta \theta + N_\theta \end{bmatrix}$$

由于我们无法知道机器人位姿的真是状态和位置，所以我们需要引入权重来评估实际人的预测分布和实际分布的差，差越大，权重越小。那么我们对用机器人预测分布进行采样，联合权重一起近似机器人的后验概率分布：

$$w = \frac{bel(x_t)}{\overline{bel}(x_t)}$$

对于某一个例子：

$$bel(x_t) = \eta p(z_t|x_t)p(x_t|x_{t-1},u_t)bel(x_{t-1})$$

$$\overline{bel}(x_t) = p(x_t|x_{t-1},u_t)bel(x_{t-1})$$

那么我们的权重为：

$$w = \frac{\eta p(z_t|x_t)p(x_t|x_{t-1},u_t)bel(x_{t-1})}{p(x_t|x_{t-1},u_t)bel(x_{t-1})}$$

$$w = \eta p(z_t|x_t)$$

这就是我们的观测模型。

如果使用不重采样的更新方式：

$$w_i = w_{i-1}p(z_t|x_t)$$

虽然粒子滤波算法可以作为解决SLAM问题的有效手段，但是该算法仍然存在着一些问题。其中最主要的问题是需要用大量的样本数量才能很好地近似系统的后验概率密度。机器人面临的环境越复杂，描述后验概率分布所需要的样本数量就越多，算法的复杂度就越高。因此，能够有效地减少样本数量的自适应采样策略是该算法的重点。另外，重采样阶段会造成样本有效性和多样性的损失，导致样本贫化现象。如何保持粒子的有效性和多样性，克服样本贫化，也是该算法研究重点。

4.4.4 基于图优化的激光SLAM方法

基于图优化的SLAM出现解决了一致性建图的问题。如果说滤波类SLAM是属于序贯估计的话，基于图优化则是属于批处理。

1、Graph-based SLAM

Graph-based SLAM需要我们找到一个图，用途中的节点表示机器人的位置，而两个节点的边表示两个位子的空间约束，并且找到一个最优的配置，调整机器人的位姿，使得里程计的预测值和里程计的观测值最小的方法。这样，我们就需要通过非线性最小二乘进行求解

2、非线性最小二乘

首先，我们需要考虑一个最简单的最小二乘问题：

$$\min_x \frac{1}{2} \|f(x)\|_2^2$$

这里的自变量是 $x \in R^n$ ， f 是非线性函数，如果我们设它有 m 维： $f(x) \in R^m$ 。则下面将讨论如何求解这个优化问题。

由于 f 导函数是一个复杂的非线性方程，并不能简单顺利的求解，那么我就可以用迭代的方式，从一个初始值出发，不断地更新当前的优化变量，是目标函数下降，具体步骤：

- (1) 给定摸个初始值 x_0
- (2) 对第 k 次迭代，寻找一个增量 Δx_k ，使得 $\|f(x_k + \Delta x_k)\|_2^2$ 达到极小值
- (3) 若 Δx_k 足够小，则停止
- (4) 否则，令 $x_{k+1} = x_k + \Delta x_k$ ，返回第二步

这样我们就让一个求解导函数的问题变成了一个不断寻找梯度下降的过程，直到某个时刻函数无法下降，算法收敛，目标达到了一个极小值。我们接下来就需要解决增量 Δx_k 如何确定。

➤ 高斯牛顿法

高斯牛顿法是最优换算法中最简单的方法之一，它的思路是将 $f(x)$ 进行一阶泰勒展开：

$$f(x + \Delta x) \approx f(x) + J(x)\Delta x$$

这里 $J(x)$ 为 $f(x)$ 关于 x 的导数，实际上是一个 $m \times n$ 的矩阵，或是雅克比矩阵。因为我们需要找到 Δx ，使得 $\|f(x_k + \Delta x_k)\|^2$ 达到最小。为了得到 Δx ，我们需要解一个线性最小二乘的问题：

$$\Delta x^* = \arg \min_{\Delta x} \frac{1}{2} \|f(x) + J(x)\Delta x\|^2$$

根据上述条件，我们对目标函数 Δx 求导，并令其导数为零，可以得到：

$$J(x)^T J(x) \Delta x = -J(x)^T f(x)$$

由于我们需要求解的是 Δx ，因此这是一个线性方程，我们也可以称其为增量方程或是高斯牛顿方程。如果我们把左边的系数定义为 H ，右边定义为 g ，那么上式变为：

$$H \Delta x = g$$

求解增量方程的核心是整个优化问题的核心所在。那么我们高斯牛顿法的步骤可以写成：

- (1) 给定摸个初始值 x_0
- (2) 对第 k 次迭代，求解出当前的雅克比矩阵 $J(x_k)$ 和误差 $f(x_k)$
- (3) 求解增量方程： $H \Delta x_k = g$
- (4) 如果 Δx_k 足够小，则停止；否则 $x_{k+1} = x_k + \Delta x_k$ ，返回第二步

➤ 列文伯格—马夸尔特法

由于高斯牛顿法采用的是近似二阶泰勒展开，只能在展开点附近有较好的近似效果，所以我们很自然地想到应该给 Δx 添加一个信赖区域，如何确定这个信赖区域呢，一个比较好的方法就是根据我们的近似模型个实际函数之间的差异来确定，在此我们考虑使用：

$$\rho = \frac{f(x + \Delta x) - f(x)}{J(x)\Delta x}$$

使用这个函数来判断泰勒近似是否够好。如果 ρ 接近1，则近似是好的；如果 ρ 太小，说明减小的值远少于近似减小的值，认为近似比较差。那么在列文伯格—马夸尔特法中，我们需要求解：

$$\min_{\Delta x_k} \frac{1}{2} \|f(x_k) + J(x_k)\Delta x_k\|^2, s.t. \|D\Delta x_k\|^2 \leq \mu$$

μ 是信赖区域的半径，我们通过这样一个子问题来获取梯度。这个问题是带不等式约束的优化问题，我们用拉格朗日乘子将它转化为一个无约束优化的问题：

$$\min_{\Delta x_k} \frac{1}{2} \|f(x_k) + J(x_k)\Delta x_k\|^2 + \frac{\lambda}{2} \|D\Delta x\|^2$$

这里的 λ 为拉格朗日乘子。类似于高斯牛顿的做法，把它展开后，我们发现该问题的核心仍是计算增量的线性方程：

$$(H + \lambda D^T D)\Delta x = g$$

此增量方程与高斯牛顿法相比，多了一项 $\lambda D^T D$ 。如果考虑到简化形式 $D = I$ ，那么相当于求解：

$$(H + \lambda I)\Delta x = g$$

通过公式，我们可以看到当参数 λ 比较小的时候， H 占主导地位，这说明二次近似模型在该范围内是比较好的，这时列文伯格—马夸尔特法更接近高斯牛顿法。反而如果 λ 比较大时 λI 占主导地位，这时列文伯格—马夸尔特法更接近一阶梯度下降法，这说明附近的二次近似不够好。列文伯格—马夸尔特法的求解方式，可以在一定程度上避免线性方程组的系数矩阵的非奇异性和兵团问题，提供更加稳定、准确的增量 Δx 。

在实际情况下，还有许多其他方法来求解函数的增量。但无论是高斯牛顿法还是列文伯格—马夸尔特法，在做最优化计算时，都需要提供变量的初始化。非线性优化的所有迭代求解方案，都需要用户来提供一个良好的初始值，用来计算非线性优化程序。

4.5 回环检测

我们已经介绍了前端和后端：前端提供特征点的提取和轨迹、地图的初值，而后端负责对所有这些数据进行优化。然后，如果像VO那样仅考虑相邻时间上的关联，那么之前产生的误差将不可避免地累积到了下一个时刻，使得整个SLAM会出现累积误差，长期估计的结果将不可靠，或者说，我们无法构建全局一直的轨迹和地图。

所以回环检测对于SLAM系统的意义重大。它关系到我们估计的轨迹和地图在长时间的正确性。另一方面，由于回环检测提供了当前数据与所有历史数据的关联，在跟踪算法糗事

之后，我们还可以利用回环检测进行重定位。因此，回环检测对整个SLAM系统精度与稳健性的提升是非常明显的。甚至在某些时候，我们把仅有前端和局部后端的系统称为VO，而把带有回环检测和全局后端的系统称为SLAM。

回环的一种简单的方法就是对任意两帧数据都做一遍特征匹配，但是缺点是我们不能盲目的假设任意两个图像都可能存在回环，使得检测的数量特别大，这种情况在大多数系统上是不实用的。而另一种是随机抽取历史数据进行回环检测，这种方法能够位置常熟时间的运算量，但是这种盲目试探的方法在帧数N增长时，抽到回环的几率又会大幅下降，使得检测效率不高。

为了解决盲目检测，大体分为两种思路：一种是基于里程计的几何关系，另一种是基于外观。几何关系是指当我们发现机器人运动到了之前的某个位置附近时，检测他们有没有回环关系，但是由于当前由于累积误差的存在，我们往往没法正确的发现运动到了某个位置，所以回环也就无从谈起。而基于外观的方法，它和前端、后端的估计都没有关系，仅根据两幅图像的相似度，确定回环检测的关系。这种方法摆脱了累积误差，使得回环检测模块成为了一个相对独立的模块。这种基于外观的回环检测方式能够有效地在不同场景下工作，逐渐成为了视觉SLAM的主流做法，并被应用于实际的系统中去。

Loop Closing步骤

- 1. 检测到回环的发生
- 2. 计算回环候选帧与当前帧的运动
- 3. 验证回环是否成立
- 4. 闭环

回环检测的结构分类

算法\事实	是回环	不是回环
是回环	真阳性（Ture Positive）	假阳性（False Positive）
不是回环	假阴性（False Negative）	真阴性（Ture Negative）

4.6 基于定位的地图构建

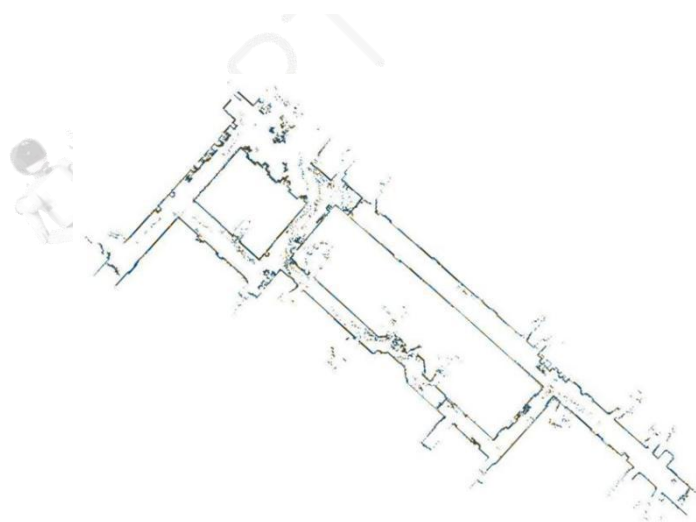
地图构建是SALM的两大目标之一（SLAM被称为同时定位与建图）。在经典的SLAM模型中，我们所谓的地图就是所有路标点的集合。一旦确定了路边店的位置，我们就可以说是完成了建图。但是由于人们对建图的需求不同，需要建立的地图也各不相同

对于人来说，我们可以看懂一张地图，但对于机器人应该给机器人一个易于理解和易于计算的数字文件。目前相关从业人士对于机器人导航地图所需信息已经研讨了很长时间，到现在为止也没有明确的论断。在此环境下，近年来出现了各种形式的机器人导航地图信息格式，有些不仅包括二维信息，而且还包括三维信息，甚至有些地图不仅包含有关移动的信息，还包含各物体的分割（segmentation）的信息。

4.6.1 地图分类

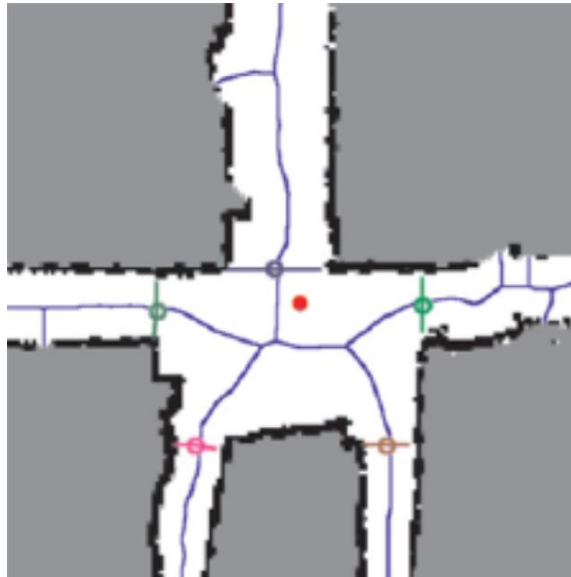
➤ 尺度地图

尺度地图中的距离和现实世界是相对应的，栅格地图、特征地图、点云地图都可以称为尺度地图，一般的定位都用到的是尺度地图，而且尺度地图是我们最常用到的地图



➤ 拓扑地图

拓扑地图只表述其中两点的连通关系，并不关心路径，在一些大场景下，我们需要构建一个拓扑地图。



➤ 混合地图

混合地图是在一张拓扑地图上,每一个点上添加一张尺度地图,在我们需要导航的时候,先对拓扑地图进行导航,再对其中的尺度地图进行导航。

本书中,我们采用ROS中常用的二维占用网格地图 (OGM, Occupancy Grid Map)。如下图所示,白色是机器人可以移动的自由区域 (free area), 黑色是机器人不能移动的占用区域 (occupied area), 灰色是未被确认的未知区域 (unknown area)。

该区域用从0到255表达的灰度 (gray scale) 值表示。该值是通过贝叶斯定理的后验概率获得的,该概率代表占用状态的占用概率: $occ = (255 - color_avg) / 255.0$ 。如果图像是24位,则是“ $color_avg = (\text{一个单元的灰度值} / 0xFFFFFFFF \times 255)$ ”。这个occ越接近1,它被占用的概率越大,越接近0,被占用的概率就越小。

当它以ROS消息发布时,会被重新定义为占有度,是[0~100]之间的整数。越接近“0”就越接近移动自由的区域 (free area), 而越接近“100”就越是不可移动的已占用的区域 (occupied area)。此外,“-1”是定义为未知区域 (unknown area)。

在ROS中,地图信息以*.pgm文件格式 (portable graymap format) 存储和使用相当于一张image图片。此外它还包含一个*.yaml文件,它也包含地图数学信息。例如,我们查看上一节所构建的地图信息 (map.yaml), 则结果类似如下所示。image是地图的文件名,而resolution是地图的分辨率,单位是meters/pixel。


```

image: mymap.pgm
resolution: 0.050000
origin: [-12.200000, -13.800000, 0.000000]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196

```

也就是说，每个像素意味着5厘米。origin是地图的原点，origin的每个数字代表x、y和yaw。地图的左下角是 $x = -10m$ ， $y = -10m$ 。negate会反转黑白。每个像素的颜色如下确定：由当占用概率超过占用阈值（occupied_thresh）时表示为黑色的占用区域，而当占用概率小于自由阈值（free_thresh）时表示为白色的自由区域（free area）。

4.6.2 覆盖栅格建图算法



栅格地图简单来说就是把整个环境分成一个个小栅格，每个栅格有两种状态，一种是有障碍物，另一种是空闲，由于设置的栅格大小不同，所以可能导致建立出的栅格地图和实际的地图有稍许差异。但由于是非参数模型，随着地图的增大，对内存的需求会急剧增大，也正是由于非参数模型导致其可以天然区分可通行的区域，非常适合进行轨迹规划。

如果我们给定及传感器的位姿和传感器的观测数据：

$$data = \{x_1, z_1, x_2, z_2, \dots, x_n, z_n\}$$

估计出最可能的地图：

$$m^* = \arg \max_m P(m|data)$$

$$\rightarrow m^* = \arg \max_m P(m|x_{1:t}, z_{1:t})$$

如果我们设占用为 $p(m_i) = 1$ ，空闲为 $p(m_i) = 0$ ，未知状态为 $p(m_i) = 0.5$ 。由于我们地图中的每一个栅格都是独立存在的，因此数学表达式可以表示为：

$$P(m) = \prod p(m_i)$$

而对地图估计问题就可以表示为：

$$P(m|x_{1:t}, z_{1:t}) = \prod p(m_i|x_{1:t}, z_{1:t})$$

因此, 如果我们需要估计整体的环境题图只需要对每一个独立栅格进行单独估计就可以得到。

如果 m_i 是一个二元随机变量, 因此:

$$p(m_i|x_{1:t}, z_{1:t}) = \frac{p(z_t|m_i, z_{1:t-1}, x_{1:t})p(m_i|z_{1:t-1}, x_{1:t})}{p(z_t|z_{1:t-1}, x_{1:t})}$$

$$p(m_i|x_{1:t}, z_{1:t}) = \frac{p(z_t|m_i, x_t)p(m_i|z_{1:t-1}, x_{1:t-1})}{p(z_t|z_{1:t-1}, x_{1:t})}$$

其中:

$$p(z_t|m_i, x_t) = \frac{p(m_i|z_t, x_t)p(z_t|x_t)}{p(m_i|x_t)}$$

由于 m_i 是一个二元随机变量, 得到:

$$p(m_i|x_{1:t}, z_{1:t}) = \frac{p(m_i|z_t, x_t)p(z_t|x_t)}{p(m_i|x_t)} \frac{p(m_i|z_{1:t-1}, x_{1:t-1})}{p(z_t|z_{1:t-1}, x_{1:t})}$$

$$p(m_i|x_{1:t}, z_{1:t}) = \frac{p(m_i|z_t, x_t)p(z_t|x_t)}{p(m_i)} \frac{p(m_i|z_{1:t-1}, x_{1:t-1})}{p(z_t|z_{1:t-1}, x_{1:t})}$$

同理, 对于 $\neg m_i$, 可得

$$p(\neg m_i|x_{1:t}, z_{1:t}) = \frac{p(\neg m_i|z_t, x_t)p(z_t|x_t)}{p(\neg m_i)} \frac{p(\neg m_i|z_{1:t-1}, x_{1:t-1})}{p(z_t|z_{1:t-1}, x_{1:t})}$$

两者相比:

$$\frac{p(m_i|x_{1:t}, z_{1:t})}{p(\neg m_i|x_{1:t}, z_{1:t})} = \frac{p(m_i|z_t, x_t)}{p(\neg m_i|z_t, x_t)} \frac{p(m_i|z_{1:t-1}, x_{1:t-1})}{p(\neg m_i|z_{1:t-1}, x_{1:t-1})} \frac{p(\neg m_i)}{p(m_i)}$$

$$\frac{p(m_i|x_{1:t}, z_{1:t})}{p(\neg m_i|x_{1:t}, z_{1:t})} = \frac{p(m_i|z_t, x_t)}{p(\neg m_i|z_t, x_t)} \frac{p(m_i|z_{1:t-1}, x_{1:t-1})}{p(\neg m_i|z_{1:t-1}, x_{1:t-1})} \frac{p(\neg m_i)}{p(m_i)}$$

对于二元随机变量：

$$\frac{p(m_i|x_{1:t},z_{1:t})}{p(\neg m_i|x_{1:t},z_{1:t})} = \frac{p(m_i|z_t,x_t)}{1-p(m_i|z_t,x_t)} \frac{p(m_i|z_{1:t-1},x_{1:t-1})}{1-p(m_i|z_{1:t-1},x_{1:t-1})} \frac{1-p(\neg m_i)}{p(m_i)}$$

对于 $p(x)$ ，定义对应的Log-Odd项：

$$l(x) = \frac{p(x)}{1-p(x)}$$

则：

$$p(x) = 1 - \frac{1}{1 + \exp(l(x))}$$

这样就可以变成：

$$l(m_i|x_{1:t},z_{1:t}) = l(m_i|x_t,z_t) + l(m_i|z_{1:t-1},x_{1:t-1}) - l(m_i)$$

其中：

$l(m_i|x_t,z_t)$ 表示激光雷达的逆观测模型(inverse measurement Model)

$l(m_i|z_{1:t-1},x_{1:t-1})$ 表示栅格 m_i 在 $t-1$ 时刻的状态

$l(m_i)$ 表示栅格 m_i 的先验值，该值对所有栅格都相同

4.7 SALM_Gmapping功能包

slam_gmapping功能包包含了来自OpenSlam的ROS软件包GMapping。GMapping是一个根据激光传感器数据来构建二维栅格地图的Rao-Blackwellized粒子滤波SLAM算法。

Rao-Blackwellized粒子滤波目前被认为是解决SLAM问题行之有效的方法。该方法使用粒子滤波方法，其中每个粒子都是承载一份独立的地图，该软件包中采用自适应蒙特卡洛算法在损失较小精度的情况下，减少了重采样的次数和采样的粒子个数，提高算法的运行效率。

在运用Gmapping构建地图，也有着相应的约束条件：

4.7.1 约束条件

➤ 移动方式

机器人必须能够使用X、Y轴平面上的平移速度（linear velocity）和theta旋转速度（angular velocity）指令进行操作。比如有左右两个可以单独驱动的差动驱动式移动机器人（differential drive mobile robot），或者具有三个以上的全向轮的全向移动机器人（omni-wheel robot）。

➤ 里程计(Odometry)

需要能获得机器人的位置估计信息。换句话说，要可以通过导航推测方法（dead reckoning）来推算机器人移动的距离和旋转量。在第三章运动学分析章节知我们采用了编码器计数据和惯性导航数据融合的组合定位方法生成里程信息，使得里程信息更加准确。

➤ 传感器

为了实现SLAM和导航，机器人需要有LDS（Laser Distance Sensor）、LRF（Laser Range Finder）和LiDAR来测量XY平面上的障碍物。深度相机（如RealSense、Kinect和Xtion）也可以将3D信息转换为XY平面上的信息。换句话说，有必要安装一个能够测量二维平面的传感器。书采用激光雷达作为测量传感器。

4.7.2 SLAM执行过程

ROS封装了的OpenSlam GMapping，gmapping功能包提供了基于激光的SLAM，在ROS系统中使用slam_gmapping节点表示。通过该节点用户可以用机器人在移动过程中根据scan信息（由传感器测量的距离值）和tf值（传感器的位置值）来创建2D栅格地图。简单的说就是机器人在未知环境中从未知位置开始移动，在移动过程中依据位置估计和地图进行自身定位，在定位基础上同步增量建立地图，而实现自主定位和导航。

节点订阅主题

- tf (tf/tfMessage)，用于激光器坐标系，基座坐标系，里程计坐标系之间转换。

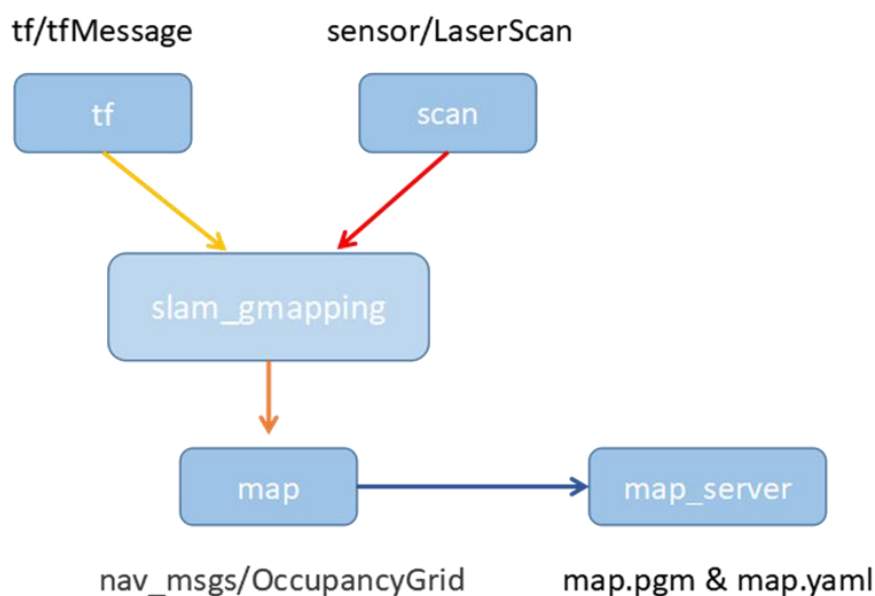
- scan (sensor_msgs/LaserScan)， 激光器扫描数据

节点发布主题

- map_metadata (nav_msgs/MapMetaData)， 周期性发布地图metadata数据
- map (nav_msgs/OccupancyGrid)， 周期性发布地图数据
- ~entropy (std_msgs/Float64)， 发布机器人姿态分布熵的估计

服务

- dynamic_map (nav_msgs/GetMap)， 调用该服务可以获取地图数据

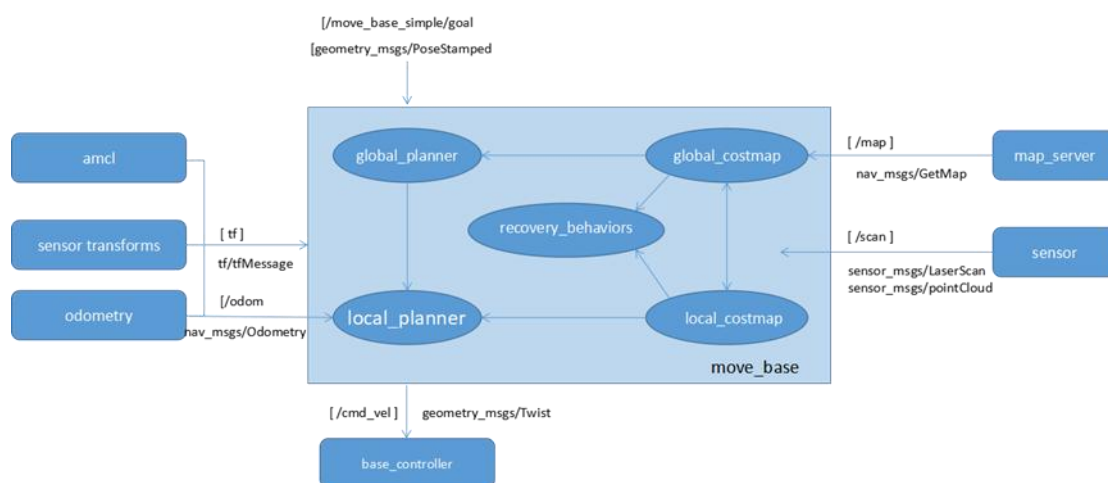


4.8 ROS机器人导航简介

机器人在未知环境中需要使用激光传感器（或者将深度传感器转换为激光数据），先进行地图建模，然后根据构建的地图进行导航与定位。在ROS中可以利用以下三个功能来实现自主导航。

- gmapping:根据激光数据（深度数据模拟激光数据）构建地图。
- move_base:将全局导航和局部导航链接在一起以完成其导航任务，全局导航用于建立到地图上最终目标或一个远距离目标的路径，局部导航用于建立到近距离目标和为了临时躲避障碍物的路径。

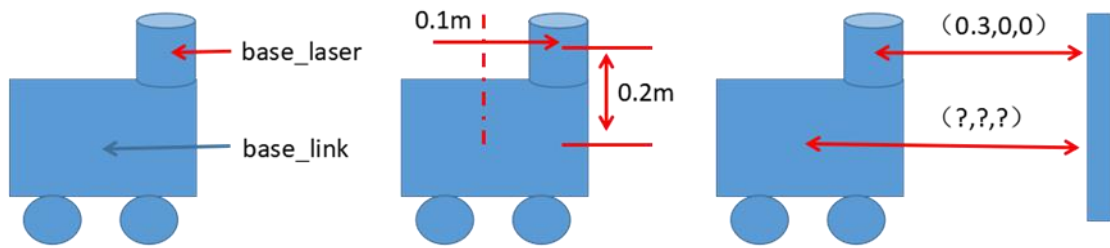
- amcl:英文全称是adaptive Monte Carlo localization，蒙特卡洛定位方法的一种升级版，使用自适应的KLD方法来更新粒子。而mcl（蒙特卡洛定位）法使用的是粒子滤波的方法来进行定位的，粒子滤波很粗浅的说就是一开始在地图空间很均匀的撒一把粒子，然后通过获取机器人的motion来移动粒子，比如机器人向前移动了一米，所有的粒子也就向前移动一米，不管现在这个粒子的位置对不对，使用每个粒子所处位置模拟一个传感器信息跟观察到的传感器信息（一般是激光）作对比，从而赋给每个粒子一个概率。之后根据生成的概率来重新生成粒子，概率越高的生成的概率越大。这样的迭代之后，所有的粒子会慢慢地收敛到一起，机器人的确切位置也就被推算出来了。



其中椭圆框内是导航必须用到的基本组件，圆角矩形框是供应用开发者需要的组件。导航功能包提供的主要组件功能如下：

（1）传感器坐标变换（sensor transforms）

变换树定义了不同坐标系之间的偏移。例如，他有可移动的基座和位于基座上方的激光雷达，那么对于这台机器人，一个坐标系原定位于机器人基座中心，另一个坐标系原点位于激光雷达传感器中心。将位于基座上的坐标系定义为base_link，位于激光雷达上的坐标系定义为base_laser。如下图所示：激光雷达到前方物体距离为0.3m，而机器人的中心是在基座的中心，所以我们就需要利用tf来实现两个坐标之间的关系变换。



如上图所得，激光雷达位于基座中心前方0.1m且上方0.2m位置。因此可以获取到base_link和base_laser的变换关系，base_link到base_laser坐标必须平移（x:0.1m,y:0.0m,z:0.2m）。由于机器人在控制时是以base_link为基准的，所以要根据两者的关系进行坐标转换。变换过程只需要将两者的位置关系告知tf,由tf软件包完成其余工作。

为了在上述例子中使用tf软件包进行坐标变换，需要创建2个节点，分别对应于base_link和base_laser坐标系并确定那一个是父节点，哪一个是子节点。需要注意的是，tf假设的所有变换都是由父节点到子节点的。我们假设base_link为父节点，因此base_link和base_laser之间的变换矩阵是（x:0.1m,y:0.0m,z:0.2m）,如下图所示，通过变换，将激光雷达数据转换到base_link坐标系下，机器人就可以利用此信息来实现避障了。

（2）通过ROS发布传感器数据信息

通过ROS发布传感器数据信息对导航功能包是十分重要的。如果导航功能包不能从传感器上接收信息，机器人就可能会撞到障碍物。可以给导航功能包提供信息的有激光、摄像头、红外、超声波等传感器，但是导航功能包只能接收sensor_msgs/LaserScan或者sensor_msgs/PointCloud格式的数据。

（3）通过ROS发布里程信息(odometry source)

导航功能包用tf软件包来确定机器人在世界坐标系中的位置和相对于静态地图的相关传感器信息，但是tf软件包不提供与机器人速度相关的任何信息，所以导航功能包要求里程计源程序发布一个变换和一个包含速度信息的nav_msgs/Odometry消息。

（4）基础控制器（base_controller）

ROS并不提供任何标准的基础控制器，机器人通过geometry_msgs/Twist类型的消息来和底层电子驱动通信来控制电机运转，这一类型也正是odometry所使用的。基础控制器必须订阅以cmd_vel为名称的主题，且生成正确的线速度和角速度命令来驱动机器人运转。

机器人路径规划

从上节导航总体框图可以看出，move_base提供了ROS导航的配置、运行和交互接口，它主要包括如下两个部分。

- 全局路径规划（global planner）:根据给定的目标位置进行总体路径规划。
- 局部实时规划（local planner）:根据所在位置附近的障碍物进行躲避规划。

4.8.1 全局路径规划

全局路径规划器使用了A*方法，A*算法是一种高效的路径搜索算法，作为Dijkstra算法的扩展，因其高效性而被广泛应用于寻路及图的遍历，如星际争霸等游戏中就大量使用。采用启发函数来估计地图上机器人当前的位置到目标位置之间的距离，并以此选择最优的方向进行搜索，如果失败会选择其他路径继续搜索直到得到最优路径。

在启发中，估价是用估价函数表示：

$$f(n) = h(n) + g(n)$$

$f(n)$ 表示是每个可能试探点的估值；

$g(n)$ 表示从起始搜索点到当前点的代价；

$h(n)$ 表示启发式搜索中最为重要的一部分，即当前结点到目标结点的估值， $h(n)$ 设计的好坏，直接影响着具有此种启发式函数的启发式算法的是否能称为A*算法。对于 $h(n)$ 的选取是保证找到最短路径（最优解的）条件，关键在于估价函数 $f(n)$ 的选取（或者说 $h(n)$ 的选取）。我们以 $d(n)$ 表达状态n到目标状态的距离，那么 $h(n)$ 的选取大致有如下三种情况：

- 如果 $h(n) < d(n)$ 到目标状态的实际距离，这种情况下，搜索的点数多，搜索范围大，效率低。但能得到最优解。

- 如果 $h(n) = d(n)$ ，即距离估计 $h(n)$ 等于最短距离，那么搜索将严格沿着最短路径进行，此时的搜索效率是最高的。
- 如果 $h(n) > d(n)$ ，搜索的点数少，搜索范围小，效率高，但不能保证得到最优解。

一种具有 $f(n) = h(n) + g(n)$ 策略的启发式算法能成为 A^* 算法的充分条件是：

- 搜索树上存在着从起始点到终了点的最优路径。
- 问题域是有限的。
- 所有结点的子结点的搜索代价值 >0 。
- $h(n) \leq h^*(n)$ （ $h^*(n)$ 为实际问题的代价值）。

当此四个条件都满足时，一个具有 $f(n) = h(n) + g(n)$ 策略的启发式算法能成为 A^* 算法，并一定能找到最优解。

4.8.2 局部实时路径规划

局部路径实时规划是使用 DWA (Dynamic Window approaches, 规划推理和动态窗口) 算法，计算机器人每个周期内应该行驶的速度和角度 $(dx, dy, d\theta)$ 。DWA算法中先离散采样机器人控制空间 $(dx, dy, d\theta)$ ，再对于每个采样速度，从机器人当前的状态，进行模拟预测，如果采样速度应用于一段时间内，将会出现什么情况，然后用合并一些特征的度量标准来，从模拟预测中，评价每个轨迹结果，如障碍物接近目标，接近全局路径和速度。舍弃不合适的路径（有障碍物碰撞的）最后挑选得分最高的轨迹，并且发布相关的速度给移动平台。

4.8.3 Costmap

机器人的位置是根据从编码器和惯性传感器（IMU传感器）获得的测位来估计的。然后，通过安装在机器人上的距离传感器来计算机器人与障碍物之间的距离。导航系统将机器人位置、传感器姿态、障碍物信息和作为SLAM地图的结果而获得的占用网格地图调用到固定地图（static map），用作占用区域（occupied area）、自由区域（free area）和未知区域（unknown area）。

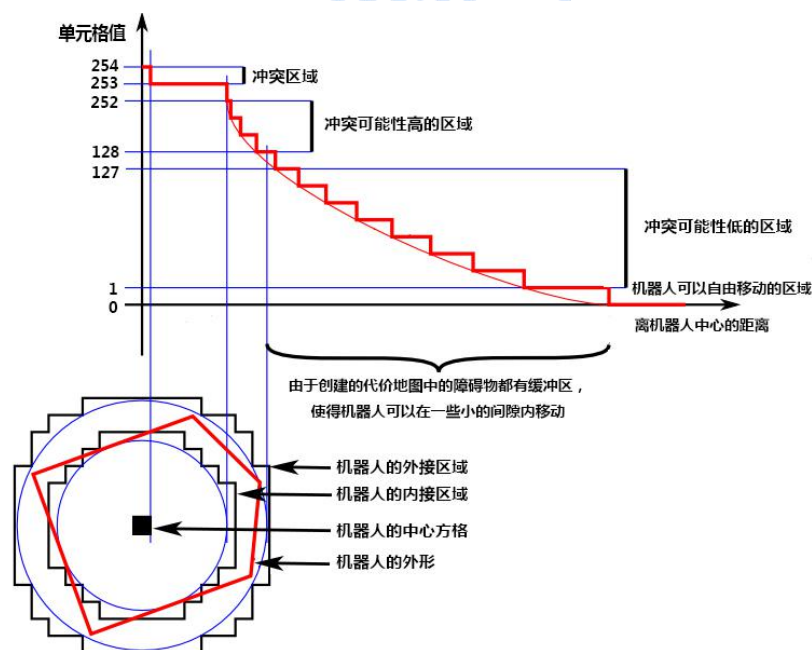
在导航中，基于上述四种因素，计算障碍物区域、预计会和障碍物碰撞的区域以及机器人可移动区域，这被称为成本地图（costmap）。根据导航类型，成本地图又被分成两部分。一个是global_costmap，在全局移动路径规划中以整个区域为对象建立移动计划，其输出结

果就是global_costmap。而另一个被称为local_costmap，这是在局部移动路径规划中，在以机器人为中心的部分限定区域中规划移动路径时，或在躲避障碍物时用到的地图。然而，这两种成本图的表示方法是相同的，尽管它们的目的不同。

costmap用0到255之间的值来表示。数值的含义如图11-19所示，简单地说，根据该值可以知道机器人是位于可移动区域还是位于可能与障碍物碰撞的区域。每个区域的计算取决于第11.6节中指定的costmap配置参数。

- 000: 机器人可以自由移动的free area（自由区域）
- 001~127: 碰撞概率低的区域
- 128~252: 碰撞概率高的区域
- 253~254: 碰撞区域
- 255: 机器人不能移动的占用区域（occupied area）

如下所示为障碍距离与costmap值的关系



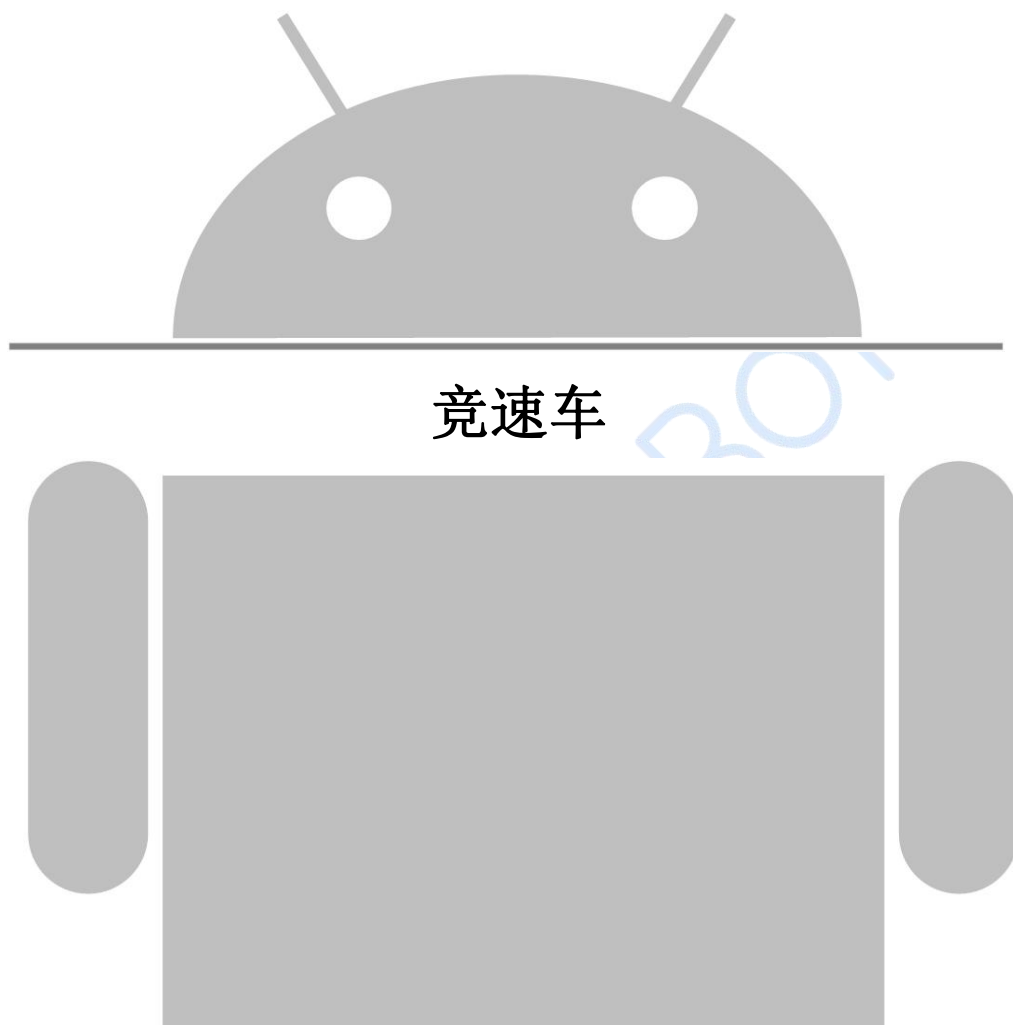
4.9 本章小结

本章我们介绍了激光SLAM的相关算法的理论知识，通过数据采集与处理、前端匹配、后端优化、回环检测最后到构建地图这几个方面，简单地叙述了SLAM的基本流程，最后还对机器人的导航和路径规划的算法进行简单的叙述。

激光SLAM现在已经被广泛试用在各个各个领域，体也同样激光SLAM的研究相对成熟，误差模型相对简单，算法上门槛相对较低，一些开源的算法已经被纳入ROS系统中成为了标配，适合初学者进行学习和开发。



Chapter 5

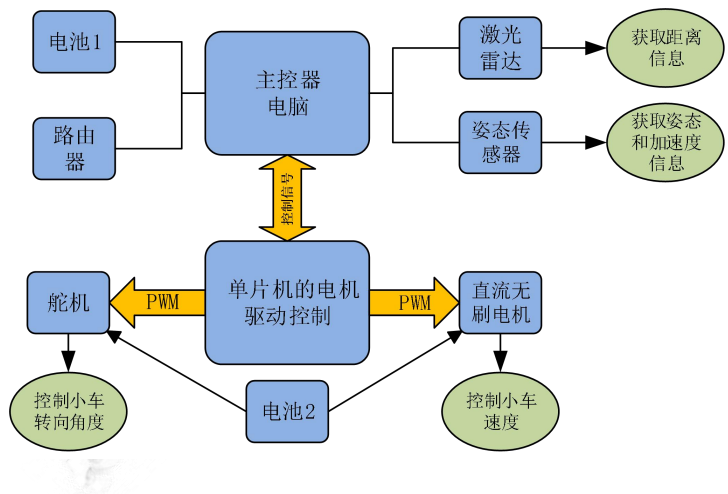


5 竞速车

ROS竞速车主要由软件和硬件两个大部分。硬件以车体结构为整体框架，以主控电脑作为控制载体，车上搭载所有需要的传感器硬件以及驱动装置；软件以ROS机器人操作系统为基础，配合SLAM构建地图和路径规划和自主导航。

5.1 硬件结构

硬件主要以主控电脑为核心，外部搭载激光雷达传感器，用来采集整车周围所有的物体相关的距离信息；同时也搭载了姿态传感器，用来获得车辆整体的姿态信息和加速度信息。使用直流无刷电机来控制车的驱动装置；使用舵机用来控制车辆整体的转向装置，通过单片机作为下位机的控制器，单片机通过接收电脑传输的控制信号，从而进一步再对直流无刷电机和舵机进行控制。由于车辆并不配有屏幕，所以车上搭载了路由器，方便操作人员进行远程控制的工作。



➤ 激光雷达

选用镭神LS01G三角法激光雷达，测量角度为 360° ，最大测量范围 ≥ 8 米，测量频率3600~4000Hz，测量精度1m以内为毫米级，1m以上为实际距离的1%，角度分辨率为 1° ，扫描频率3~11Hz。

➤ 姿态传感器

使用ART-IMU-02A姿态传感器，姿态角测量范围姿态角：测量范围(pitch/roll): $\pm 90/\pm 180$ 度，动态精度：0.5度，分辨率：0.1度；航向角(yaw): ± 180 度，动态精度：2 (RMS)

分辨率：0.1度；陀螺仪：测量范围(pitch/roll/yaw): ± 1000 度/s，零偏稳定性：50度/h 非线性度：0.2%FS，加速度计：三轴测量范围: $\pm 2g$ ，零偏稳定性:5mg，非线性度:0.5%FS；磁力计：三轴测量范围: $\pm 12Guass$ ，分辨率:0.003Guass，非线性度:0.1%FS；气压计：高度分辨率:1cm，测量范围:10~1200mbar。

➤ 直流无刷电机

使用ART-M1-K2150直流无刷电机，kv值为2150，功率为2400w，最高电压19V，最高转速45000rpm。

➤ 舵机

使用ART-SG995舵机，工作频率为1520 μs /330Hz，工作电压为4.8V~6.0V。

5.2 软件框架

软件主要由以ROS机器人操作系统为框架主体，在ROS机器人操作系统上，添加上激光雷达和IMU的姿态传感器的驱动程序，这样，我们就可以获得相应的信息周围环境的信息和车整体的姿态信息；通过无刷直流电机和舵机控制电机的移动，通过移动，我们可以采集到整体的环境信息。将这些采集到的数据进行处理，通过SLAM的算法对整个环境的地图的构建，这样我们就可以获得整体环境的地图，再通过路径规划和自主导航，就可以控制车到达想要到的相应的位置上。在软件上，我们使用ssh远程访问访问，用以控制远程控制整个系统。

